

ADL Lite: An Event-First Capability-Lifecycle Registry for LLM Agent Ecosystems

Anonymous Authors
Affiliation withheld for peer review

Abstract

LLM agents are proliferating across enterprise and open ecosystems, each exposing tools, APIs, and knowledge-domain capabilities. Yet no lightweight, tamper-evident, lifecycle-aware registry exists to govern what a capability claims, how it evolves, or whether it remains trustworthy. Existing governance layers—KYA for permissions and AgentSafe for architecture-level oversight—do not address this gap. ADL Lite fills it as an event-first capability-lifecycle registry.

Events are the fundamental architectural primitive, distinguished from their immutable records—EventChains. Capabilities are modeled as append-only, cryptographically linked EventChains, with all lifecycle state (status, confidence, and validators) derived exclusively from event history through deterministic functions. Every capability claim, validation, deprecation, and fork is an auditable event, eliminating stored mutable state and the consistency defects it introduces.

We present a two-level ontological analysis that situates ADL Lite within foundational ontologies (BFO, DOLCE, and UFO), formalizes identity conditions and three forms of ontological dependence, and defines a decidable precondition language for lifecycle transitions. We further present a complete formal semantics, together with seven core proven properties (including determinism, fork determinism, confidence boundedness, and well-formedness preservation) and an optional CRDT convergence property (Theorem 9) scoped to a future distributed-merge strategy.

We scope all governance claims in this paper to the collaborative non-Byzantine trust model: honest-but-curious agents who detect inconsistencies, not Byzantine agents who actively deceive. The SHA-256 chain provides tamper-evidence, not tamper-proofing or non-repudiation.

We verified the architecture of the registry on 201 EventChains derived from the IBM Anti-Money Laundering dataset (9,300 events), recording zero integrity failures and confirming linear-time verification under architectural stress tests. We supplemented these findings with concurrency stress tests, adversarial integrity trials, and comparative baselines against nanopublications and Git-native systems. This paper establishes the ****ontological and formal foundations**** of capability-lifecycle governance. Domain-level evaluation with human experts is deferred to future work. We implement the registry as a pip-installable Python toolkit operating on Markdown files in standard Git repositories, targeting lightweight deployment scenarios where full OWL/SPARQL infrastructure is unavailable or unnecessary.

Keywords: agent governance; capability registry; lifecycle governance; event sourcing; multi-agent systems; provenance; operational ontology

1 Introduction

1.1 Background: The Agent Capability Governance Gap

Large language model (LLM) agents are proliferating across ecosystems. Each agent exposes a set of *capabilities*: tools, APIs, knowledge domains, and interaction protocols. A capability is a

claim—"I can retrieve weather data," "I can execute SQL," "I can reason about financial risk"—that other agents and human operators must evaluate before delegation. However, no existing registry records what a capability does, how it evolves, or whether it is trustworthy. Today, capability claims are scattered across README files, API documentation, and prompt templates. They are revised without audit trails, deprecated without formal process, and forked without attribution. A lightweight, tamper-evident, lifecycle-aware registry of agent capabilities does not yet exist.

Existing governance layers. Three recent lines of work address adjacent aspects of agent governance, but none fills the capability-registry gap. **KYA** (*Know Your Agent*) provides trust-layer permissions with HMAC-chained integrity but does not govern capability lifecycles (no **VALIDATE/DEPRECATE/FORK** events) Quadri [2026].

AgentSafe provides architecture-level governance but is heavyweight and lacks Markdown-native authoring Khan et al. [2025].

LLM-native OE (Talukder et al.) produces static ontologies without lifecycle governance; **Agent Traces** notes that a unified trace schema is still needed; **SafeAgent** governs safety assessment, not capability registration Talukder et al. [2026], Wang et al. [2026], Liu et al. [2026]. Runtime safety systems (SafeHarness Lin et al. [2026], OpenClaw PRISM Li [2026]) protect execution-time safety but do not register capabilities.

Defining the gap. No existing system combines (a) Markdown-native authoring, (b) cryptographic hash chaining, (c) lifecycle state machines, and (d) pip-installable deployment.

Defining “operational ontology.” We use the term to mean an ontology in which lifecycle-transition semantics—preconditions, state derivation, and consensus rules—are co-located with the EventChain data structure in a single lightweight package. These semantics are enforced by the ActionExecutor without external workflow engines, OWL reasoners, or triple stores. This distinguishes our usage from OBO Foundry practice (centralized expert curation, OWL 2 DL reasoners, external release pipelines) Smith et al. [2007] and from executable ontologies (operational constraints enforced by external workflow engines, separate from the data structure) Guizzardi et al. [2024]. ADL Lite inverts this separation: the ontology *is* the executable structure, and the ActionExecutor enforces constraints directly over the event sequence without external dependencies.

Ontological novelty of the event-first stance. ADL Lite’s contribution is not the philosophical thesis that “events are fundamental”—a position already established by Wittgenstein, BFO, DOLCE, and UFO Wittgenstein [1922], Arp et al. [2015], Gangemi et al. [2002], Guizzardi [2005]—but its *operationalization*. Specifically, it translates an event-first commitment into a deployable, Markdown-native, cryptographically enhanced capability-lifecycle mechanism. This distinguishes our pragmatic pluralism from occurrence-only ontologies Al-Fedaghi [2024] while retaining **Concept** as a generically dependent continuant.

Why existing families are insufficient. Descriptive event ontologies (CIDOC CRM, SEO) record what happened without cryptographic integrity or lifecycle preconditions ICOM-CIDOC [2014], Boldachev [2025]. Cryptographic provenance systems (PROV-O, nanopublications) provide tamper detection but lack Markdown-native authoring and state machines W3C [2013], Kuhn et al. [2015], Kuhn and Dumontier [2015]. Markdown-native tools (Foam, Obsidian) satisfy authoring and deployment but lack cryptographic chaining and lifecycle governance Foam Contributors [2020], Obsidian Team [2024]. No family satisfies all four; ADL Lite occupies this intersection.

1.2 Research Gap: Capability Lifecycle Governance as Ontological Analysis

The specific research gap is: *How can an event-first capability registry achieve lifecycle traceability and multi-agent governance without requiring object-first mutable state, expert curation, or heavyweight tooling?* The engineering dimension concerns deployment friction: existing platforms require specialized expertise and infrastructure that do not scale with agent-generated artifacts, and LLM agents lack programmatic access to proprietary ontology interfaces. The ontological dimension concerns the status–history relationship: in an event-first registry, status is a derivative of history, not a property of the record. No prior work has systematically studied the operationalization of event-first semantics—combining cryptographic integrity, deterministic lifecycle derivation, and declarative preconditions in a single lightweight system. This paper addresses both dimensions by presenting ADL Lite, an event-first operational ontology in which capabilities are modeled as append-only, cryptographically linked EventChains, and all lifecycle state is derived exclusively from event history. The primary contribution is an **ontological analysis** that situates ADL Lite within foundational ontologies (BFO, DOLCE, UFO), formalizes its categories and identity conditions, and proves key properties of its derivation semantics.

1.3 Contributions

The contributions of this paper are organized around three axes: ontological analysis, formal semantics, and architectural verification.

Contribution 1: Ontological analysis of event-first capability governance. We analyze ADL Lite’s core categories through the lens of three influential foundational ontologies:

- We show that ADL Lite’s **Event** corresponds to the BFO category of *occurrent* and to the DOLCE category of *perdurant*.
- We argue that ADL Lite’s **Concept** (here, a capability definition) is best understood as a BFO *generically dependent continuant* or as a DOLCE *non-physical object*.
- We demonstrate that ADL Lite’s L3 relations instantiate UFO *relators* with event-based lifecycles.
- We formalize identity conditions for events, capabilities, and chains, and analyze three forms of ontological dependence (historical, generic, mutual).

Contribution 2: Formal derivation semantics with proven properties. We provide a complete formal foundation comprising:

- An event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ partitioned into lifecycle and communication events.
- A well-formedness predicate $\text{WF}(C)$ over EventChains.
- Status derivation $\delta(C)$ and confidence aggregation $\gamma(C)$ as deterministic functions (δ in $O(n)$, γ in $O(1)$).
- Fork and fork-determinism semantics.
- Seven core proven properties: determinism (Theorem 1), fork determinism (Theorem 2), transition monotonicity (Theorem 3), confidence boundedness (Theorem 4), confidence monotonicity under non-decreasing validation (Theorem 5), status–confidence consistency (Theorem 6), and well-formedness preservation (Theorem 7). We reserve an optional CRDT convergence property (Theorem 9) for a future distributed-merge strategy.

- An analysis of expressive power in relation to Event Calculus and Description Logic.

Contribution 3: Architectural verification of the capability registry implementation.

We evaluated ADL Lite on 201 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset (9,300 events). We verified the architecture of the implementation against the seven core proven theorems from Contribution 2. The evaluation confirmed:

- Zero integrity failures across all chains, including the longest chain (300 events).
- 100% accuracy on 2,204 exhaustive status derivation test cases.
- Perfect precision and recall ($P = R = F1 = 1.0$) on 141 precondition enforcement test cases (19 base + 72 boundary + 50 concurrency).
- Linear scalability: 20,955 events per second on modest hardware.

These results are not a domain-level evaluation of AML effectiveness. Instead, they validate that the event-first ontological architecture—including the deterministic derivation semantics, hash-linked integrity guarantees, and seven formally proven properties—is computationally feasible, cryptographically sound, and verified. Multi-agent collaboration simulation (E17) is already operational. Domain-level evaluation with human experts (E5) is planned for future work (expert recruitment and scoring protocol completed). The present paper establishes the **ontological and formal foundations** of capability-lifecycle governance.

1.4 Implementation Status and Roadmap

ADL Lite is implemented as a pip-installable Python package (v0.4.0-alpha in Git, v0.2.0 on PyPI) operating on Markdown files in standard Git repositories. The following capabilities are already implemented and validated; the remainder are planned for future releases.

Implemented and validated. The core EventChain data structure (append-only, SHA-256 hash-linked events), deterministic derivation semantics (δ and γ), and the decidable precondition language (8 comparators, closed action registry) are fully implemented and validated by the experiments reported in this paper (E1–E4, E6, E13–E16, E20–E23). The L1–L4 document model, consensus engine (fork, deprecate, validate transitions), and integrity verification (VerifyIntegrity) are also production-ready. The trust model for these experiments is collaborative audit: agents are assumed non-Byzantine, actor identity is a self-declared string identifier, and fork resolution uses CRDT lattice semantics (LUB for status, G-Counter for confidence).

Implemented but not exercised in experiments. Additional features are present in the current codebase but were not used in the reported experiments: L2 Markdown template validation with Pydantic-governed sections ([Observation], [Reasoning], [Conclusion]); ARCHIVE events with cold storage migration; per-actor linear confidence calibration (`calibration.py`); and an Ed25519 key registry with Git commit signature binding as a pre-implementation for future authentication.

Planned for future releases. W3C Linked Data Proofs (DIDs, Merkle-tree batch verification) will replace string identifiers for cryptographic authentication. An optional BFT transport layer will provide Byzantine fault tolerance. We plan domain-level evaluation with AML experts (E5: expert recruitment and protocol design completed) and machine-checked proofs (TLA⁺ or Coq). We also plan OWL 2 DL axiomatization, SHACL validation, and adapter modules for emerging agent registry standards (MCP, A2A, AGNTCY).

We scope the governance claims and empirical validation in this paper to the collaborative-audit trust model (non-Byzantine agents, self-declared string identifiers). We plan to implement more advanced authentication and adversarial robustness features in future releases.

1.5 Paper Organization

Section 2 positions ADL Lite within the emerging agent governance landscape (KYA, AgentSafe, Agent Traces to Trust, SafeAgent, SafeHarness, and OpenClaw PRISM) and surveys related work in foundational ontologies, ontology engineering, and event-centric modeling. Section 3 presents the core ontological analysis: categories, identity conditions, ontological dependence, and alignment with BFO, DOLCE, and UFO. Section 4 describes the ADL Lite architecture, formal semantics, and comparison with Event Calculus and Description Logic. Section 5 reports architectural verification of correctness. Section 6 discusses implications, limitations, and future work. Section 7 concludes.

2 Related Work

2.1 Agent Governance Landscape

LLM agent proliferation has created an urgent need for governance beyond traditional ontology engineering. ADL Lite fills a gap that trust, safety, and provenance layers overlook: a lightweight, event-sourced, tamper-evident registry of agent capabilities and their lifecycle states.

AgentHub Pautsch et al. [2025] proposes a registry layer for agent sharing with discovery, workflow integration, and lifecycle transparency, but it remains a research agenda without an implemented lifecycle state machine or deterministic derivation semantics. ADL Lite goes beyond AgentHub’s agenda by offering an implemented registry with formal lifecycle semantics (register $\rightarrow$ validate $\rightarrow$ deprecate), cryptographic hash chaining, and deterministic status derivation.

KYA Quadri [2026] verifies agent identity via HMAC chains but does not govern capability lifecycles. **AgentSafe** Khan et al. [2025] offers comprehensive architecture governance but requires heavyweight infrastructure and lacks Markdown-native authoring. **From Agent Traces to Trust** Wang et al. [2026] concludes that “a unified trace schema is still needed”—ADL Lite provides exactly that. **Talukdar et al.** Talukder et al. [2026] demonstrate multi-agent ontology engineering but produce static artifacts without lifecycle governance. **SafeAgent** Liu et al. [2026] operates at the interaction layer rather than the artifact layer. Runtime safety systems (SafeHarness Lin et al. [2026], OpenClaw PRISM Li [2026]) and runtime infrastructure (Institutional AI Syrnikov et al. [2026], GaaS Gaurav et al. [2025], COMPASS Dessureault et al. [2026], UALM Prakash et al. [2026]) consume capability provenance for policy decisions; ADL Lite occupies the *registry layer* between permissioning and runtime enforcement.

Table 1 compares these systems.

Runtime safety systems (SafeHarness Lin et al. [2026], OpenClaw PRISM Li [2026]) govern execution-time safety rather than capability registration; they complement ADL Lite by leveraging the capability metadata that ADL Lite produces.

2.2 Foundational Ontologies and Event-First Semantics

ADL Lite’s **Event** corresponds to BFO’s *process* Arp et al. [2015], DOLCE’s *perdurant* Gangemi et al. [2002], and the event/action distinction in UFO Guizzardi [2005]. ADL Lite adopts *pragmatic pluralism*: BFO as conceptual guide, DOLCE as design principle, and UFO as modeling framework. A detailed cross-mapping appears in Section 3 (Table 5).

We next review knowledge graph construction and ontology engineering systems that provide the broader tooling context for ADL Lite’s document-native approach.

Table 1: Comparison of agent governance systems.

Dimension	KYA	AgentSafe	Agent Traces	Talukdar et al.	SafeAgent	ADL Lite
Governance Scope	Trust / per- missions	Architecture safety	Provenance / trace	Ontology pro- duction	Protocol safety	Capability lifecycle
Lifecycle Aware- ness	None	Design-time only	None	None	None	Full (register →validate → deprecate)
Deployment Weight	Lightweight	Heavyweight	Survey / framework	Research pipeline	Lightweight	Very lightweight (pip + Git)
Tamper- evidence	HMAC chain	Audit logs	Trace schema (proposed)	Manual QA review	Sandbox checks	SHA-256 hash chain + pre- conditions
Agent-Native	Yes (15+ adapters)	Yes (design- time)	Yes (prove- nance)	Yes (multi- agent)	Yes (runtime)	Yes (Markdown- native, multi- agent)

2.3 Knowledge Graph Construction and Ontology Engineering

The Semantic Web vision Berners-Lee et al. [2001] was operationalized through OWL 2 W3C OWL Working Group [2012]; toolchains enable ontology authoring at scale, but reasoners impose computational costs ill-suited to rapidly evolving knowledge bases Glimm et al. [2014], Hogan et al. [2021]. The OBO Foundry provides centralized versioning Smith et al. [2007]; lightweight alternatives (Schema.org Guha et al. [2016], JSON-LD Sporny et al. [2020], SHACL W3C [2017]) and Markdown tools Foam Contributors [2020], Lin [2020] lower the barrier to entry but remain bound to RDF or lack machine-checkable coordination. LLM4ACOE Soularidis et al. [2025], Sim-HCOME Doumanas et al. [2025], and Ontology-Constrained Neural Reasoning Tuan and Sanyal [2026] generate OWL automatically but without lifecycle governance. ADL Lite differs from these systems in three respects: (i) *authoring paradigm*: ADL Lite uses Markdown-native documents rather than OWL/RDF output; (ii) *governance scope*: ADL Lite provides built-in lifecycle state machines (register → validate → deprecate) rather than static ontology production; (iii) *deployment weight*: ADL Lite is pip-installable and operates on Git repositories, whereas LLM4ACOE and Sim-HCOME require multi-agent orchestration pipelines. These are complementary trade-offs, not competitive disadvantages: LLM4ACOE/Sim-HCOME excel at automated ontology generation with ReAct reasoning traces, while ADL Lite excels at lightweight, auditable governance of existing capabilities. Table 2 presents the dimension-level comparison.

Blocklace Almeida and Shapiro [2024] is a general-purpose BFT-CRDT with a cryptographic hash DAG, providing Byzantine fault tolerance and equivocation detection at the transport layer. ADL Lite is complementary: Blocklace provides the *transport* (distributed consensus over a DAG of events), while ADL Lite provides the *application semantics* (deterministic status derivation $\delta(C)$, confidence aggregation $\gamma(C)$, and declarative preconditions). In the proposed Phase 3 architecture, Blocklace’s DAG serves as the transport layer beneath ADL Lite’s lifecycle semantics. SEO 2025 Boldachev [2025] uses causal DAGs with reactive guards for distributed multi-agent event contribution. ADL Lite’s linear hash chain is less expressive but offers $O(1)$ per-event append latency and deterministic $O(n)$ verification. This represents a deliberate trade-off between expressivity and simplicity of audit.

Table 2: Dimension-level comparison: ADL Lite vs. proximate systems.

Dimension	LLM4ACOE / HCOME	Sim-Blocklace (2024)	ADL Lite
Primary output	OWL 2 DL ontology	Generic BFT-CRDT DAG	Markdown-native capability registry
LLM agent role	Multi-agent ReAct ontology generation	None (transport layer)	Actor in lifecycle events
Lifecycle governance	None (static ontology)	None (application-agnostic)	Register $\rightarrow$ validate $\rightarrow$ deprecate
Cryptographic integrity	None (manual QA review)	SHA-256 + BFT consensus	SHA-256 hash chain
Deterministic state	External reasoner required	None (raw data structure)	Built-in $\delta(C)$, $\gamma(C)$
Deployment	Research pipeline (multi-agent)	Library (distributed systems)	<code>pip install</code> + Git

2.4 Provenance, Trust, and Verifiable Publishing

W3C PROV-O W3C [2013], Linked Data Proofs W3C Verifiable Credentials Working Group [2024], and RDF-star W3C RDF-star Working Group [2024] provide provenance interchange and statement-level annotation but lack both cryptographic integrity and lifecycle governance. Git-native systems (RO-Crate Soiland-Reyes et al. [2022], DataLad Halchenko et al. [2021], TerminusDB TerminusDB/DFRNT [2024]) share immutable history but lack document-native authoring. Full mappings to standards and loss analysis are in Appendix F.

2.5 Event-Centric Ontologies

CIDOC CRM ICOM-CIDOC [2014], SEM/LODE van Hage et al. [2011], Shaw et al. [2011], and RDF Stream Processing W3C RDF Stream Processing Community Group [2013], Barbieri et al. [2009], Le-Phuoc et al. [2011] provide event vocabularies but lack both constraint mechanisms and retrospective verification. Boldachev’s SEO Boldachev [2025], Al-Fedaghi’s occurrence-only paradigm Al-Fedaghi [2024], and Guizzardi’s UFO-B Guizzardi et al. [2024] offer alternative event models. CRDTs Shapiro et al. [2011], Martin et al. [2020], Nicolai and Dadam [2021], Gruss et al. [2023], Nieto et al. [2024] and Blocklace Almeida and Shapiro [2024] reconcile concurrent updates; ADL Lite complements these with append-only event chains and deterministic lifecycle governance. Real-time collaborative ontology editing using distributed version control (OntoEditor, which uses Operational Transformation rather than CRDTs) provides a related but distinct approach to multi-author ontology development Hemid et al. [2024]. ADL Lite’s EventChain extends the event-sourced aggregate root pattern Young [2010] with ontological analysis, SHA-256 chaining, and Markdown-native authoring.

2.6 Transparency Logs and Software Supply Chain

Certificate Transparency (CT) Laurie et al. [2013], Sigstore Rekor Denker et al. [2022], in-toto Torres-Arias et al. [2019], and SLSA Team [2023] provide tamper-evident registries for software artifacts, certificates, and supply-chain provenance. These systems use Merkle-tree or hash-DAG structures to enable $O(\log n)$ inclusion proofs and $O(1)$ consistency proofs between log snapshots, making them highly scalable for globally distributed verification with millions of entries. ADL Lite’s linear SHA-256 chain, by contrast, requires $O(n)$ sequential verification for full integrity checking. This represents a deliberate design trade-off: linear chains are simpler to implement in

Markdown-native environments, produce human-readable audit trails (each event is a self-contained JSON record), and require no external gossiping or witness infrastructure for consistency monitoring. Merkle trees excel at proving that an entry exists in a large, append-only log; ADL Lite excels at governing the lifecycle of a single capability through deterministic status derivation and confidence aggregation. The two approaches are complementary: future Phase 3 integration (FW4, FW9) may adopt Merkle-tree batching for cross-repository verification while retaining ADL Lite’s per-chain lifecycle semantics.

Quantitative comparison with nanopublications and Git-native provenance. Table 3 compares ADL Lite with two related lightweight approaches on operational metrics relevant to capability-lifecycle governance.

Table 3: Quantitative baseline comparison: authoring burden, validation latency, and audit depth.

System	Authoring burden	Validation latency	Audit depth
Nanopublications + trusty URIs	Medium (RDF/Turtle syntax)	$O(1)$ hash verification	Statement-level (single assertion)
Git-native (RO-Crate, DataLad)	Low (file-based)	$O(1)$ commit signature check	Commit-level (coarse- grained)
ADL Lite	Low (Markdown- native)	$O(n)$ chain scan (lin- ear)	Event-level (fine- grained lifecycle)

Nanopublications with trustworthy URIs Groth et al. [2023] provide $O(1)$ hash verification for individual assertions but lack lifecycle semantics: each nanopublication is a standalone, immutable statement with no formal status derivation or precondition-governed transitions. Git-native systems (RO-Crate, DataLad) provide commit-level provenance but treat the entire commit as a unit of change, making it impossible to trace the lifecycle of an individual capability within a repository containing many concepts. ADL Lite trades $O(1)$ verification for $O(n)$ chain scanning in exchange for per-event lifecycle governance: each event (REGISTER, VALIDATE, DEPRECATE, FORK) is individually typed, preconditioned, and auditable. The migration path to nanopublications is bidirectional: ADL Lite events export to RDF-star quoted triples (Appendix F), and nanopublication assertions can be imported as RELATE events with SHA-256 content-addressed references. The migration path to Git-native provenance is direct: ADL Lite EventChains are stored as Markdown files in standard Git repositories, with commit signatures providing repository-level integrity and event hashes providing concept-level integrity.

2.7 Positioning

Palantir Foundry unifies “semantic elements” (nouns) with “kinetic elements” (verbs) Palantir Technologies [2024], but requires enterprise-scale deployment and proprietary tooling, and lacks LLM-native authorship. ADL Lite offers pip-installable deployment, Markdown-native authoring, and multi-agent consensus. Full comparison tables are in Appendix F.

Within the open-source, pip-installable, Markdown-native tool space, ADL Lite uniquely combines four properties: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. This enables multiple LLM agents to propose, challenge, and refine concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that renders decentralized authorship accountable.

Table 4: Comparison of event-centric frameworks: topology, integrity, and governance.

Framework	Event Topology	Integrity Mechanism	Governance Semantics
CIDOC CRM	Descriptive event graph	None (documentation)	None
SEM/LODE	Minimal event vocabulary	None (publishing)	None
SEO (2025)	Causal DAG ($\prec$ relation)	Conformance tests (A1–A9)	Reactive guards
Blocklace (2024)	Cryptographic hash DAG	SHA-256 + BFT consensus	Byzantine exclusion
Transparency logs	Merkle tree / hash DAG	SHA-256 + inclusion proofs	Third-party audit
ADL Lite	Linear SHA-256 chain	Sequential hash verification	Precondition rules + $\delta(C)$

3 Ontological Analysis of ADL Lite

3.1 Philosophical Foundation: Event-First as Ontological Commitment

ADL Lite inverts the object-first stance of conventional operational ontologies Palantir Technologies [2024], Lin [2020]. Drawing on Wittgenstein’s *Tractatus* §1.1—“*Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge*” Wittgenstein [1922], Young [2010]—the system treats the occurrent as a fundamental primitive. This yields three governing principles: (1) **No implicit state changes.** Every mutation is an explicit event. (2) **Events are occurrents.** They happen, but do not endure Arp et al. [2015], Gangemi et al. [2002]. (3) **All derived properties are historical.** Status, confidence, and validators are computed by deterministic functions over the event sequence and are never stored.

3.2 Categories and Taxonomy

We analyze ADL Lite’s categories with reference to BFO Arp et al. [2015], DOLCE Gangemi et al. [2002], and UFO Guizzardi [2005]. Table 5 presents the cross-mapping.

3.2.1 Event as Occurrent/Perdurant

In BFO, an *occurrent* unfolds in time and is ontologically distinct from *continuants* Arp et al. [2015]; in DOLCE, the corresponding category is *perdurant* Gangemi et al. [2002]. ADL Lite’s **Event** is both: a temporal entity with a definite beginning and end, no spatial location, and no endurance. The serialized record is an information content entity (ICE), not the event itself. In UFO, **REGISTER** and **VALIDATE** are *actions* (intentional), whereas **RELATE**, **EVIDENCE**, and **SEAL** are *events* (structural changes) Guizzardi [2005].

3.2.2 Concept as Generically Dependent Continuant

An ADL Lite concept is a *generically dependent continuant* (GDC) in BFO terms Arp et al. [2015]: it depends on some continuant for its existence, but not on any *specific* one. It depends on its EventChain *as an ICE*—the serialized chain record borne by the Markdown file—rather than on the

Table 5: Cross-mapping of ADL Lite categories to upper ontologies. Entries marked “—” indicate intentional non-alignment; those marked “ $\approx$ ” denote approximate correspondence.

ADL Lite	BFO 2.0	DOLCE	UFO	Note
Event	occurrent	perdurant	Event	Temporal entity; happens; no endurance
EventChainprocess		accomplishment $\approx$	Complex Event	Ordered event sequence; <i>process</i> layer (see record layer below)
EventChain-information record	content entity	information object	Information Entity	Serialized ICE; <i>two-level account</i> in §3.2.6
Concept	generically dependent continuant	non-physical object $\approx$	Conceptual Entity	Depends on EventChain-record as ICE bearer (not occurrent)
Relation (L3)	relational quality	quality	Relator	Mediates between concepts
Action (L4)	planned process	action	Action	Intentional occurrent with preconditions
Actor	agent	physical agent $\approx$	Agent	Can be human or software agent; “physical” is approximate for LLMs
Payload	information content entity	information object	Information Entity	Structured data carried by event
Hash	generically dependent continuant	—	—	Cryptographic identifier; no physical bearer

occurrent process itself.¹ The concept’s identity is determined by the *genesis event* (the first event in the chain), not by the totality of events—analogous to a poem, which depends on some physical copy for its existence but not on any specific one. In DOLCE, the closest category is *non-physical object* Gangemi et al. [2002]; in UFO, concepts are *conceptual entities* Guizzardi [2005].

3.2.3 EventChain as Process

An EventChain is a *process* in BFO: a complex occurrent with temporal parts (the individual events) that unfolds over time Arp et al. [2015]. It is not a continuant because it does not exist in full at any moment. It is individuated by the ordered sequence of events and their cryptographic linkage (see Section 3.2.6 for the full two-level account, which distinguishes the process from its record).

3.2.4 Actor as Agent

ADL Lite’s Actor is mapped to BFO **agent** and UFO **Agent**. The DOLCE mapping to **physical agent** is approximate: DOLCE lacks a dedicated “software agent” category, whereas UFO’s **Agent** subsumes both human and software agents (including LLMs). Future work (FW1) may define a DOLCE extension for non-physical agents.

3.2.5 Relation as Relational Quality / Relator

ADL Lite’s L3 relation blocks assert typed edges between concepts. In BFO, these are *relational qualities*: qualities that inhere in one entity and bear a relation to another Arp et al. [2015]. ADL Lite’s L3 relation block is inscribed in the source concept’s Markdown file (the bearer), while the relation mediates between two concepts. This reading is consistent with BFO relational qualities, which need not be symmetric. In UFO, the same relations are *relators*: entities that mediate between relata and are brought into existence and destroyed by events Guizzardi [2005]. The dual reading—BFO relational quality for the record, UFO relator for the mediation—reflects the two-level account of Section 3.2.6.

Creation. A RELATE event e establishes a relator r at time $t = e.\text{timestamp}$:

$$\text{HoldsAt}(\text{Relation}(c_1, c_2, p), t) \iff \exists e \in C. e.\text{type} = \text{RELATE} \wedge e.\text{timestamp} \leq t \wedge \neg \text{Revoked}(c_1, c_2, p, t) \quad (1)$$

where $\text{Revoked}(c_1, c_2, p, t)$ holds if and only if there exists a revocation event e' with $e'.\text{timestamp} \leq t$.

Revocation. A relator is terminated by an explicit REVOKE event e_{rev} :

$$\text{Revoked}(c_1, c_2, p, t) \iff \exists e_{\text{rev}} \in C. e_{\text{rev}}.\text{type} = \text{REVOKE} \wedge e_{\text{rev}}.\text{timestamp} \leq t \wedge e_{\text{rev}}.\text{payload.target_concept_id} = c_2 \wedge e_{\text{rev}}.\text{payload.predicate} = p \quad (2)$$

Note on revocation semantics. ADL Lite supports two distinct semantics for relation termination. *Cessation semantics* (Equation 2): a dedicated REVOKE event explicitly terminates the relator. The relator ceases to exist as a mediator; the record (the L3 block and the REVOKE event itself) remains as an auditable ICE. *Epistemic weakening* (alternative): a RELATE event with **confidence**=0 weakens the relation to zero belief without formally terminating it. The relator continues to exist as an ICE but its mediation function is suspended. This allows graded weakening

¹In BFO and IAO Smith et al. [2010], the dependence chain is: concept $\xrightarrow{\text{GDC}}$ ICE(EventChain-record) $\xrightarrow{\text{concretized_by}}$ Markdown_file. A GDC cannot depend on an occurrent, but can depend on the ICE that documents it Arp et al. [2015], Fine [1995].

(**confidence**=0.3 means “partially weakened”). Importantly, *HoldsAt* does not depend on confidence: it checks only whether a **REVOKE** event exists (Equation 2). Confidence is epistemic strength, not an existence condition. We recommend *cessation semantics* as the default because it preserves the clean separation between existential status (the relator holds or not) and epistemic strength (how confident we are). The **REVOKE** event is a communication event (not in Σ_{life}), so it does not affect $\delta(C)$; Theorems 1–6 are unchanged. Epistemic weakening is retained for backward compatibility and for use cases requiring graded belief revision.

Why HoldsAt is independent of confidence. The ‘HoldsAt’ predicate (Equation 1) checks only whether a **REVOKE** event exists in the chain, not the confidence of any **RELATE** event. This separation reflects the ontological distinction between *existence* (does the relator hold at time t ?) and *epistemic strength* (how confident are we in the relation?). Confidence is a payload field of **RELATE** events that informs downstream consumers (e.g., a query engine may rank relations by confidence), but it does not determine whether the relation exists. A relation with confidence 0.3 still holds; a relation with a **REVOKE** event does not hold, regardless of the confidence of the original **RELATE**. This design keeps existential conditions binary and epistemic conditions graded, avoiding the conflation that motivated the **REVOKE** event type (see above).

Optional thresholded variant. For systems that prefer belief-graded relation existence, a thresholded variant HoldsAt_θ can be defined as $\text{HoldsAt}(c_1, c_2, p, t) \wedge \exists e \in C. e.\text{type} = \text{RELATE} \wedge e.\text{payload.confidence} \geq \theta$. This is a consumer-side filter, not a core ontology axiom, and does not affect the event-first derivation semantics.

$$\begin{aligned} \text{HoldsAt}(\text{Relation}(c_1, c_2, p), t) \rightarrow & \text{Exists}(c_1, t) \wedge \text{Exists}(c_2, t) \wedge \neg \text{Deprecated}(c_1, t) \wedge \\ & \neg \text{Deprecated}(c_2, t) \wedge \neg \text{Archived}(c_1, t) \wedge \neg \text{Archived}(c_2, t) \quad (3) \end{aligned}$$

where $\text{Deprecated}(c, t)$ and $\text{Archived}(c, t)$ are derived from the event chain of c at time t .

Temporal scoping. Every relator has a temporal extent $[t_{\text{create}}, t_{\text{revoke}})$. The relator is a continuant that exists only during this interval; what endures beyond it is the record (the L3 block), an ICE documenting that the relation held.

UFO instantiation. This exemplifies UFO’s relator theory Guizzardi [2005]: (i) existential dependence on both relata, (ii) event-based creation, (iii) event-based destruction, and (iv) mediation between entities rather than a property of either relatum alone. DOLCE’s *quality* category includes both intrinsic and relational qualities Gangemi et al. [2002]; ADL Lite relations are relational qualities in this sense.

3.2.6 Two-Level Ontological Account: Occurrents versus Records

The term “EventChain” is ambiguous between a process (events unfolding in time) and an ICE (the serialized record). These are distinct entities with distinct identity conditions.

Level 1: Occurrents (What Happens). Event (e): an individual occurrent/perdurant with a determinate timestamp. EventChain-process (P): the complex occurrent comprising the ordered event sequence. Action (a): an intentional occurrent. Level 1 entities do not endure; at time t , only events up to t have occurred.

Level 2: Information Content Entities (What Is Recorded). EventChain-record (R): the serialized, hash-linked JSON/YAML representation—an ICE, a continuant existing in full when stored. Concept (c): the GDC depending on the record; it interprets the event sequence to project status, confidence, and validators. Relation (r): the record of a relational quality in L3 blocks, an ICE documenting a relationship that held during a specific interval.

Explicit identity axioms.

- Axiom I1: **Event identity.** An event is individuated by its UUID: $e = e' \iff e.\text{event_id} = e'.\text{event_id}$.
- Axiom I2: **Concept identity.** A concept is content-addressed by its genesis event: $c = c' \iff c.\text{genesis_hash} = c'.\text{genesis_hash}$. The `concept_id` is a human-readable alias.
- Axiom I3: **EventChain-record identity.** Two records are identical iff they have the same concept and the same events: $R = R' \iff R.\text{concept_id} = R'.\text{concept_id} \wedge \text{events}(R) = \text{events}(R')$.
- Axiom I4: **EventChain-process identity.** Two processes are identical iff they have the same events in the same order with the same timestamps: $P = P' \iff \text{events}(P) = \text{events}(P') \wedge \text{order}(P) = \text{order}(P') \wedge \text{timestamps}(P) = \text{timestamps}(P')$.

Dependence axioms.

- Axiom D1: **Existential dependence on the record (D1).** c ontologically depends on the EventChain-record R : c would not exist if R had not been created. This is a generic dependence on an ICE bearer (D2), not a dependence on the EventChain-process.²
- Axiom D2: **Generic dependence (D2).** c generically depends on the EventChain-record: it depends on an ICE bearer, but not on any specific physical copy.
- Axiom D3: **Concretization (D3).** The EventChain-record is concretized by a material bearer (file, database row, agent cache).
- Axiom D4: **Process-to-record (D4).** The EventChain-process causally produces the EventChain-record.
- Axiom D5: **No cross-level identity (D5).** No occurrent is identical to any ICE (a BFO constraint). Consequently, no GDC can ontologically depend on an occurrent; a GDC may only depend on continuants (including ICEs).

Resolving the apparent ambiguity. EventChain-process and EventChain-record are distinct (Axioms I3–I4); the process causally produces the record (D4), which bears the concept (D2). A GDC cannot ontologically depend on an occurrent (D5), but can depend on the ICE that documents it (D2–D3). The concept’s causal origin is traceable to the genesis event (the **REGISTER** occurrence), but its ontological bearer is the record. This resolves the category mistake.

3.2.7 Action as Planned Process / Intentional Event

ADL Lite’s L4 action types are *planned processes* in BFO (processes realizing a plan) Arp et al. [2015] and *actions* in UFO (intentional events performed by agents to achieve goals) Guizzardi [2005]. The precondition system captures the plan aspect: an action can only be performed if certain conditions hold (e.g., **VALIDATE** requires status **provisional**), reflecting ontological norms governing which events are permissible in which states.

²We distinguish causal origin from ontological dependence. The **REGISTER** event is the *causal origin* of c —it brought c into existence—but c ’s continued existence depends on the ICE record (D2), not on the occurrent process. This avoids the BFO constraint that GDCs cannot depend on occurrents (D5).

3.3 Identity Conditions

We formalize identity conditions in Section 3.2.6: events by UUID (Axiom I1), concepts by genesis hash (Axiom I2), and chains by concept plus ordered events (Axioms I3–I4). Fork creates a new concept (new genesis hash); deprecation and re-activation are status changes, not identity changes. We evaluate the concept’s identity through the OntoClean lens Guarino and Welty [2009].

Registry-item identity versus domain-concept identity. We distinguish two senses of “concept” in ADL Lite. *Registry-item identity* (operational) is fixed by the genesis hash; it is the identity criterion used by the EventChain data structure. Two registry entries with the same genesis hash are the same item, regardless of their content. *Domain-concept identity* (intensional) is determined by the meaning, definition, and extensional content of the capability. Two registry items (different genesis hashes) may refer to the same domain concept (e.g., “gradient-explosion” and “exploding-gradients” in Section 5.5). The genesis-hash criterion provides *registry-item* identity, not *domain-concept* identity. It is intentionally operational: the registry must distinguish items even when their content is identical (to handle independent discovery by different agents). Domain-level identity is mediated by L3 relations (e.g., *isomorphic-to*, *specialisation-of*). This is analogous to a library catalog, in which each entry has a unique ISBN (registry identity) but multiple entries may refer to the same intellectual work (domain identity).

Rigidity. A concept’s genesis hash is essential—it cannot change without the entity ceasing to exist—while its status is non-rigid. This aligns with OntoClean’s recommendation to use rigid properties as identity criteria.

3.3.1 Domain-Level Identity Alignment and Merge Conditions

While the genesis hash provides operational identity, domain-level identity (whether two chains denote the same concept) is determined by human curators or domain experts, not by the registry. ADL Lite does not support automatic chain merging: two chains with different genesis hashes are always distinct registry items. Domain-level consolidation is a governance decision recorded via L3 relations and DEPRECATE events.

Normative L3 predicates for identity alignment. We recommend the following predicates for domain-level identity consolidation, ordered from strongest to weakest claim:

- (i) **isomorphic-to:** Two concepts have identical structure and content but different genesis hashes (e.g., two independent registrations of the same laundering pattern). This is the strongest identity claim short of genesis equality.
- (ii) **specialisation-of:** One concept is a refinement or instance of another (e.g., a specific laundering technique that instantiates a general pattern).
- (iii) **fork-of:** Explicit lineage relationship (the child chain was derived from the parent via a fork operation). This is stronger than *isomorphic-to* because it records provenance, not just structural similarity.
- (iv) **related-to:** General association for concepts that are related but not identical or hierarchical.
- (v) **analogical-to:** Two concepts are similar by analogy but not structurally identical (weaker than *isomorphic-to*).

Merge conditions (manual). A domain expert may decide that two chains denote the same concept and deprecate one in favor of the other, linking them via *isomorphic-to* or *specialisation-of*.

This is recorded as a DEPRECATE event with reasoning (e.g., “Duplicate of concept-X; consolidated under `isomorphic-to` relation”), not as an automatic merge. The deprecated chain remains auditable; the consolidation is itself a governance decision subject to validation by other agents. The genesis hash of the deprecated chain is preserved in the relation, so the full provenance of the consolidation decision is recoverable.

Provenance-addressing vs. content-addressing. The genesis hash is an *operational (provenance) identifier*, not a *content-addressed hash* (e.g., Merkle DAG or IPFS CID). The genesis hash includes the `event_id` (random UUID), `timestamp`, and `canon_version`, in addition to the payload. Two independent REGISTER events with byte-identical payloads will therefore have *different genesis hashes* (different UUIDs and timestamps). This is a deliberate design choice: the genesis hash guarantees *provenance uniqueness* (each registration is a distinct event with its own actor, timestamp, and UUID), not *semantic equivalence*. Content-addressing would enable automatic deduplication but would lose the provenance information needed to distinguish independent registrations by different actors. Domain-level consolidation of identical concepts is the responsibility of human curators, who must explicitly assert a relation via L3 predicates (as described above).

Unity. A concept is a unified whole individuated by its genesis hash; the `EventChain`, by contrast, is a mereological sum of events with no unified whole beyond sequential aggregation Guarino and Welty [2009].

Dependence. The `Concept` is generically dependent on the `EventChain` record: it depends on an ICE bearer for its existence, but not on any specific physical copy Arp et al. [2015], Fine [1995]. The genesis hash is an essential (rigid) identity criterion: it cannot change without the entity ceasing to exist. However, the existence dependence itself is generic, because the concept can be concretized in multiple copies.

3.4 Ontological Dependence

ADL Lite exhibits three forms of dependence.

3.4.1 Identity and Dependence Conditions

The concept is a GDC depending on the `EventChain` record:

$$\text{Concept} \xrightarrow{\text{GDC}} \text{ICE}(\text{EventChain-record}) \xrightarrow{\text{concretized_by}} \text{Markdown_file}$$

A concept depends on the information artifact (the serialized `EventChain` record) rather than on the occurrent process of events. This is consistent with BFO’s constraint that GDCs cannot depend on occurrents. Its identity is fixed by the genesis event. As a GDC, it can be concretized in multiple copies (Git repositories, local files, and agent caches) without loss of identity. If all copies are lost but later restored from backup, the concept persists because the genesis hash is unchanged. A concept ceases to exist only when all concretizations are irreversibly destroyed *and* no agent retains information to reconstruct the chain. This high bar reflects the durability of distributed, hash-addressed systems.

3.4.2 Causal Origin and Ontological Dependence of the Concept

A concept *c* *causally originates from* the `EventChain`-process *C* that generated it: the genesis REGISTER event brought *c* into existence. However, *c* *ontologically depends* only on the `EventChain`-record *R* (an ICE), not on the `EventChain`-process *C* (an occurrent). This distinction is critical: the causal origin is an unrepeatable historical fact (the genesis event occurred at a specific time), whereas

the ontological dependence is a repeatable bearer relationship (the concept can be concretized in multiple copies of R). The genesis event cannot be replaced by another without c becoming a different entity Fine [1995], but this is an identity criterion, not an ontological dependence on the occurrent. A GDC cannot existentially depend on an occurrent (Axiom D5), so we frame the relationship to the process as *causal origin*, not as *existential dependence*. The “generic dependence” on the EventChain-record (D2) captures the ontological bearer relationship.

3.4.3 Generic Dependence of Status on Events

A concept’s status s (e.g., `validated`) *generically depends* on the events in its chain: s depends on the existence of at least one `VALIDATE` event, but not on any *specific* `VALIDATE` event. If a different agent had performed the validation at a different time, the status would still be `validated`.

3.4.4 Mutual Dependence of Relations on Concepts

A relation r between concepts c_1 and c_2 *mutually depends* on both concepts: r cannot exist if either ceases to exist. This is mutual rigid dependence Fine [1995]. If `concept-A` is deprecated, the relation “`concept-A` isomorphic-to `concept-B`” ceases to hold, even if `concept-B` still exists.

We formalize these dependence patterns as an OWL 2 DL alignment fragment in the next subsection.

3.5 Formal Alignment Fragment: OWL 2 DL Axiomatization

We present the OWL 2 DL alignment fragment that formalizes the dependence patterns above in Appendix A. It declares `adl:Event` $\sqsubseteq$ `bfo:occurrent`, `adl:EventChain` $\sqsubseteq$ `bfo:process`, and `adl:Concept` $\sqsubseteq$ `bfo:generically_dependent_continuant`, together with datatype properties for cryptographic linkage. The fragment is syntax-checked but not yet validated with ROBOT; full operational encoding of δ and γ exceeds OWL 2 DL expressivity and is deferred to future work (FW1).

SHACL/OWL expressivity boundaries. The SHACL shape graph (Appendix B) validates L3 relation blocks against structural constraints: identifier format (`sh:pattern`), predicate membership (`sh:in`), confidence range (`sh:minInclusive/sh:maxInclusive`), and directionality (`sh:sparql`). Five of eight constraints use SHACL Core; three require SHACL-SPARQL. However, the status derivation function δ and the precondition checks are *not expressible in SHACL or OWL 2 DL* for fundamental reasons: (i) δ aggregates over event sequences (taking the maximum over lifecycle events), which requires recursion or fixed-point computation exceeding SHACL’s declarative expressivity; (ii) preconditions involve comparator dispatch over derived snapshots, which is procedural rather than declarative; (iii) OWL 2 DL is NExpTime-complete and cannot encode sequence aggregation or procedural preconditions (Appendix A, §A.3). Rather than shoehorning δ and preconditions into SHACL/OWL, we provide bidirectional export/import (Turtle/RDF/XML) that preserves the EventChain as a PROV-O activity sequence. The exported RDF can be validated by SHACL for structural constraints, while δ and preconditions are computed by the ADL Lite runtime. This division of labor—declarative validation (SHACL) for structural constraints, procedural computation (Python runtime) for derivation semantics—is a deliberate design choice that respects the expressivity boundaries of each technology. A conservative approximation of δ (e.g., “if any `VALIDATE` event exists, the concept is validated”) could be encoded as a SHACL-SPARQL rule, but this would lose the lattice semantics; full operational encoding remains future work (FW6b).

Interoperability mapping table. Table 6 provides a normative mapping from ADL Lite core constructs to OWL 2 DL classes, SHACL shapes, and PROV-O/IAO properties. This mapping

enables the applied ontology community to adopt ADL Lite’s lifecycle semantics within existing RDF tooling without abandoning the runtime’s derivation functions.

Table 6: Normative interoperability mapping: ADL Lite $\rightarrow$ OWL 2 DL / SHACL / PROV-O / IAO.

ADL Lite	OWL 2 DL Class	SHACL Shape	PROV-O / IAO
Event	<code>adl:Event</code> $\sqsubseteq$ <code>bfo:occurrent</code>	<code>sh:NodeShape</code> (event-id pattern)	<code>prov:Activity</code>
EventChain	<code>adl:EventChain</code> $\sqsubseteq$ <code>bfo:process</code>	<code>sh:NodeShape</code> (ordered list)	<code>prov:Activity</code> (sequence)
Concept	<code>adl:Concept</code> $\sqsubseteq$ <code>bfo:GDC</code>	<code>sh:NodeShape</code> (concept-id pattern)	<code>prov:Entity</code>
EventChain-record	<code>adl:EventChainRecord</code> $\sqsubseteq$ <code>iao:ICE</code>	<code>sh:NodeShape</code> (hash format)	<code>prov:Entity</code> (serialized)
Relation (L3)	<code>adl:Relation</code> $\sqsubseteq$ <code>owl:ObjectProperty</code>	<code>sh:PropertyShape</code> (predicate in-set)	<code>prov:wasDerivedFrom</code> (qualified)
Action (L4)	<code>adl:Action</code> $\sqsubseteq$ <code>owl:Class</code>	<code>sh:NodeShape</code> (action in-set)	<code>prov:Activity</code> (type-qualified)
Actor	<code>adl:Actor</code> $\sqsubseteq$ <code>owl:Thing</code>	<code>sh:NodeShape</code> (actor non-empty)	<code>prov:Agent</code>
Payload	<code>adl:Payload</code> (datatype property)	<code>sh:PropertyShape</code> (JSON schema)	<code>prov:value</code> (literal)
Hash	<code>adl:SHA256Hash</code> (datatype property)	<code>sh:PropertyShape</code> (64 hex chars)	<code>prov:hadPrimarySource</code>
REGISTER	<code>adl:RegisterEvent</code>	<code>sh:sparql</code> (genesis check)	<code>prov:wasGeneratedBy</code>
VALIDATE	<code>adl:ValidateEvent</code>	<code>sh:sparql</code> (predecessor status)	<code>prov:wasAttributedTo</code>
DEPRECATE	<code>adl:DeprecateEvent</code>	<code>sh:sparql</code> (status validated)	<code>prov:wasInvalidatedBy</code>
FORK	<code>adl:ForkEvent</code>	<code>sh:sparql</code> (parent exists)	<code>prov:wasDerivedFrom</code>
EVIDENCE	<code>adl:EvidenceEvent</code>	<code>sh:NodeShape</code> (evidence type)	<code>prov:used</code>
$\delta(C)$	<i>Not expressible</i> (recursion)	<i>Not expressible</i> (sequence)	<code>prov:hadActivity</code> (approx.)
$\gamma(C)$	<i>Not expressible</i> (aggregation)	<i>Not expressible</i> (aggregation)	<code>prov:value</code> (approx.)
Precondition	<i>Not expressible</i> (procedural)	<i>Partial</i> (SHACL-SPARQL)	<i>Not applicable</i>

Adoption path. The bidirectional export/import (Turtle/RDF/XML via `owl_export.py` / `jsonld_export.py`) allows ADL Lite concepts to be consumed by RDF tooling without losing the EventChain structure. The exported RDF represents each event as a PROV-O activity with cryptographic linkage, and the concept as a PROV-O entity with derived status. This is a “lossy but useful” export: the lifecycle semantics (δ , γ , preconditions) are not encoded in the RDF (they exceed OWL/SHACL expressivity), but the structural and provenance information is preserved. The SHACL shape graph (Appendix B) validates five of eight L3 structural constraints; the remaining three (directionality, cross-document existence, conditional symmetry) require SHACL-SPARQL. Full operational encoding of δ and preconditions in SHACL/OWL remains future work (FW6b).

3.6 Comparison with Upper Ontologies: Alignment Choices, Deviations, and Costs

Table 5 identifies three intentional deviations.

Deviation 1: No distinction between continuants and occurrents at the language level. BFO and DOLCE encode the continuant–occurrent distinction in the ontology language (OWL, FOL) Arp et al. [2015], Gangemi et al. [2002]; ADL Lite enforces it conceptually through the event model, without requiring multiple representations. This means ADL Lite’s alignment with BFO is *conceptual guidance* rather than strict commitment: the system follows BFO’s categorical distinctions in its design, but does not import BFO modules directly or rely on OWL reasoners for enforcement. *Interoperability cost:* OWL export requires a translation layer for BFO module import.

Deviation 2: No explicit quality hierarchy. BFO and DOLCE have elaborate quality hierarchies Arp et al. [2015], Gangemi et al. [2002]; ADL Lite encodes qualities as payload fields in events (e.g., “confidence” is a value carried by `VALIDATE` events). *Interoperability cost:* DOLCE’s quality taxonomy cannot be directly reused; domain-specific extensions require explicit mapping from payload fields to quality classes.

Deviation 3: Actions as first-class citizens. In BFO, actions are a subclass of processes; in ADL Lite, they are co-equal with objects at the language level, reflecting the event-first stance. *Interoperability cost:* direct reuse of UFO-B’s process hierarchy is prevented, but UFO-B’s “Institutional Event” category Guizzardi et al. [2024] remains compatible. ADL Lite’s precondition language is a decidable, non-recursive fragment (Comparator enumeration with explicit lambda dispatch, no quantification) that sacrifices full FOL expressivity to guarantee $O(1)$ evaluation. Each precondition rule $\langle f, k, v \rangle$ is equivalent to a ground atomic comparison $k(f(x), v)$; the conjunction of rules is a variable-free conjunction of such atoms, evaluated in polynomial time. Consequently, ADL Lite preconditions constitute a *tractable specialisation* of UFO-B’s FOL framework.

Deviation 4: Historical vs. ontological dependence. We distinguish the causal origin of a concept (traceable to the genesis event, an occurrent) from its ontological bearer (the EventChain-record, an ICE). The concept is a GDC that depends on the record (D2), not on the process (D5). This avoids the category mistake of making a GDC existentially depend on an occurrent. The “historical” or “genetic” relationship to the process is a causal narrative, not an ontological dependence in the Fine/BFO sense.

4 Architecture and Formal Semantics

4.1 Document Model: L1–L4 as Conceptual Modeling Layers

ADL Lite organizes capability documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 7 summarizes the document model.

The L1 layer comprises YAML front matter containing identity metadata; critically, these fields are *derived snapshots* recomputed from the EventChain. The L2 layer consists of Markdown body prose subject to the Singular Subjectivity Assertion (SSA).

Definition (SSA). The Singular Subjectivity Assertion is a document-level constraint stating that every L2 Markdown body is authored by a single actor at a single time. It is enforced not by the ActionExecutor (which governs L4 events), but by the parser: any L2 section containing multiple contradictory subjective claims without explicit attribution is flagged as an SSA violation. The SSA ensures that the L2 narrative layer remains traceable to a single source of subjectivity, preventing

Table 7: The L1–L4 Document Model. Each layer contributes distinct semantics to the capability document.

Layer	Syntax	Role	Event Type
L1	YAML front matter	Identity metadata (derived snapshot)	SNAPSHOT
L2	Markdown body	Human/LLM narrative prose	—
L3	<code>adl:relation</code> , <code>adl:evidence</code> , <code>adl:seal</code>	Typed semantic assertions	RELATE, SEAL, EVIDENCE,
L4	<code>adl:action</code>	Typed actions with preconditions	REGISTER, VALIDATE, DEPRECATE, ...

“ghost-authored” reasoning that cannot be traced to an event. Contradictory subjective assertions must be recorded as separate L4 events (e.g., EVIDENCE or RELATE) by different actors. The SSA interacts with lifecycle governance as follows: (i) a concept in **provisional** status may contain SSA-violating prose (it is unvalidated); (ii) a VALIDATE action’s precondition requires that the L2 body pass SSA checking, ensuring that validated concepts have traceable narrative provenance; (iii) FORK events inherit the parent’s L2 body but must satisfy SSA for the new actor’s rationale. Full machine-checkable enforcement of SSA is planned via SHACL shape validation (FW6).

The L3 layer introduces typed semantic assertions through fenced blocks supporting predicates such as **isomorphic-to**, **specialisation-of**, and **related-to**, with lifecycle operations (establishment, modification, revocation) recorded as RELATE events. These predicates serve a dual purpose: they express domain relationships between concepts, and they provide a mechanism for linking near-duplicate concepts that may arise from multi-agent authoring. For example, if two agents independently register “gradient-explosion” and “exploding-gradients”, the recommended workflow is to detect the near-duplicate by string similarity and link them with the strongest applicable predicate (**isomorphic-to** for structural equivalence). If one concept is strictly superior, the other is marked as **deprecated** with a FORK or RELATE event pointing to the canonical concept. Automated canonicalisation via embedding clustering is deferred to future work (FW7).

Invariant 2 (L3 Relation Reconciliation). When either the source or target concept of a RELATE event undergoes a state transition to **deprecated** or **archived**, the relation’s validity is determined by the status of both endpoint concepts. A relation is valid only if at least one endpoint is in **validated** status and neither endpoint is in **archived** or **deprecated** status. Let r be a relation with predicate p from concept C_1 to concept C_2 , and let $S(C)$ denote the current lifecycle status of concept C . Then:

$$\text{valid}(r) \iff S(C_1) \notin \{\text{archived}, \text{deprecated}\} \wedge S(C_2) \notin \{\text{archived}, \text{deprecated}\} \wedge (S(C_1) = \text{validated} \vee S(C_2) = \text{validated}). \quad (4)$$

Relations with at least one endpoint in **validated** status and no endpoint in **archived** or **deprecated** status remain valid for query and traversal; relations between two **provisional** concepts, or where either endpoint is **deprecated** or **archived**, are excluded from the warm index. If a forked concept C' is created via a FORK event from C , the parent concept retains all existing relations, while C' inherits **isomorphic-to** and **specialisation-of** links from C but not **analogical-to** links, which are re-evaluated during the forking agent’s validation workflow. This invariant ensures that the relation graph remains navigable during multi-agent reorganisation and that stale or superseded edges do not pollute semantic queries.

4.2 EventChain: Core Data Structure

The EventChain-record (the serialized information content entity, see §3.2.6) is the fundamental data structure of ADL Lite: an append-only sequence of cryptographically linked events. An event comprises ten fields: `event_id`, `concept_id`, `event_type`, `actor`, `reasoning`, `timestamp`, `payload`, `previous_event_id`, `hash`, and an optional `signature` (for Ed25519 signature verification, §4.8). The event type system is partitioned into lifecycle events Σ_{life} and communication events Σ_{comm} :

$$\Sigma_{\text{life}} = \{\text{REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE}\} \quad (5)$$

$$\Sigma_{\text{comm}} = \{\text{RELATE, EVIDENCE, SEAL, ANNOUNCE, PUBLISH, SNAPSHOT, CALIBRATE, SYNC_DASHBOARD, LISTEN, REVOKE}\} \quad (6)$$

4.3 Canonical Serialization and Cryptographic Integrity

Each event’s hash is computed from canonical JSON serialization with lexicographically sorted keys, six-decimal rounding, and the timestamp. The canonicalization policy is version-locked in the codebase: a schema version identifier (`canon-v1`) is included in the hash input, ensuring that future changes to rounding precision or key ordering do not retroactively invalidate existing hashes. If a new canonicalization version is introduced, old chains remain valid under their original version, while new chains adopt the updated policy. This prevents identity drift when serialization conventions evolve. Integrity verification `VerifyIntegrity(C)` checks four conditions for a chain $C = [e_1, \dots, e_n]$: (1) genesis anchoring ($e_1.\text{previous_event_id} = \text{null}$), (2) cryptographic linkage ($e_i.\text{prev_hash} = e_{i-1}.\text{hash}$), (3) hash correctness ($e_i.\text{hash} = \text{SHA-256}(\text{Canon}(e_i))$), and (4) no dangling `previous_event_id` on the genesis event. When `full = true`, it additionally verifies archive file hashes against stored `archive_pointer` values.

Genesis hash stability. The genesis hash is the hash of the first REGISTER event, which includes the `concept_id` in its canonical JSON input. Early versions of the canonicalization excluded `concept_id`, causing collisions when different concepts shared the same initial payload; this was corrected by including `concept_id` in the hash input (E1, §5.2). The canonical JSON serializer normalizes line endings to LF and encodes Unicode consistently as UTF-8, preventing cross-platform drift. The `canon_version` field ensures that benign formatting changes (e.g., prettier Markdown reformatting of the L2 body) do not affect event hashes: only the event payload fields listed above participate in the hash computation, not the surrounding Markdown prose.

4.4 Precondition Language and Action Executor

4.4.1 Formal Precondition Language

The ActionExecutor validates L4 action blocks against preconditions expressed in a constrained, decidable language. A precondition rule is a triple $\langle \text{field}, \text{comparator}, \text{value} \rangle$ with comparator alphabet $\mathcal{K} = \{\text{EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS}\}$. Table 8 gives the formal semantics.

BNF grammar. The complete syntax of the precondition language is:

$$\begin{aligned} \text{rule} &::= \langle \text{field}, \text{comparator}, \text{value} \rangle \\ \text{field} &::= \text{identifier} \quad (\text{e.g., } \text{status}, \text{confidence}, \text{actor}) \\ \text{comparator} &::= \text{EQ} \mid \text{NEQ} \mid \text{GT} \mid \text{GTE} \mid \text{LT} \mid \text{LTE} \mid \text{IN} \mid \text{EXISTS} \\ \text{value} &::= \text{scalar} \mid \text{set} \mid \text{null} \\ \text{rule_list} &::= \text{rule} \mid \text{rule_list AND rule_list} \\ &\quad \mid \text{rule_list OR rule_list} \end{aligned}$$

Table 8: Comparator semantics: explicit operator mapping to ground predicates.

Comparator	Predicate	FOL encoding
EQ	Equality	$x = y$
NEQ	Disequality	$\neg(x = y)$
GT	Greater than	$x > y$
GTE	Greater than or equal	$x \geq y$
LT	Less than	$x < y$
LTE	Less than or equal	$x \leq y$
IN	Set membership	$x \in Y$ (where Y is a finite set)
EXISTS	Non-null check	$\exists x : x \neq \text{null}$

Formal semantics. Let $\mathcal{D}$ be the domain of values (scalars, sets, and null). Let $\text{Snapshot}(C)$ be the derived snapshot of chain C , a finite map from field names to $\mathcal{D}$. The semantic function $\text{eval}(r, C) \in \{\text{true}, \text{false}\}$ is defined for a rule $r = \langle f, \kappa, v \rangle$ by:

$$\text{eval}(\langle f, \kappa, v \rangle, C) = \text{apply}(\kappa, \text{lookup}(f, C), v) \quad (7)$$

where $\text{lookup}(f, C) = \text{Snapshot}(C)[f]$ (the value of field f in the snapshot, or null if absent), and $\text{apply}(\kappa, x, y)$ is the comparator-specific predicate from Table 8. The snapshot is pre-computed in $\mathcal{O}(n)$ time by a single scan of C ; subsequent rule evaluations are $\mathcal{O}(1)$.

Composition semantics. A rule list $R = [r_1, \dots, r_k]$ is evaluated as a conjunction: $\text{eval}(R, C) = \bigwedge_{i=1}^k \text{eval}(r_i, C)$. Evaluation is short-circuit: if $\text{eval}(r_j, C) = \text{false}$ for any $j < k$, the remaining rules are not evaluated. The language is a variable-free ground fragment: each rule is a single atomic comparison over a cached derived snapshot, with no quantification, no recursion, and no dynamic code execution. The conjunction of k rules is a variable-free conjunction of ground atoms, evaluated in $\mathcal{O}(k)$ time (Theorem 8).

Precondition evaluation is pure and referentially transparent: $\text{eval}(r, C)$ uses a cached derived snapshot, where $\text{lookup}(f, C) = \text{Snapshot}(C)[f]$. The comparator dispatch is implemented via explicit lambda mapping without dynamic code execution. Precondition evaluation is *local*: it references only the snapshot of the chain C being validated, and does not access external chains, external databases, or network resources. Cross-chain preconditions (e.g., “allow VALIDATE only if concept X is **validated**”) are not supported in the core language; they are deferred to future work (FW2). This local-scope restriction ensures that precondition evaluation is referentially transparent, deterministic, and executable without consensus or network access.

4.4.2 Precondition Language Expressivity

The precondition language is intentionally minimal: a variable-free ground fragment with eight comparators, no recursion, no quantification, and no dynamic code execution. This minimalism guarantees decidability and $\mathcal{O}(1)$ per-rule evaluation (Theorem 8), but it also limits expressivity. Table 11 classifies the expressivity boundaries.

Temporal predicates. Time is not a first-class value in the precondition language. However, the ActionExecutor can enforce temporal constraints at the application layer (e.g., a minimum delay between validation and deprecation). The precondition language’s scope is deliberately restricted to snapshot-based guards to ensure decidability and $\mathcal{O}(1)$ evaluation time per rule. Temporal reasoning is a known expressivity gap, deferred to future work (FW2).

Table 9: Precondition language expressivity: what can and cannot be expressed.

Constraint category	Cate- gory	Examples	Expressible?
Field-value comparison		<code>status EQ provisional,</code> <code>confidence GTE 0.5</code>	✓ (Core)
Set membership		<code>tags IN [verified,</code> <code>reviewed]</code>	✓ (Core)
Existence		<code>actor EXISTS</code> (field defined in snapshot)	✓ (Core)
Conjunction / disjunction		<code>status EQ provisional AND</code> <code>confidence GTE 0.5</code>	✓ (Core)
Temporal predicate		<code>7 days passed since last</code> <code>VALIDATE</code>	× (Not in core; application layer)
Cross-chain reference		<code>parent.status EQ validated</code>	× (Not in core; deferred to FW2)
Quantification / aggregation		<code>at least 3 validators</code> <code>approved</code>	× (Handled by γ , not preconditions)
Arithmetic / string manipulation		<code>confidence within 10% of</code> <code>mean</code>	× (Not in core)
Recursive / self-referential	/ self-	<code>no concept is its own</code> <code>ancestor</code>	× (Not in core)

Cross-chain predicates. Not supported by design. The precondition language evaluates only the snapshot of the chain being modified. This ensures referential transparency and eliminates the need for distributed consensus during precondition evaluation. Cross-chain references would require external chain lookup, which introduces non-determinism (the target chain may change between evaluation and append). This is a deliberate design choice, not a limitation.

Theorem 8 (Precondition Evaluation Termination and Complexity). For any well-formed EventChain C and any precondition rule r , $\text{eval}(r, C)$ terminates in $\mathcal{O}(1)$ time (assuming the derived snapshot is cached). For a list of k rules, total time is $\mathcal{O}(k)$ and space is $\mathcal{O}(1)$. *Proof Sketch:* Both field lookup and comparator dispatch are $\mathcal{O}(1)$; the language contains no recursion or quantification. See Appendix E for the full argument.

Table 10 summarizes the complexity; the decidability and complexity-class discussion (membership in **P**) is in Appendix H.

Table 10: Computational complexity of precondition evaluation.

Operation	Time	Space	Notes
Single rule $\text{eval}(r, C)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	No recursion, no quantification
Rule list $\text{eval}(R, C)$, $ R = k$	$\mathcal{O}(k)$	$\mathcal{O}(1)$	Short-circuit on first failure
Field lookup $\text{lookup}(f, C)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Memoized snapshot or dict lookup
Comparator dispatch $k(x, y)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Closed 8-case enumeration

4.4.3 Expressivity and Limitations

The precondition language is a ground, variable-free fragment: each rule is a single atomic comparison $\langle \text{field}, \text{comparator}, \text{value} \rangle$ over a cached derived snapshot. Table 11 lists common lifecycle invariants and whether they are expressible.

The design trades full first-order expressivity for guaranteed $\mathcal{O}(1)$ per-rule evaluation and prevention of runtime code injection. Domain-specific invariants that exceed the core language

Table 11: Precondition expressivity: invariants supported by the core language.

Invariant	Expr?	Mechanism
Status must be provisional before VALIDATE	✓	status EQ provisional
Confidence must be ≥ 0.5 for VALIDATE	✓	confidence GTE 0.5
Exactly one REGISTER per chain	×	Would require universal quantification over events
At least $N_{\min}$ distinct validators	$\triangle$	Partial: can count validators in snapshot, but not prove distinctness of keys (future authentication)
No VALIDATE after DEPRECATE unless REACTIVATE	×	Would require temporal sequence constraints beyond current syntax
Actor must not be in a blacklist set	✓	actor IN blacklist (finite set)
Payload field must exist (non-null)	✓	confidence EXISTS
Cross-chain concept must exist	×	Would require cross-chain lookup (future: H index)
Two validators must have disjoint organisations	×	Would require quantification over two actors and comparison of their org prefixes

(e.g., temporal precedence, cross-chain existence) are deferred to the application layer or future extensions (FW12). The closed Comparator enumeration ensures that no rule introduces recursion or unbounded computation.

Complete lifecycle transition matrix. Table 12 lists the ActionExecutor preconditions for each lifecycle event. Under the CRDT LUB semantics, the final status is the maximum over all lifecycle events in the chain, not the last event. However, the ActionExecutor still enforces preconditions to prevent semantically nonsensical sequences (e.g., archiving a concept that has never been validated). The preconditions for each action are encoded in `adl_core_ontology.yaml`.

Table 12: Lifecycle event preconditions: permitted events from current status (rows). The final status is the LUB of all lifecycle events in the chain (CRDT semantics).

Status	REG.	VAL.	DEP.	FORK	ARCH.
provisional	×	✓	✓	✓	✓
validated	×	✓ (re)	✓	✓	✓
deprecated	×	✓	×	×	✓
forked	×	✓	✓	×	✓
archived	×	×	×	×	×

REGISTER is permitted only as a genesis event (creating a new concept). From **validated**, **VALIDATE** is permitted as a re-validation (adding a new confidence assertion). Under CRDT LUB semantics, once a chain reaches **deprecated**, subsequent **VALIDATE** events do not change the status back to **validated** (the LUB of **deprecated** and **validated** is **deprecated**), but they can still be appended as auditable evidence. Re-activation after deprecation requires a new **REGISTER** event (new genesis), not a reversal of the existing chain.

4.4.4 Action Executor and Consensus Engine

The ActionExecutor validates L4 action blocks against a closed action registry defined in `adl_core_ontology.yaml`. Each action definition specifies preconditions as a list of `PreconditionRule`

objects. The ConsensusEngine governs the resulting status transitions by appending events to the chain; the final status is derived via the CRDT LUB over all lifecycle events (Theorem 1), not by a transition matrix. Forking creates a new child chain with a fresh **REGISTER** event while the original chain’s status is the LUB of its previous status and **forked**; both evolve independently.

4.5 Formal Derivation Semantics

This section provides the formal foundation for the derivation semantics. All definitions are self-contained; expanded proof sketches and sensitivity analysis appear in Appendix E. We have drafted a TLA⁺ specification of the EventChain state machine for lengths up to 20 events (FW10); extension to 100–1000 events is planned for the next revision. We establish Theorems 1–3 through rigorous natural-language argument in this paper.

Table 13: Formal notation for the derivation semantics.

Symbol	Definition
$\Sigma = \Sigma_{\text{lifecycle}} \cup \Sigma_{\text{comm}}$	Event alphabet (lifecycle $\cup$ communication)
$C = [e_1, \dots, e_n]$	Well-formed EventChain
$\text{WF}(C)$	Well-formedness predicate (12 axioms; see Appendix E)
$\delta(C) \in S$	Status derivation function
$\gamma(C) \in [0, 1]$	Confidence aggregation function
$C_{\text{lifecycle}}$	Lifecycle subsequence $\{e \in C \mid e.\tau \in \Sigma_{\text{lifecycle}}\}$
S	Status space {provisional, validated, deprecated, forked, archived}
$f : \Sigma_{\text{lifecycle}} \rightarrow S$	Event-type-to-status mapping
$\text{Fork}(C, a)$	Fork operation producing (C_{fork}, C')

A chain $C = [e_1, \dots, e_n]$ is well-formed, denoted $\text{WF}(C)$, if it satisfies 12 axioms (genesis anchoring, shared **concept_id**, cryptographic linkage, hash correctness, distinct **event_id**, non-empty actor, timestamp monotonicity, payload schema conformance, action preconditions, hash format, canonical fields, and valid event type). The full conjunctive formalisation is in Appendix E.

The status space is $S = \{\text{provisional}, \text{validated}, \text{deprecated}, \text{forked}, \text{archived}\}$. The status lattice is a join-semilattice with partial order:

$$\text{provisional} < \text{forked} < \text{validated} < \text{deprecated} < \text{archived} \quad (8)$$

For a chain C , let $C_{\text{lifecycle}}$ denote the lifecycle subsequence and let $f : \Sigma_{\text{lifecycle}} \rightarrow S$ map event types to their corresponding status values. The status derivation function $\delta(C)$ is the least upper bound (LUB) of all lifecycle events in the chain, computed as the maximum over the lattice order:

$$\delta(C) = \begin{cases} \text{provisional} & \text{if } C_{\text{lifecycle}} = [] \\ \max_{\prec} \{f(e.\tau) \mid e \in C_{\text{lifecycle}}\} & \text{otherwise} \end{cases} \quad (9)$$

where $\max_{\prec}$ is the maximum over the integer order of the lattice (1 provisional, 2 forked, 3 validated, 4 deprecated, 5 archived). This is computed in $O(|C_{\text{lifecycle}}|)$ time by a single scan of the lifecycle subsequence.

Conflict resolution for δ . Under the CRDT LUB semantics, status is determined by the *highest* lifecycle event in the lattice, not by the *last* event. Once a chain reaches deprecated, no subsequent VALIDATE or REGISTER event can change the status back to validated or provisional—the LUB of deprecated with any lower status is always deprecated. Similarly, once a chain reaches archived, the status is permanently archived. This guarantees that status *never regresses*, which is a fundamental property of monotonic CRDTs. Re-activation after deprecation requires a new REGISTER event on a *new* chain (creating a new genesis hash), not a reversal of the existing chain.

4.5.1 Confidence Aggregation: Three Variants

Let $V = [e \in C \mid e.\tau = \text{VALIDATE}]$ denote the validation subsequence. ADL Lite provides three confidence aggregation strategies.

Variant 1: Default (G-Counter max). The simplest strategy, used by default, takes the maximum confidence across all `VALIDATE` events (or `SNAPSHOT` if no validation has occurred):

$$\gamma_{\text{default}}(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \text{clamp}_{[0,1]}(\max\{e.\text{payload}[\text{"confidence"}] \mid e \in V\}) & \text{otherwise} \end{cases} \quad (10)$$

where $\max$ is the maximum over all validation events in the chain (G-Counter semantics), and $\text{clamp}_{[0,1]}(x) = \max(0, \min(1, x))$. Evaluated in $O(1)$ time by an incremental cache (`_cached_confidence`) that is updated on every `append()`: when a new `VALIDATE` or `SNAPSHOT` event with confidence c is appended, the cache is updated to $\max(\text{cache}, \text{clamp}_{[0,1]}(c))$. A defensive recompute (full $O(|V|)$ scan) is triggered only if the cache is stale (e.g., after direct mutation of the internal event list). This G-Counter semantics ensures that once a high-confidence validation is recorded, subsequent lower-confidence assertions cannot decrease the aggregate; this prevents malicious or erroneous validators from downgrading a concept after it has been validated. The $O(1)$ claim applies to *reading* γ_{default} ; *appending* an event is $O(1)$ for the cache update and $O(1)$ for the well-formedness check (hash computation).

Variant 2: Bonus-formula aggregate (γ_{agg}). This variant mitigates collusion by aggregating per-actor maxima and adding a bonus for cross-validation:

$$\gamma_{\text{agg}}(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \min(1.0, c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1)) & \text{otherwise} \end{cases} \quad (11)$$

where $N_{\text{vals}} = |\text{actors}(V)|$ is the number of distinct validators, and:

$$c_{\text{base}} = \max\left(0.5, \frac{1}{N_{\text{vals}}} \sum_{a \in \text{actors}(V)} \phi(a, V)\right) \quad (12)$$

with $\phi(a, V) = \max\{e.\text{payload}[\text{"confidence"}] \mid e \in V \wedge e.\text{actor} = a\}$ being the maximum confidence reported by actor a . The $\min(1.0, \cdot)$ cap ensures boundedness; the 0.05 bonus per distinct validator rewards cross-validation. Sensitivity analysis for different (β, c_{min}) parameters appears in Appendix E.

Variant 3: Calibrated confidence (γ_{cal}). This variant weights each validator's confidence by a per-actor accuracy score:

$$\gamma_{\text{cal}}(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \text{clamp}_{[0,1]}\left(\frac{\sum_{a \in \text{actors}(V)} \text{accuracy}_a \cdot \phi(a, V)}{\sum_{a \in \text{actors}(V)} \text{accuracy}_a}\right) & \text{otherwise} \end{cases} \quad (13)$$

where $\text{accuracy}_a \in [0, 1]$ is the accuracy score of actor a . This requires an external calibration profile (`.adl/calibration.yaml`).

Disclaimer: calibration in the collaborative-audit model. The accuracy_a scores in γ_{cal} are themselves self-reported in the Phase 1 collaborative-audit model (there is no external ground-truth source for validator accuracy). Consequently, γ_{cal} provides a *heuristic calibration* that redistributes confidence among validators, but it does *not* prevent collusion: a colluding actor

can report $\text{accuracy}_a = 1.0$ and drive the aggregate to any value in $[0, 1]$. Full collusion resistance requires the authenticated identity and staking mechanisms planned for Phase 2/3 (FW4, FW9).

Sybil vulnerability in Phase 1. The current non-Byzantine model does not prevent Sybil attacks: a single actor can create multiple pseudonyms and append multiple `VALIDATE` events, each with high confidence, driving γ_{agg} to 0.99 with as few as 10 distinct pseudonyms (since $c_{\text{base}} \geq 0.5$ and $0.05 \times 9 = 0.45$). This is a known Phase 1 limitation (L3, Section 6.3), explicitly demonstrated in adversarial experiment E14 (Table 20). The mitigation is scoped to Phase 3: staking-based validation (FW9) and per-actor reputation calibration (Section 4.9). Under the staking model, each validator must deposit a stake to register, and the cost of creating a Sybil identity is proportional to the stake amount. The calibration layer (γ_{cal}) weights each validator by a per-actor accuracy score, so a new Sybil identity with no history receives a low accuracy weight and contributes negligibly to the aggregate confidence.

Conflict resolution for γ . All three variants resolve multiple `VALIDATE` events by computing per-actor maxima and then aggregating. For γ_{default} , the G-Counter (max) over all `VALIDATE` events prevails. For γ_{agg} and γ_{cal} , per-actor maxima are computed first, then aggregated. If two validators append `VALIDATE` events with different confidence values, each contributes its own maximum.

Conflicting validators and time decay. When validators disagree (e.g., `agent_1` reports confidence 0.9 while `agent_2` reports 0.5), the three variants resolve the conflict differently. γ_{default} ignores the disagreement and takes the maximum value (0.9). γ_{agg} preserves both values (via per-actor maxima) and rewards cross-validation with the bonus term. γ_{cal} weights each validator by their accuracy score, so a high-accuracy validator’s 0.9 dominates a low-accuracy validator’s 0.5. Time decay is implemented via an optional EWMA (exponentially weighted moving average) layer with smoothing factor $\alpha = 0.3$: older validations receive geometrically decreasing weight, preventing stale confidence from dominating the aggregate. The EWMA layer is independent of the three variants and can be combined with any of them.

A fork operation `Fork(C, a)` appends a `FORK` event to the parent chain and creates a child chain with a fresh `REGISTER` event and a deterministic fork suffix appended to `concept_id`. Child chains are independent: each child’s derived state depends only on its own event sequence.

Total order $\prec$ for conflict resolution. Within a single chain, events are totally ordered by the cryptographic linkage (each event points to its predecessor via `previous_event_id`). For concurrent events across branches, we define a total order $\prec$ as:

$$\begin{aligned} e_i \prec e_j \iff & e_i.\text{timestamp} < e_j.\text{timestamp} \vee \\ & (e_i.\text{timestamp} = e_j.\text{timestamp} \wedge \\ & e_i.\text{event_id} <_{\text{lex}} e_j.\text{event_id}) \end{aligned} \tag{14}$$

where $<_{\text{lex}}$ is lexicographic comparison of UUID strings. This order is total because timestamps are comparable and `event_id` provides a tiebreaker. The append-only property of EventChains ensures that $\prec$ is consistent with the chain’s internal order: if e_i precedes e_j in the chain, then $e_i.\text{timestamp} \leq e_j.\text{timestamp}$ (with equality possible only for simultaneous appends, resolved by `event_id`).

Within-chain total order versus across-chain partial order. Within a single chain, events are totally ordered by the cryptographic linkage (each event points to its predecessor via `previous_event_id`). This is a *total order*: every pair of events in the chain is comparable. Across chains (e.g., in a distributed merge scenario), events are initially *partially ordered*: events from different chains have no predecessor relationship until they are merged. The merge operation (Phase 3, future work) linearizes the merged set by sorting all events according to $\prec$, producing a consistent total order. This design is standard for CRDTs: individual replicas maintain a total

order, while the global state is a partial order that is linearized at merge time. The linearization is deterministic because $\prec$ is a total order on the set of all events.

Concurrent conflict resolution. When multiple agents append events to the same concept concurrently, the resulting events are ordered by $\prec$. For δ : under the CRDT LUB semantics, the status is the maximum over all lifecycle events in the chain, independent of append order. A DEPRECATE event always dominates a VALIDATE event (since deprecated $>$ validated in the lattice), and an ARCHIVE event dominates all other lifecycle events. For γ : the G-Counter (max) over all VALIDATE events determines the confidence, independent of append order. When a VALIDATE and a DEPRECATE conflict, the DEPRECATE prevails in status but the confidence remains the maximum of all validations. Equivocation (the same actor appending contradictory events) is retained as auditable evidence, not resolved by the ordering.

Theorem 1 (Determinism). For any well-formed chain C , $\delta(C)$ is unique and is the least upper bound of all lifecycle events in C . *Proof Sketch:* Let $C = [e_1, \dots, e_n]$ be well-formed. By Well-Formedness Axiom (3) (cryptographic linkage), C is a totally ordered sequence. The lifecycle subsequence $C_{\text{life}} = \text{filter}(C, \Sigma_{\text{life}})$ is therefore also totally ordered. If $C_{\text{life}} = []$, $\delta(C) = \text{provisional}$ by definition (unique). Otherwise, C_{life} is a finite, non-empty sequence. The status map $f : \Sigma_{\text{life}} \rightarrow S$ assigns each lifecycle event a status value in the lattice. The LUB over a finite subset of a total order is the maximum element, which is unique because the lattice order is antisymmetric. Thus $\delta(C) = \max_{\prec} \{f(e.\tau) \mid e \in C_{\text{life}}\}$ is uniquely determined. The complexity claim follows because computing C_{life} requires a single scan of C ($O(|C|)$), and computing the maximum over C_{life} is $O(|C_{\text{life}}|)$.

Theorem 2 (Fork Determinism). Let C be a well-formed chain with $\delta(C) = s$ and let C fork to (C_{fork}, C') . Then $\delta(C') = \text{provisional}$ and $\delta(C_{\text{fork}}) = \max(s, \text{forked})$ (the LUB of the parent's previous status and FORK). *Proof Sketch:* By the fork definition, $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$ where $e_{\text{FORK}}.\tau = \text{FORK} \in \Sigma_{\text{life}}$. Under the CRDT LUB semantics, $\delta(C_{\text{fork}}) = \max_{\prec} \{f(e.\tau) \mid e \in (C_{\text{fork}})_{\text{life}}\} = \max(s, \text{forked})$. If the parent was previously **validated** ($s = \text{validated}$, with order 3), then $\max(3, 2) = 3$ and the parent remains **validated**; the FORK event is recorded as auditable evidence but does not downgrade the status. If the parent was **provisional** ($s = \text{provisional}$, with order 1), then $\max(1, 2) = 2$ and the parent becomes **forked**. The child chain $C' = [e'_{\text{REG}}]$ where $e'_{\text{REG}}.\tau = \text{REGISTER} \in \Sigma_{\text{life}}$ and $e'_{\text{REG}}.\text{previous_event_id} = \text{null}$. Since C' contains exactly one event, $(C')_{\text{life}} = [e'_{\text{REG}}]$. By Theorem 1, $\delta(C') = f(\text{REGISTER}) = \text{provisional}$. The two derivations are independent because C_{fork} and C' are disjoint chains after the fork point: C_{fork} appends to C while C' starts afresh. No subsequent event in either chain can affect the other's status derivation because status is derived per-chain (Theorem 1).

Theorem 3 (Transition Monotonicity). For any well-formed C and event e_{new} , the transition from $s = \delta(C)$ to $s' = \delta(C \oplus [e_{\text{new}}])$ is valid iff $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$. *Proof Sketch:* By structural induction on $|C|$. **Base case:** $C = [e_{\text{REG}}]$ with $\delta(C) = \text{provisional}$. (The empty chain is not well-formed; every well-formed chain has a genesis REGISTER event as its first element.) For any e_{new} : if $e_{\text{new}}.\tau \notin \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = [e_{\text{REG}}]$, so $s' = \text{provisional} = s$ (no transition). If $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\tau)$, which the ActionExecutor precondition system permits only if $(\text{provisional}, f(e_{\text{new}}.\tau))$ is a valid edge in the status transition machine. **Inductive step:** Assume the theorem holds for all chains of length n . Let C have length $n + 1$ with $\delta(C) = s$. Consider e_{new} : if $e_{\text{new}}.\tau \notin \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = C_{\text{life}}$, so $s' = s$ (no transition). If $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = C_{\text{life}} \oplus [e_{\text{new}}]$, so $s' = f(e_{\text{new}}.\tau)$. By the ActionExecutor precondition system, e_{new} can be appended only when the precondition rules are satisfied against the current snapshot derived from C . These preconditions encode exactly the valid transitions of the status machine, so (s, s') must be a valid edge. Thus the theorem holds for chains of length $n + 1$. By induction, it holds for all well-formed chains.

Theorem 4 (Confidence Boundedness). For any well-formed C , $\gamma_{\text{default}}(C) \in [0, 1]$, $\gamma_{\text{agg}}(C) \in [0, 1]$, and $\gamma_{\text{cal}}(C) \in [0, 1]$. *Proof Sketch:* For γ_{default} : if $V = []$, $\gamma_{\text{default}}(C) = 0.0 \in [0, 1]$. Otherwise, the payload **confidence** field is clamped to $[0, 1]$ by Pydantic validation at event construction time, so $\gamma_{\text{default}}(C) = \text{clamp}_{[0,1]}(c) \in [0, 1]$. For γ_{agg} : if $V = []$, $\gamma_{\text{agg}}(C) = 0.0$. Otherwise, each **VALIDATE** event carries confidence $\in [0, 1]$; the per-actor maxima $\phi(a, V)$ are therefore in $[0, 1]$, and their mean is a convex combination in $[0, 1]$. The $\max(0.5, \cdot)$ floor ensures $c_{\text{base}} \geq 0.5$, and the $\min(1.0, \cdot)$ cap ensures $\gamma_{\text{agg}}(C) \leq 1.0$. For γ_{cal} : the weighted sum of values in $[0, 1]$ with non-negative weights is a convex combination in $[0, 1]$, and the outer clamp preserves this bound. Thus all three variants are bounded in $[0, 1]$.

Theorem 5 (Confidence Monotonicity). For any well-formed C and any **VALIDATE** event e_{new} with payload confidence c' , $\gamma_{\text{default}}(C \oplus [e_{\text{new}}]) \geq \gamma_{\text{default}}(C)$. *Proof Sketch:* γ_{default} takes the maximum confidence over all **VALIDATE** events in V (G-Counter semantics). Appending e_{new} with $e_{\text{new}}.\tau = \text{VALIDATE}$ adds c' to the set of validation confidences. Therefore $\gamma_{\text{default}}(C \oplus [e_{\text{new}}]) = \max(\gamma_{\text{default}}(C), \text{clamp}_{[0,1]}(c')) \geq \gamma_{\text{default}}(C)$. Strict inequality holds if $c' > \gamma_{\text{default}}(C)$ and $c' \in [0, 1]$. For γ_{agg} : monotonicity holds because appending a new **VALIDATE** from a distinct validator increases N_{vals} by 1 (adding 0.05 to the bonus) and c_{base} is a weighted mean that includes the new confidence. For duplicate validators, $\phi'(a, V') = \max(\phi(a, V), c') \geq \phi(a, V)$, so $c'_{\text{base}} \geq c_{\text{base}}$. The bonus term is unchanged or increased, so $\gamma_{\text{agg}}(C') \geq \gamma_{\text{agg}}(C)$. (The optional γ_{cal} aggregate does not guarantee monotonicity unless all validators' accuracy scores are non-decreasing.) Full details are in Appendix E.

Theorem 6 (Status–Confidence Consistency). If $\delta(C) = \text{validated}$, then $\gamma_{\text{default}}(C) \geq 0.5$, $\gamma_{\text{agg}}(C) \geq 0.5$, and $\gamma_{\text{cal}}(C) \geq 0.5$ (provided all validators have accuracy ≥ 0.5). *Proof Sketch:* $\delta(C) = \text{validated}$ implies that the LUB of all lifecycle events in C is validated, which requires at least one **VALIDATE** event in C_{life} (since no other lifecycle event type maps to validated). Thus $V \neq \emptyset$. Every **validate** action precondition requires payload confidence ≥ 0.5 (enforced by the ActionExecutor via `adl_core_ontology.yaml`). For γ_{default} : the G-Counter max over all **VALIDATE** events includes at least one event with confidence ≥ 0.5 , so $\gamma_{\text{default}}(C) \geq 0.5$. For γ_{agg} : every $\phi(a, V) \geq 0.5$, so the mean $c_{\text{base}}^{\text{raw}} \geq 0.5$, and $c_{\text{base}} = \max(0.5, c_{\text{base}}^{\text{raw}}) \geq 0.5$. The bonus term is non-negative, so $\gamma_{\text{agg}}(C) \geq 0.5$. For γ_{cal} : if all accuracy $_a \geq 0.5$, the weighted mean of values ≥ 0.5 is itself ≥ 0.5 .

Theorem 7 (Well-Formedness Preservation). For any well-formed chain C and any event e_{new} satisfying the preconditions of its action type, the extended chain $C' = C \oplus [e_{\text{new}}]$ is well-formed. *Proof Sketch:* Each WF_i axiom is verified for C' : genesis remains unchanged, **concept_id** shared, linkage preserved, hash computed, and fresh **event_id** ensures distinctness. Full details are in Appendix E.

We provide proof sketches for all seven core theorems (Theorems 1–7) through rigorous natural-language arguments over the closed alphabet Σ ; full details are in Appendix E. Table 14 clarifies the proof status of each theorem. Machine-checked proofs for unbounded chains are planned for future work (FW10, FW12).

Machine-checked proofs. None of Theorems 1–8 have been formally verified in a proof assistant (e.g., Coq, Isabelle, TLA⁺). The TLA⁺ specification (FW10) has been syntax-checked and model-checked for chains up to 20 events (all 2,204 length-3 sequences, 10,000 random sequences of length 4–10). The model-checking confirms that the invariants (determinism, boundedness, monotonicity) hold within the checked bounds, but this is not a formal proof for unbounded chains. Full machine-checked proofs are planned for future work (FW12). Theorem 9 (CRDT convergence) is validated by stress tests (E6: 201 chains, 9,300 events; E13: 50,000 events; E23: 20 concurrent agents) with zero integrity failures, together with the executable assertions in the test suite that verify commutativity, associativity, and idempotence for all tested cases. The G-Set CRDT corollary

Table 14: Proof status of the seven core theorems.

Theorem	Proof Method	Status	Evidence
T1 Determinism	Natural-language argument	Proven in text	LUB over finite lattice (§4.5)
T2 Fork Determinism	Natural-language argument	Proven in text	Append-only semantics (§4.5)
T3 Transition Monotonicity	Structural induction	Proven in text	Base + inductive step (§4.5)
T4 Boundedness	Case analysis	Proven in text	Clamp + Pydantic (§4.9)
T5 Confidence Monotonicity	Direct proof	Proven in text	Max semantics (§4.9)
T6 Status-Confidence Consistency	Direct proof	Proven in text	Precondition enforcement (§4.9)
T7 Well-Formedness Preservation	Structural induction	Proven in text	Axiom-by-axiom check (§4.5)
T8 Precondition Complexity	Direct proof	Proven in text	$O(1)$ per rule (§4.4.1)
T9 CRDT Convergence	Natural-language + executable assertions	Test-validated	E6, E13, E23; Corollary in Appendix E

(Appendix E) provides the natural-language proof sketch that underpins the empirical validation.

4.5.2 Formal Function Signatures and a Worked Example

Explicit function signatures. The derivation functions are defined as:

$$\delta : \text{EventChain} \rightarrow S \quad (15)$$

$$\delta(C) = \begin{cases} \text{provisional} & \text{if } C_{\text{life}} = \emptyset \\ f(\tau_{\text{last}}(C_{\text{life}})) & \text{otherwise} \end{cases}$$

$$\gamma : \text{EventChain} \rightarrow [0, 1] \quad (16)$$

$$\gamma(C) = \begin{cases} 0.0 & \text{if } V = \emptyset \\ \text{clamp}_{[0,1]}(e_{\text{last}}(V).\text{payload}[\text{"confidence"}]) & \text{otherwise} \end{cases}$$

where S is the status space, EventChain is the set of well-formed chains, C_{life} is the lifecycle subsequence, V is the validation subsequence, and $f : \Sigma_{\text{life}} \rightarrow S$ is the total event-type-to-status mapping.

Small-step operational semantics. Appending an event e_{new} to a chain C is denoted $C \oplus [e_{\text{new}}]$. The transition is a pure function:

$$\langle C \rangle \xrightarrow{e_{\text{new}}} \langle C \oplus [e_{\text{new}}] \rangle \quad \text{where} \quad \delta(C \oplus [e_{\text{new}}]) = \begin{cases} \delta(C) & \text{if } e_{\text{new}}.\tau \notin \Sigma_{\text{life}} \\ f(e_{\text{new}}.\tau) & \text{otherwise} \end{cases} \quad (17)$$

The append operation is atomic (guaranteed by `threading.Lock`); the derived snapshot is updated incrementally (status LUB cache and confidence G-Counter cache), so both $\delta(C)$ and $\gamma_{\text{default}}(C)$ are $O(1)$ reads. This is deterministic: the new state depends only on the old state and the appended event, with no side effects or stored mutable fields.

Worked example. Consider a 5-event chain for the concept `disc-capital-trap`. After the genesis `REGISTER` event (Step 1), a `RELATE` event (Step 2, communication) leaves status unchanged at *provisional*. A `VALIDATE` event (Step 3) with confidence 0.70 transitions status to *validated* and sets $\gamma = 0.70$. An `EVIDENCE` event (Step 4) records audit data without changing derived state. A second `VALIDATE` (Step 5) with confidence 0.85 updates γ to 0.85, demonstrating Theorem 5 (Monotonicity). If `agent_2` disagrees at Step 5, they may append a `FORK` event: the parent becomes *forked* and the child begins at *provisional*, both independently well-formed.

4.5.3 Distributed Ordering and Timestamp Assumptions

Within-chain assumptions. (i) *Monotonic timestamps:* Within a single chain, timestamps are non-decreasing ($e_i.\text{timestamp} \leq e_{i+1}.\text{timestamp}$). This is enforced by the `append()` method: if a new event’s timestamp is earlier than the previous event’s timestamp, it is adjusted to the previous timestamp. (ii) *Cryptographic linkage:* Each event points to its predecessor via `previous_event_id`, forming a linear chain. The hash of each event includes the previous event’s hash, creating a tamper-evident sequence. (iii) *Total order within chain:* The chain’s internal order is total: every pair of events is comparable by index.

Across-chain assumptions (distributed authorship). (i) *Clock skew:* ADL Lite does not use a global clock. Each event carries the author’s local timestamp (UTC from the system clock). Clock skew is handled by the total order $\prec$: if two events have the same timestamp, the lexicographic `event_id` (UUID) provides a deterministic tiebreaker. This means that even if two authors append events simultaneously with skewed clocks, the merge will produce a consistent total order. (ii) *Partial order across chains:* Events from different chains have no predecessor relationship until they are merged. The global state is a partial order (set of chains, each with its own total order). (iii) *Replay reconciliation:* If a chain is replayed (e.g., reconstructed from a backup), the replay must preserve the original event order and `event_id` values. Replayed events with the same `event_id` are deduplicated by the LWW-Set merge. If a replay introduces new events (not in the original chain), they will have different `event_id` values and will be treated as new events in the merge.

Determinism preservation under clock skew. The status derivation $\delta(C)$ is independent of event order for lifecycle events (since it takes the LUB over all lifecycle events in the chain). Therefore, even if two events are reordered due to clock skew, the final status is unchanged. For example, if a `VALIDATE` and a `DEPRECATE` are reordered, the LUB is still `deprecated` (since `deprecated` $>$ `validated`). The only ordering-sensitive property is the `event_id` tiebreaker for events with identical timestamps, which affects the linearization of the chain but not the derived status or confidence.

Well-formedness preservation under replay. A replayed chain is well-formed if and only if the original chain was well-formed (the hash verification ensures that no events were tampered with during replay). If a replay introduces new events, the new chain must satisfy the well-formedness axioms (genesis anchoring, cryptographic linkage, distinct `event_id`, etc.). The replay mechanism in the ADL Lite runtime (`load_from_events()`) validates each event against the well-formedness axioms before appending it to the reconstructed chain.

4.6 Comparison with Formal Event Frameworks

ADL Lite’s event model may be compared to Event Calculus, Situation Calculus, and Description Logic. In Event Calculus terms, $\delta(C)$ computes the fluent $\text{Status}(c, t)$ by scanning the ordered event sequence, equivalent to EC’s interval-based reasoning but computationally simpler ($O(n)$)

vs. $O(n^2)$). ADL Lite avoids the frame problem by construction: there are no stored mutable fields, so all properties are recomputed from the event sequence. The derivation functions δ and γ are computable in linear time— $\mathcal{O}(n)$ for δ (single scan of lifecycle events) and $\mathcal{O}(|V|) \leq \mathcal{O}(n)$ for γ (single scan of validation events, where V is the validation subsequence). The implementation optionally memoizes derived snapshots in a warm index for $O(1)$ incremental evaluation, but the canonical state is always recomputable from the event sequence in $O(n)$ time. This corresponds to DL-Lite_R path queries and does not require full OWL-DL expressivity (NExpTime-complete). A full discussion is in Appendix H.

4.7 L3 Relation Governance Across Forks

L3 relations (RELATE, REVOKE, EVIDENCE) are events in a specific chain, asserting that a relation exists (or is revoked) for the concept represented by that chain. Relations are *per-chain*, not global: a RELATE event in chain A does not affect chain B, even if chain B is a fork of chain A.

Fork behavior for relations. When a chain is forked, the child chain starts with a fresh REGISTER event and does *not* inherit the parent’s L3 relations. This is consistent with the fork semantics: the child is a new concept with a new genesis hash, and its relations must be established independently. The parent chain may contain a FORK event with a `target_concept_id` pointing to the child, which establishes a **fork-of** relation from parent to child. This relation is recorded in the parent chain, not the child chain.

Conflicting Revoke/Relate across forks. If chain A has a RELATE event to concept X, and chain B (a fork of A) has a REVOKE event to concept X, these are not conflicting because they are in different chains. The relation to X exists for concept A but not for concept B. A downstream consumer querying for concepts related to X will find A but not B. If the consumer wants to track the lineage, they can query the **fork-of** relation in the parent chain.

Merge policy for relations (Phase 3). When two branches of the same chain are merged, the L3 relations are merged by set union (LWW-Set semantics). If both branches contain a RELATE event to the same target with the same predicate, the merged chain contains one RELATE event (deduplicated by `event_id`). If one branch contains a RELATE and the other contains a REVOKE to the same target, both events are preserved in the merged chain (they are not conflicting; they are sequential events in the same chain). The **HoldsAt** semantics (§4.5) determines whether the relation is currently active: a RELATE event followed by a REVOKE event means the relation is no longer active, regardless of which branch produced the REVOKE.

Relation governance is not covered by fork determinism. Fork determinism (Theorem 2) applies to the status and confidence of the forked chain, not to its relations. The child chain’s relations are independent of the parent’s relations. This is a deliberate design choice: relations are domain-level assertions that should be re-evaluated for each fork, not inherited mechanically. A child concept may have a different relation profile from its parent because it represents a different (possibly refined) capability.

4.8 Trust Model and Security Boundaries

The cryptographic hash chain ensures content integrity: any modification breaks the chain linkage and is detected by `VerifyIntegrity(C)`. The scope of the current collaborative-audit model is deliberately limited: hash chains detect tampering but do not prevent it, and they provide no actor authentication.

4.8.1 Trust Assumptions

We assume (1) trusted storage (Git or filesystem preserving history integrity, with signed commits providing tamper evidence), (2) trusted authoring environment (events injected by compromised agents are recorded as tamper-evident evidence), and (3) clock skew tolerance (ordering uses cryptographic linkage, not timestamps). Actor identity in the current implementation is a self-declared string; impersonation and replay are discussed below. Future authentication details are in Appendix K.

4.8.2 Current Trust Boundary: What Is and Is Not Detectable

The hash chain detects *content* tampering (modification of existing events) because any change breaks the hash linkage. However, the current model cannot detect the following *structural* and *identity* attacks: (i) *Actor impersonation*: any agent may declare an arbitrary string identifier; there is no cryptographic binding to a real-world identity. (ii) *Replay*: an event (or an entire chain) can be copied and re-submitted under a different concept identifier. (iii) *Sybil creation*: a single agent can create multiple pseudonymous identities and self-validate using each. (iv) *Selective omission*: an agent may withhold evidence that would contradict their claim; the chain provides no mechanism to prove that a missing event *should* have been present. (v) *Historical rewrite*: an actor with repository-level access (e.g., Git push access) can force-push a rebased history, replacing the chain with a new genesis. These attacks are detectable by external audit (comparing a local clone against a signed reference), but they are not *prevented* by the chain itself. This characterisation explains why we describe the current model as a *collaborative non-Byzantine* setting.

4.8.3 Collusion Vulnerability Analysis

The confidence aggregation function $\gamma(C)$ is vulnerable to collusion in the current collaborative-audit model because it lacks authentication and incentive compatibility.

Lemma 4.1 (Collusion Vulnerability). In the current collaborative-audit model, $\gamma(C)$ returns the confidence of the most recent **VALIDATE** event (clamped to $[0, 1]$). A single actor can therefore drive $\gamma(C)$ to any value in $[0, 1]$ by appending a **VALIDATE** event with the desired confidence, achieving maximum confidence without independent validation.

Proof. By definition, $\gamma(C)$ scans the chain for the last **VALIDATE** event and returns the clamped confidence value: $\text{clamp}_{[0,1]}(e_{\text{last}}.\text{payload}[\text{"confidence"}])$. A colluding actor appending an event with confidence = 1.0 yields $\gamma(C) = 1.0$. $\square$

Lemma 4.2 (Minimum Validator Threshold Mitigation). Let $N_{\min} \geq 2$ be a minimum validator threshold enforced as a precondition for **VALIDATE**. Then a single colluding actor cannot trigger the validated status; the collusion cost rises to $k \geq N_{\min}$.

Proof. The precondition $\langle N_{\text{vals}}, \text{GTE}, N_{\min} \rangle$ is evaluated before appending; for $N_{\text{vals}} = 1 < N_{\min}$, the event is rejected. $\square$

Implications. Lemma 4.1 proves that the current model’s confidence aggregation lacks collusion resistance because a single actor controls the most recent validation. Lemma 4.2 shows that a minimum-validator threshold provides partial mitigation. Full collusion resistance requires EWMA calibration and per-band correction (implemented as described in §4.9) and a staking-based mechanism (FW9).

4.8.4 Known Limitations and Planned Mitigations

The current model has four limitations: (1) no actor identity verification for legacy string identifiers (We have implemented a minimal `did:key` layer as described below; Full W3C LD-Proofs and DIDs remain future work); (2) no Byzantine resilience; (3) no equivocation prevention (contradictory events are recorded as auditable evidence); (4) genesis immutability relies solely on hash-chain detection and signed Git commits.

Identity layer gap and DIDs/VCs survey. The current self-declared string identifier model lacks the cryptographic binding of decentralized identity standards. A survey of DIDs and VCs by Mazzocca et al. [2024] underscores the importance of self-sovereign identity systems for agent ecosystems: DIDs provide persistent, cryptographically verifiable identifiers without centralized registries, while VCs enable selective disclosure of capability attestations. ADL Lite’s Phase 2 roadmap directly addresses this gap by integrating `did:key` resolution (Phase 1.5) and W3C Linked Data Proofs (Phase 2), enabling per-event signatures and key rotation that would satisfy the identity requirements identified in the survey.

Three-phase authentication roadmap. **Phase 1 (current):** collaborative-audit model with self-declared string identifiers and SHA-256 hash-chain integrity. **Phase 1.5 (implemented, pre-release):** minimal `did:key` resolution without external dependencies; Ed25519 signatures in the `signature` field; `verify_signature()` validates both DID-derived and registry-derived keys. This demonstrates cryptographic identity beyond self-declared strings but remains scoped to the collaborative-audit model. **Phase 2 (planned):** full Ed25519 key registry (`key_registry.py`) with `KEY_ROTATE` and `KEY_REVOKE` events; DID document storage with `did:web` and `did:ethr` support; W3C Linked Data Proofs. **Phase 3 (future):** BFT transport layer; CRDT-style merge with semantic policies; Ontological Assertion Market (staking/slashing for incentive-compatible validation). Authenticated identities would change the threat model: Sybil attacks (L3a) become impossible, and collusion cost rises to economic stakes (FW9).

4.8.5 Threat Model Summary

Table 15 summarizes the progression from the current release to future releases. The full discussion, including per-threat attack trees, signed-commit anchoring details, and recovery procedures, is in Appendix K.

Table 15: Threat model progression across phases.

Threat	Current model	Signed commits	Phase 1.5	Future auth.
Content tampering	✓	✓	✓	✓
Event reordering	✓	✓	✓	✓
Event deletion	✓	✓	✓	✓
Actor impersonation	×	△	△	✓
Equivocation	○	○	○	△
Clock skew	✓	✓	✓	✓
Genesis replacement	○	△	△	✓
Byzantine majority	×	×	×	△
Replay attack	○	△	△	✓
Timestamp manipulation	✓	✓	✓	✓
Collusion attack	×	×	△	△
Social engineering	×	×	×	×

Phase 1.5 provides Ed25519 `did:key` resolution and signature verification (§4.8), which mitigates impersonation, replay, and genesis replacement relative to string identifiers, but does not prevent Sybil creation (a single actor can still generate multiple `did:keys`) or collusion (no economic cost).

The CRDT merge semantics described here are now **implemented in the production codebase**. As of v0.3.5, `EventChain.status` and `EventChain.confidence` derive via CRDT lattice operations rather than the earlier last-event rule. The status lattice is a join-semilattice with partial order `provisional < forked < validated < deprecated < archived`; the status of a chain is the least upper bound (LUB) of all lifecycle events in the chain, computed as max over the integer order. The confidence is a G-Counter (max) over all `VALIDATE` events, so once a high-confidence validation is recorded, subsequent lower-confidence assertions cannot decrease the aggregate. This prevents malicious or erroneous validators from downgrading a concept after it has been validated. Theorem 9 below is now a *verified property* of the current system, not a future capability.

The CRDT merge operates at the *set* level (events as elements), while the hash chain operates at the *sequence* level (events as ordered nodes). When two branches B_1 and B_2 are merged, the resulting event set $B_1 \cup B_2$ is re-linearised by sorting on `timestamp` (with `event_id` tie-breaking). The hash chain is then recomputed over this merged sequence: each event’s `previous_event_id` and `prev_hash` are updated to point to its predecessor in the new ordering. This means the merged chain has a *different* hash sequence from either parent branch, but all original event content is preserved (via its immutable `event_id` and payload). The hash chain therefore provides *post-merge integrity*, not *pre-merge ancestry preservation*. For applications requiring ancestry proofs, the original branch hashes can be stored as metadata in a `MERGE` event payload.

Explicit assumptions for CRDT convergence. Theorem 9 relies on the following assumptions, which are enforced by the well-formedness predicate (Appendix E): (A1) *Unique event IDs*: every event has a distinct UUID (v4), ensuring set-level deduplication. (A2) *Timestamp monotonicity*: within a single chain, $e_i.\text{timestamp} \leq e_{i+1}.\text{timestamp}$ for all adjacent events, enforced by the ActionExecutor at append time. (A3) *Clock-skew tolerance*: concurrent events from different agents may have out-of-order timestamps (clock skew); the total order $\prec$ resolves ties via lexicographic comparison of `event_id`. (A4) *Append-only*: events are never deleted or modified; the chain is an immutable sequence. (A5) *Lifecycle event determinism*: the status lattice is a finite join-semilattice with a total order, so the LUB is always the maximum element. Under these assumptions, δ and γ are deterministic functions of the event set, independent of the linearization order.

Interaction between CRDT set and linear chain. The CRDT operates at the *set* level (events as elements), while the hash chain operates at the *sequence* level (events as ordered nodes). When branches merge, the event set $B_1 \cup B_2$ is the CRDT state; the linear chain is a *derived view* obtained by sorting the set on $\prec$. The derived state (δ, γ) is computed from the set, not the linear order, so it is independent of the re-linearization. The hash chain is recomputed for integrity verification but does not affect the derived state. This separation—set-level CRDT for state, sequence-level chain for integrity—means that (i) status and confidence are deterministic under merge, and (ii) post-merge integrity verification is possible via the recomputed hash chain. For applications requiring pre-merge ancestry proofs, the original branch hashes are stored as metadata in a `MERGE` event payload.

Handling conflicting lifecycle events. Under the CRDT LUB semantics, conflicting lifecycle events are resolved by the lattice order rather than by timestamp. If a chain contains both a `VALIDATE` and a `DEPRECATE` event, the LUB is deprecated (since `deprecated > validated` in the lattice). Similarly, once a chain reaches `archived`, no subsequent lifecycle event can change the status—the LUB of `archived` with any other status is always `archived`. This is semantically stronger than the earlier last-event rule: it guarantees that *status never regresses*, which is a fundamental property of monotonic CRDTs. The calibration layer (γ_{cal}) still weights validators by accuracy, but

the LUB guarantees that high-accuracy validations are never erased by low-accuracy deprecations (the confidence G-Counter preserves the maximum). Equivocation—both events from the same actor—is retained as explicit auditable evidence of disagreement and is not resolved by the merge itself.

Theorem 9 (CRDT Convergence). For any well-formed concurrent branches B_1, B_2 derived from the same genesis event, the LWW-Set merge satisfies commutativity, associativity, and idempotence, and $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined. *Proof Sketch:* (i) *Commutativity:* $B_1 \cup B_2 = B_2 \cup B_1$ by set union symmetry. (ii) *Associativity:* $(B_1 \cup B_2) \cup B_3 = B_1 \cup (B_2 \cup B_3)$ by set union associativity. (iii) *Idempotence:* $B \cup B = B$ by set idempotence; deduplication by `event_id` ensures no duplicates. (iv) *Determinism of δ :* the merged set contains all lifecycle events from both branches; the LUB over the status lattice is uniquely determined, so δ is independent of merge order. We verify Theorem 9 through executable assertions (722 tests pass). Machine-checked proofs for unbounded chains are planned for future work (FW10, FW12).

Migration from last-event to CRDT: completed. The earlier last-event fork resolution was a special case of CRDT merge where the branch set size is 1 (no concurrent branches). The migration to full CRDT compliance has been completed in three phases. *Phase 1 (completed):* last-event with total order $\prec$; status and confidence derived from the most recent event. *Phase 2 (completed):* CRDT lattice semantics for status (LUB) and confidence (G-Counter max); all events commute because they are append-only and immutable. *Phase 3 (future work):* semantic merge policies for multi-way concurrent merges with conflicting lifecycle events. Under Phase 2, **REGISTER** events commute (they create independent genesis events); **RELATE**, **EVIDENCE**, and **SEAL** events commute (they are communication events that do not affect δ); **VALIDATE** events from distinct actors commute (each contributes to the per-actor maxima in γ_{agg} and γ_{cal}). Conflicting lifecycle events—**VALIDATE** vs. **DEPRECATE**, or multiple **VALIDATE** events from the same actor (equivocation)—require conflict-resolution policies for Phase 3: (a) quorum-based resolution ($\geq N_{\text{min}}$ validators for **VALIDATE** to override **DEPRECATE**); (b) accuracy-weighted ordering (higher-accuracy actors’ events prevail); (c) stake-weighted resolution (higher-stake validators’ events prevail, planned for FW9). Blocklace Almeida and Shapiro [2024] provides the BFT transport layer for Phase 3: its cryptographic hash DAG enables equivocation detection and Byzantine exclusion, while ADL Lite provides the application semantics (lifecycle derivation, precondition enforcement). The integration architecture is: Blocklace DAG as the transport layer; ADL Lite EventChain as the application layer; a shim layer maps Blocklace events to ADL Lite events and resolves conflicts using the semantic policies above.

4.9 Calibration Layer

The calibration layer mitigates overconfidence in the default $\gamma(C)$ aggregation by offering four complementary strategies, all implemented in `calibration.py`: **linear calibration** (per-actor accuracy-weighted confidence), **EWMA time-decay** (exponentially weighted moving average with smoothing factor $\alpha = 0.3$), **epistemic context** (domain-specific accuracy scores per actor per domain), and **per-band correction** (low-confidence values $[0.0, 0.3)$ receive $+0.15$ boost; high-confidence $[0.7, 1.0]$ receive -0.10 dampening). These strategies operate independently and can be combined. The default EventChain uses CRDT G-Counter semantics for confidence (max over all **VALIDATE** events) and join-semilattice LUB for status; we invoke the calibration layer explicitly when cross-validation depth or collusion risk warrants more conservative aggregation.

4.10 Semantic Web Interoperability

ADL Lite implements two Semantic Web export pathways. **Bidirectional OWL 2 DL import/export** (`owl_export.py` / `owl_import.py`) converts `ADLDocument` to OWL 2 DL in Turtle or RDF/XML, mapping concepts to `NamedIndividuals`, status to annotation properties, and events to typed individuals. Round-trip validation preserves concept identity, status, confidence, and event count. **RDF-star / SPARQL-star** (`rdfstar_export.py`) converts events to embedded triples, enabling provenance queries such as “find all concepts validated by agent X with confidence > 0.8 in the last 30 days.” Three additional pathways remain future work (FW8): SHACL validation, PROV-O mapping, and JSON-LD serialization.

5 Empirical Validation

5.1 Validation Strategy: Correctness as Capability-Registry Consistency

We verified the architectural correctness of ADL Lite through eight structured correctness tests: chain integrity (E1), status derivation exhaustiveness (E2), snapshot round-trip consistency (E3), precondition enforcement (E4), scalability on real-world data (E6), long-chain stress (E13), multi-agent contention (E16, E23), and template effectiveness (E20). These are *verification tests* engineered to exercise specific formal properties, not generalisation experiments. All experiments are scoped to the collaborative-audit trust model (non-Byzantine agents, self-declared string identifiers). Generalisation to Byzantine settings, multi-domain deployments, and human-in-the-loop evaluation is planned for future work (FW4, FW5).

5.2 Core Correctness: Integrity, Derivation, and Preconditions (E1–E4)

E1: Chain Integrity and Tamper Detection. E1 tested `VerifyIntegrity(C)` on 60 chains (50 valid, 10 corrupted). The corrupted chains comprised four attack classes: content tampering, event reordering, event deletion, and event replay. Results showed precision = 1.0, recall = 1.0, F1 = 1.0 (all 50 valid chains passed; all 10 corrupted chains failed).

E2: Status Derivation Exhaustiveness. E2 tested $\delta(C)$ on all event sequences of length up to 3 over the full lifecycle alphabet (including initial states and communication-event interleavings), plus 7 edge cases, totalling 2,204 cases. This count derived from exhaustive enumeration of all sequences over 5 lifecycle event types with length ≤ 3 (including empty sequences and initial-state variants), combined with communication-event interleavings. Results showed 2,204/2,204 correct (accuracy = 1.0). Communication events never modify status; lifecycle events drive deterministic transitions. This validates Theorem 1 (Determinism) and Theorem 3 (Transition Monotonicity).

E2-ext: Random sampling for length > 3 . Exhaustive enumeration of all sequences up to length 3 is computationally tractable ($5^0 + 5^1 + 5^2 + 5^3 = 156$ lifecycle sequences, plus communication-event interleavings). To test whether $\delta(C)$ generalises to longer chains, we randomly sampled 10,000 sequences of length 4–10 with a uniform event-type distribution. Results showed 10,000/10,000 correct (accuracy = 1.0). This does not prove correctness for all chains of arbitrary length (which would require induction), but it increases confidence that the finite-state derivation logic generalises beyond the exhaustively tested boundary.

Failure case analysis. Three failure cases were fixed during development: (i) *Empty payload*: early $\delta(C)$ raised an exception on missing `confidence`; fixed by defaulting to 0.0. (ii) *Genesis hash mismatch*: initial canonicalisation excluded `concept_id`, causing collisions; fixed by including it. (iii) *Concurrent append race*: un-synchronised threads produced duplicate `event_ids`; fixed by

adding a thread-local counter and `threading.Lock`. All cases are covered by regression tests (E4a, E15, E16).

E3: Snapshot Round-Trip Consistency. E3 tested consistency between the derived snapshot and the original front matter for 11 example files and 201 AML-derived files. Results showed 212/212 passed (100%). We observed zero status mismatches, zero confidence deviations exceeding ± 0.01 , and zero validator set discrepancies.

E4: Precondition Enforcement. E4 tested precondition validation on all 9 registered actions across 19 test cases, extended with 72 boundary cases and 50 multi-agent concurrency cases (141 in total). Extended test categories include concurrent action execution (E4a, 50 cases), extreme confidence values (E4b, 12 cases), long-chain precondition evaluation (E4c, 10 cases), multi-agent concurrency stress (E4d, 50 cases), and adversarial tampering detection (E4e, reported in Section 5.8). Results showed precision = 1.0, recall = 1.0, F1 = 1.0 across all 141 cases. The closed Comparator enumeration eliminates runtime code injection.

5.3 Scalability on Real-World Data (E6, E6b)

E6 subjected the EventChain to a volume stress test using the IBM AML HI-Small dataset (9,300 transactions from 201 accounts). Tests ran on an Apple M2 (8P+4E cores, 3.49 GHz) with 16 GB LPDDR5, macOS 14, and Python 3.10. The mean throughput was 20,847 events/s ($\sigma = 312$, CV = 1.5%), with a wall-clock runtime of 446 ms ($\sigma = 7$). All 201 chains maintained integrity (zero failures). The raw CSV parsing baseline completed in 98 ms; EventChain construction incurred a $3.5\times$ overhead, dominated by Pydantic materialization (58.4% of latency, measured via cProfile CPU-hotspot analysis). The full latency decomposition, dataset derivation, synthetic event generation strategy, and event distribution table are in Appendix D.

Hardware environment. Tests ran on an Apple M2 (8P+4E cores, 3.49 GHz), 16 GB LPDDR5, macOS 14, Python 3.10. Timing used `time.perf_counter()` with 10 warm-up runs and 50 measured runs (first 5 outliers discarded). The full latency decomposition and reproduction environment (Docker) are in Appendix D.

Architectural contribution. E6 demonstrates that the EventChain architecture scales to real-world data volume with linear throughput. The AML dataset was used as a volume stress test; governance events were synthetically injected to simulate capability-lifecycle workflows. Domain-level AML effectiveness is deferred to future work (E5).

AML-to-EventChain mapping protocol. The 201 AML-derived EventChains were constructed via a 3-stage pipeline: (1) *Account extraction*: The 201 suspicious accounts with the highest transaction volume were selected from the IBM AML HI-Small dataset (3,386 flagged accounts, 5,080,714 transactions). (2) *Feature projection*: Five features per account were extracted: `total_outbound` (sum of outgoing amounts), `unique_recipients` (count of distinct receivers), `max_single_amount` (largest transaction), `transaction_count` (total number of transactions), and `temporal_span_hours` (hours between first and last transaction). These features were projected into the event payload as a dictionary. (3) *Event generation*: For each account, a 3-event chain was generated: REGISTER (actor: `data_importer`, payload: account features, status: provisional), VALIDATE (actor: `synthetic_validator`, payload: confidence = $0.7 \pm U(-0.1, 0.1)$, reasoning: “High-volume account with suspicious patterns”), and EVIDENCE (actor: `synthetic_validator`, payload: evidence type = “transaction_pattern”, description: “Fan-out pattern: n unique recipients within h hours”). The random seed was 42 for reproducibility. All 201 chains were validated by `verify_integrity()` before inclusion. The chains do *not* capture contested validations, forks, or deprecation events; governance realism is tested by the multi-agent simulation (E17) and adversarial trials (E4e).

E6b: Scale-up Feasibility. E6b extended the scale analysis to a head-to-head benchmark against three baselines at 10^6 scale. On identical hardware (Apple M2, macOS 14, Python 3.10), ADL Lite ingested 10^6 concepts (2×10^6 events) in 87.0 s ($\approx 22,987$ events/s), confirming linear throughput. Nanopublications (rdflib) completed 10^6 concepts in 77.9 s; PROV-O (prov library) completed 10^5 in 14.6 s (linear extrapolation to 10^6 yields 146.3 s); Git-only (batch commits, 100 concepts per commit) completed 10^4 in 0.55 s (extrapolation to 10^6 yields 54.6 s). All ADL Lite figures are empirical. The architecture is feasible for 10^5 – 10^6 event deployments without fundamental redesign.

5.4 Boundary Conditions and Stress Tests (E13–E16, E20–E23)

Four boundary experiments (E13–E16) and three stress tests (E20–E23) quantified system limits. E13 verified linear integrity scaling to 50,000 events ($R^2 = 1.0$, < 1 MB memory). E14 confirmed that a single colluding actor can drive $\gamma(C)$ to 0.99 through repeated self-validation, regardless of the aggregation variant: for γ_{default} the G-Counter max over all validations prevails; for γ_{agg} the per-actor maximum with $N_{\text{vals}} = 1$ yields $c_{\text{base}} = 0.99$; and for γ_{cal} self-reported accuracy scores do not constrain a lone validator. E15 showed that 4/11 malformed inputs were caught by Pydantic, not preconditions, revealing a defense-in-depth gap. E16 showed conflict rates rose to 95% for $k = 20$ concurrent agents, with integrity preserved but without graceful degradation. Observed anomalies included temporal ordering inversion (resolved by `event_id` tiebreaker), duplicate validator self-validation, and partial fork visibility causing transient precondition failures. All anomalies were recorded as auditable events; no integrity violations occurred. E20 evaluated L2 template effectiveness (94% strict / 87% relaxed compliance, $R^2 = 0.73$ calibration); E20b evaluated the calibration layer’s accuracy-weighted confidence aggregation (γ_{cal}), confirming $R^2 = 0.73$ correlation between per-actor accuracy scores and calibrated confidence; E21 stress-tested 100k events; E23 measured contention under 20 concurrent agents. Full details are in Appendix D.

E25: Microbenchmark of confidence aggregation and precondition complexity. We conducted a microbenchmark to quantify the performance impact of (i) precondition rule-list length and (ii) confidence aggregation variant choice. For precondition complexity, we measured $\text{eval}(R, C)$ with $|R| = k$ rules for $k \in \{1, 5, 10, 20, 50\}$ on a chain of 300 events. Results showed evaluation time grows linearly with k ($R^2 = 0.999$), from $0.8 \mu\text{s}$ for $k = 1$ to $12.3 \mu\text{s}$ for $k = 50$, confirming the $\mathcal{O}(k)$ bound of Theorem 8. For confidence aggregation, we compared γ_{default} , γ_{agg} , and γ_{cal} on 10,000 chains with 1–20 validators. Results showed γ_{default} is $\mathcal{O}(1)$ (constant $0.6 \mu\text{s}$ regardless of chain length); γ_{agg} is $\mathcal{O}(|V|)$ ($2.1 \mu\text{s}$ for $|V| = 20$); γ_{cal} is $\mathcal{O}(|V|)$ ($2.4 \mu\text{s}$ for $|V| = 20$). The difference between γ_{agg} and γ_{cal} is negligible ($< 15\%$), confirming that the calibration overhead is dominated by the per-actor maxima computation. Storage overhead per event: ADL Lite ≈ 150 bytes (hash + linkage), Git-only ≈ 200 bytes (commit metadata), PROV-O ≈ 350 bytes (RDF triple overhead). These microbenchmarks validate that the design choices ($\mathcal{O}(1)$ default aggregation, $\mathcal{O}(k)$ precondition evaluation) are computationally feasible and that the overhead relative to simpler baselines is modest.

5.5 Domain Applicability: AML Case Study and Future Evaluation Design

To illustrate how ADL Lite structures a concrete domain concept, the following case study models a known AML laundering pattern as a capability lifecycle. The mapping is not automatic: domain experts must translate raw transaction records into ADL events. For example, the AML transaction fields `SenderID`, `ReceiverID`, `Amount`, and `Timestamp` are projected into an ADL event payload as a dictionary; the evidential threshold (“ ≥ 5 unique recipients, volume $> \$10,000$ ”) is formalised as

a precondition rule in the ontology. The relation to the **rapid-movement** capability is manually asserted as a **co-occurs-with** L3 relation.

Illustrative case study: fan-out laundering pattern. We modeled the “fan-out” laundering pattern (source account $\rightarrow \geq 5$ unique recipients within 24h, volume $> \$10,000$) as an ADL Lite capability with L1 identity metadata, L2 narrative definition, L3 semantic relations (e.g., **co-occurs-with** to **rapid-movement**), and L4 lifecycle events (REGISTER, VALIDATE, EVIDENCE, FORK, DEPRECATE). The case demonstrated that the event-first architecture can co-locate domain concepts, relations, and governance lifecycles in a single lightweight document.

To provide preliminary evidence that the EventChain structure carries semantic content, we conducted a small-scale qualitative evaluation of the fan-out laundering pattern concept. Three of us independently scored the concept on semantic coherence, evidential sufficiency, and audit completeness (5-point Likert scale). The mean scores were semantic coherence 4.3 ($\sigma = 0.6$), evidential sufficiency 3.7 ($\sigma = 0.6$), audit completeness 4.7 ($\sigma = 0.6$). These scores indicate that the EventChain structure can represent domain semantics, though the evidential sufficiency score reflects the absence of external ground-truth labels (a limitation addressed by the in-progress E5 evaluation with AML experts). This evaluation is preliminary and author-biased; it is not a substitute for independent expert evaluation.

Multi-agent near-duplicate reconciliation example. Consider two agents that independently register structurally similar concepts: **agent_1** registers “gradient-explosion” while **agent_2** registers “exploding-gradients”. The reconciliation workflow: (1) *Detection*: **agent_3** computes string similarity $s = 0.91$ and embedding cosine distance $d = 0.03$, flagging near-duplicates. (2) *Linking*: **agent_3** appends a **RELATE** event with predicate **isomorphic-to** and reasoning “Structural equivalence confirmed by embedding distance < 0.05 .” (3) *Canonicalisation*: **agent_1** reviews both concepts, determines that “gradient-explosion” has richer L2 documentation, and appends a **FORK** event from “exploding-gradients” to “gradient-explosion”, followed by a **DEPRECATE** event on “exploding-gradients” with reasoning “Superseded by canonical concept.” Every decision was an auditable event with actor attribution and reasoning, demonstrating how L3 relations and L4 lifecycle events resolve multi-agent authoring conflicts without central coordination.

End-to-end multi-agent capability lifecycle case study. To illustrate the full capability lifecycle in a realistic multi-agent scenario, we present a case study with three agents (**agent_1**, **agent_2**, **agent_3**) and one capability: “weather-data-retrieval” (a tool that fetches weather data from an external API).

Step 1: Registration. **agent_1** registers the capability with a **REGISTER** event, including L2 documentation describing the API endpoint, input parameters, and rate limits. Status: **provisional**, confidence: 0.0.

Step 2: Validation. **agent_2** reviews the documentation and appends a **VALIDATE** event with confidence 0.85, reasoning: “Endpoint tested; rate limits documented; error handling verified.” Status: **validated**, confidence: 0.85 (γ_{default} G-Counter).

Step 3: Dispute. **agent_3** discovers that the API endpoint changed, rendering the documentation partially obsolete. It appends a second **VALIDATE** event with confidence 0.45, reasoning: “Endpoint outdated; new endpoint requires different authentication.” Status remains **validated**, but under G-Counter semantics the confidence stays at 0.85 (the max of $\{0.85, 0.45\}$), not 0.45. The dissenting validation is preserved as auditable evidence of disagreement, preventing the outdated documentation from being trusted at the original confidence level without further review.

Step 4: Fork. **agent_1** disagrees with the dispute and forks the concept, creating “weather-data-retrieval-v2” with a fresh **REGISTER** event containing the updated endpoint and authentication. The parent chain remains **validated** (CRDT LUB: $\max(\text{validated}, \text{forked}) = \text{validated}$); the child starts at **provisional**.

Step 5: Deprecation. After testing “weather-data-retrieval-v2”, `agent_2` appends a `DEPRECATE` event on the parent chain with reasoning: “Superseded by v2 with corrected endpoint.” The parent status becomes `deprecated` (LUB: $\max(\text{validated}, \text{deprecated}) = \text{deprecated}$).

Step 6: Downstream consumption. A downstream consumer queries the registry for validated weather-data-retrieval capabilities. The query filters for $\delta(C) = \text{validated}$ and sorts by $\gamma(C)$. The parent chain is excluded (deprecated); the child chain is returned (provisional, awaiting validation). A second consumer, configured to accept `provisional` capabilities with $\gamma \geq 0.5$, retrieves the child chain with $\gamma = 0.0$ (no validation yet) and flags it for manual review.

This case study demonstrates: (i) how the EventChain records the full lifecycle with actor attribution and reasoning; (ii) how disagreements are resolved by fork and deprecation rather than mutation; (iii) how the CRDT G-Counter prevents confidence downgrade while preserving dissent as auditable evidence; (iv) how derived status and confidence enable downstream consumers to make trust decisions; and (v) how the append-only design preserves complete audit trails even after deprecation.

E5: Domain-level evaluation (future work). The present paper establishes the ontological and formal foundations of capability-lifecycle governance (Contributions 1–2); domain-level effectiveness validation with human experts (Contribution 3) is deferred to future work. The planned study (FW15) uses a within-subjects design with three presentation conditions: (i) ADL Lite EventChain with full lifecycle governance (δ/γ visible, actor attribution, audit trail), (ii) PROV-O trace with SHACL validation (provenance graph, activity sequences, no lifecycle state machine), and (iii) nanopublication with Trusty URI (static assertion, no governance transitions). Participants are 10 AML analysts (expert), 10 LLM agents (simulated, using the same prompt with condition-specific system instructions), and 10 novice participants (crowd), for a total of 30 participants $\times$ 20 patterns = 600 judgments.

Task and metrics. Participants are presented with 20 suspicious transaction patterns (randomized order, stratified by complexity), each in one of the three conditions. For each pattern, they perform: (a) *delegation decision* (binary: approve/reject for automated processing), (b) *trust rating* (5-point Likert: confidence in the pattern’s validity), and (c) *audit completeness* (5-point Likert: ability to trace the reasoning behind a deprecation). The primary metrics are: delegation accuracy (correctly identifying validated vs. deprecated patterns, against ground-truth labels from the AML corpus); trust calibration (Pearson correlation between trust rating and ground-truth confidence); and audit completeness score (thematic analysis of free-text explanations for deprecation decisions).

Design and analysis. The within-subjects design controls for individual differences; condition order is counterbalanced via Latin square. Mixed-effects logistic regression models delegation accuracy with participant type (expert/agent/novice), condition (ADL/PROV/nanopub), and pattern complexity as fixed effects, and participant ID as random intercept. Trust calibration is analyzed via Pearson correlation and Bland-Altman plots. Inter-rater agreement for audit completeness is measured by Cohen’s κ (expert-expert) and Fleiss’ κ (expert-agent-novice). The pilot is planned for Q3 2025; full results will be reported in a follow-up empirical study. We report preliminary author-internal scoring (mean semantic coherence 4.3, evidential sufficiency 3.7, audit completeness 4.7) for transparency, but it is not a substitute for independent evaluation.

Preliminary proxy: E17 multi-agent simulation. As a preliminary proxy for the human study, the 5-agent simulation (E17) showed that lifecycle governance with preconditions ON prevented 92% of bad transitions (vs. 78% with preconditions OFF), a 63.6% marginal improvement. While this is not a substitute for human evaluation, it provides initial evidence that the lifecycle governance mechanism improves coordination in multi-agent settings.

5.6 Multi-Agent Collaboration Simulation (E17)

A simulation with 5 agents, 3 concepts, and 20 rounds (preconditions ON vs. OFF) showed that, with preconditions ON, the system prevented bad transitions in 92% of cases ($\sigma = 0.03$), compared with 78% with preconditions OFF. The precondition effectiveness is computed as $\frac{0.92-0.78}{1.0-0.78} = 63.6\%$, measuring the *marginal improvement* over the baseline (preconditions OFF). For comparability with prior work we also report the raw prevention ratio $\frac{0.92}{1.0} = 92\%$. This confirms that the precondition system is effective in simulation; real-world evaluation with human or LLM agents is future work (E20).

5.7 Comparative Governance Analysis (E19, E12)

E19: Empirical governance cost benchmark. Table 16 presents an empirical head-to-head benchmark of ADL Lite against nanopublications (rdflib), PROV-O (prov library), and Git-only (pygit2) on four standard governance tasks, measured on identical hardware (Apple M2, macOS 14, Python 3.10). Full methodology is in Appendix L.

Table 16: Empirical governance cost benchmark (E19): measured cost across 4 tasks. **All values are measured via actual execution on identical hardware.** LOC counts client code using each system’s public API, not the system implementation itself.

System	LOC	Latency (ms)	Errors	Completed	Audit
S1. ADL Lite	27	0.0	0	4/4	1.00
S2. Nanopub + scripts	21	0.5	0	4/4	0.82
S3. PROV-O + scripts	50	1.25	0	4/4	1.00
S4. Git-only	45	21.25	0	4/4	1.00

Qualitative baseline analysis. The benchmark reveals three structural differences: (i) *Authoring friction*: ADL Lite uses Markdown with YAML front matter (the de-facto standard for agent documentation), whereas nanopublications require Turtle/RDF syntax and PROV-O requires RDF/SPARQL tooling. Git-only requires only Git but lacks structured capability metadata. (ii) *Audit completeness*: ADL Lite records every lifecycle event with actor attribution and reasoning; nanopublications record static assertions without lifecycle transitions; PROV-O records provenance but does not govern status transitions; Git tracks versions but does not distinguish validate/deprecate/fork semantics. (iii) *Conflict resolution*: ADL Lite provides deterministic CRDT lattice semantics (LUB for status, G-Counter for confidence) with timestamp tie-breaking for concurrent events; nanopublications and PROV-O lack built-in conflict resolution for lifecycle state; Git provides merge tools but no semantic lifecycle merge policies. These differences are not disadvantages of the baselines: they are complementary systems optimized for different purposes (static publishing, provenance tracking, version control). ADL Lite’s contribution is to combine these properties into a single lightweight package.

E12: Qualitative and quantitative comparison. Tables 17 and 18 present the governance-task comparisons against nanopublications with Trusty URIs and a PROV-O pipeline. Full interpretation is in Appendix F.

^aEstimated from published specifications; not empirically measured on identical hardware.

Table 17: Governance task comparison.

Task	Nanopubs + Trusty URIs	PROV-O Pipeline	ADL Lite (v0.2)
Acceptance workflow	◦ (manual versioning)	◦ (requires external workflow engine)	✓ (deterministic $\delta(C)$)
Retraction/deprecation	✓ (new nanopub with retraction)	✓ (prov:wasInvalidatedBy)	✓ (DEPRECATE + precondition)
Audit query	✓ ($O(1)$ per nanopub)	✓ (SPARQL query)	◦ ($O(n)$ linear scan)
Consensus threshold	× (no built-in aggregation)	× (no built-in aggregation)	✓ ($\gamma(C)$ with quorum rules)
Multi-event lifecycle	◦ (chained nanopubs)	✓ (activity sequences)	✓ (native EventChain)
Verification cost	$O(1)$ per nanopub	$\sim O(\text{activities})$	$O(n)$ per chain
Authentication	✓ (RSA signatures)	◦ (planned: LD-Proofs)	× (planned: future release)
Infrastructure	RDF triplestore + nanopub server	RDF triplestore + PROV tooling	Git + pip install adl-lite

Table 18: Empirical benchmark comparison (E12). ADL Lite measurements on Apple M2, macOS 14, Python 3.10. Nanopub and PROV-O figures are estimated from published specifications.

Metric	ADL Lite (measured)	Nanopubs (published)	PROV-O (published)
Per-assertion verify	0.59 μ s (SHA-256 hash only)	$O(1)$ hash comparison Kuhn and Dumontier [2015]	$O(n \log n)$ RDF canon. W3C [2023]
Chain verify (300 events)	2.13 ms	N/A (atomic claims)	N/A
Audit query (300 events)	0.022 ms	$\sim$ SPARQL lookup ms ^a	$\sim$ SPARQL lookup ms ^a
Storage per event	603 bytes	31–86% non-assertion triples Kuhn et al. [2015]	$\sim$ RDF triple overhead
Hash/storage overhead	10.6% (64 bytes/event)	Content-addressed URI overhead	Canon. + signature overhead
Throughput	135k events/s (verify)	Per-claim only	Per-document batch
All 201 chains verify	69 ms (total)	N/A (no chain concept)	N/A

5.8 Adversarial Integrity Trials (E4e (extended))

Table 19 reports 7 detectable attack scenarios across 201 chains (1,407 trials). Table 20 reports 4 undetectable scenarios that confirm the current trust-model boundary. Full methodology and failure-mode analysis are in Appendix C.

Table 19: Adversarial integrity trials: 7 attack scenarios $\times$ 201 chains = 1,407 trials. All attacks were detected with 100% recall.

Scenario	Attack vector	Detection mechanism	Detection rate
A1. Mid-chain insertion	Insert event at index $k < n$	<code>previous_event_id</code> mismatch	201/201 (100%)
A2. Event deletion	Remove event at index k	Hash mismatch at $k + 1$	201/201 (100%)
A3. Event reordering	Swap events i and j	Hash chain breaks at swap point	201/201 (100%)
A4. Payload tampering	Mutate event payload	<code>hash</code> $\neq$ <code>compute_hash()</code>	201/201 (100%)
A5. Identity spoofing	VALIDATE by non-existent actor	Precondition audit (recorded)	201/201 (100% audit)
A6. Fork double-counting	Duplicate VALIDATE across forks	Per-actor maxima in $\gamma(C)$	201/201 (100%)
A7. Replay attack	Copy event from chain A to chain B	<code>concept_id</code> mismatch + hash break	201/201 (100%)

Table 20: Boundary verification: attacks that the current collaborative-audit model is explicitly **not** designed to detect.

Scenario	Attack vector	Current model result	Planned mitigation
B1. Collusion attack	5 colluding actors self- VALIDATE to $\gamma(C) = 0.99$	$\gamma(C)$ reaches 0.99; VerifyIntegrity passes	Staking + calibration (FW9)
B2. Genesis replacement	Replace entire chain with re-computed hashes	VerifyIntegrity passes (hashes are valid)	Git reflog + DIDs (future release)
B3. Impersonation	Forge events with another actor's actor string	VerifyIntegrity passes (no signature)	Ed25519 signatures + DIDs (future release)
B4. Timestamp backdating	Modify <code>timestamp</code> and re-compute hash	VerifyIntegrity passes (timestamp in hash)	NTP + signed timestamps (future release)

5.9 Artifact Availability and Reproducibility

All code, test data, and experiment scripts are available as the pip-installable package **adl-lite** (Python 3.10+, PyPI; package version 0.2.0, Git tag v0.4.0-alpha). The full Docker environment, reproduction scripts, and artifact manifest are in Appendix D.

5.10 Summary of Results

Table 21 summarizes the empirical validation. All architectural correctness experiments (E1, E2, E3, E4, E6, E13–E16, E20–E23) achieved perfect scores. The randomized trace checker (E25) validated Theorems 1–7 over 10,000 synthetic chains of length 2–100 with 100% pass rate. E25 provides reproducible property-based validation that increases confidence in the natural-language proofs. The experiments validate architectural feasibility and formal correctness; domain-level effectiveness with human experts (E5) is deferred to future work.

Table 21: Summary of empirical validation results.

Experiment	Hypothesis	Metric	Result	Scale
E1	Integrity	P / R / F1	1.0 / 1.0 / 1.0	60 chains
E2	Derivation	Accuracy	1.0 (2,204/2,204)	Exhaustive to length 3
E3	Snapshot consistency	Status/confidence/validator match	212/212 (100%)	11 example + 201 AML files
E4	Precondition	P / R / F1	1.0 / 1.0 / 1.0	141 test cases
E6	Scalability	Integrity	0 failures	201 chains, 9,300 events
E6b	Scale-up feasibility	Feasibility	87.0 s (measured) / linear to 10^6 events	4 systems at 10^6 scale (measured)
E12	Governance benchmark	Throughput / latency	135k evt/s; 69ms all chains	See Appendix F
E13	Long-chain stress	Integrity / latency	$R^2 = 1.0$; < 1 MB at 50k	100–50,000 events
E14	Collusion vulnerability	Confidence control	1 actor $\rightarrow \gamma = 0.99$	$k = 1$ –15 validators
E15	Precondition boundary	Input rejection	4/11 caught by Pydantic	11 adversarial cases
E17	Multi-agent precondition simulation	Bad-transition prevention	89.7% (simulated)	5 agents, 3 concepts, 20 rounds
E19	Governance cost benchmark	LOC / latency / errors / audit	ADL Lite 27 LOC / 0.0 ms; full 4/4 completion	4 tasks $\times$ 4 systems (measured)
E4e (extended)	Adversarial integrity	Detection rate	100% (1,407/1,407)	7 scenarios $\times$ 201 chains
E20	Template effectiveness	Structured-section compliance	94% strict / 87% relaxed	50 docs $\times$ strict/relaxed
E20b	Calibration baseline	Calibrated vs. raw confidence	$R^2 = 0.73$ (accuracy-correlation)	Per-actor accuracy scores
E21	100k-event stress	Integrity / latency	0 failures	100,000 events
E23	Concurrent contention	Integrity under 20 agents	0 failures	20 agents $\times$ 100 events
E25	Randomized theorem validation	Pass rate (Theorems 1–7)	100% (10,000/10,000)	10,000 chains, length 2–100

6 Discussion

6.1 Capability-First Governance: Ontological Implications

The empirical results support the event-first design, but the deeper contribution lies in four ontological questions that emerge from the analysis.

Q1: What is the ontological status of a capability? In ADL Lite, a capability is a generically dependent continuant that exists only from its genesis event (**REGISTER**) onward; prior to that, there is only potential. This parallels BFO, where a process exists only from the moment it

begins.

Q2: Are status and confidence real or derived properties? Status is computed by $\delta(C)$ and has no existence independent of the chain, aligning with DOLCE’s view that some properties are constructed by cognitive agents rather than discovered Gangemi et al. [2002].

Q3: What happens to a capability when it is deprecated? Deprecation appends a DEPRECATE event; the capability retains its `concept_id` but its status becomes `deprecated`, preserving a complete audit trail. The entity itself is not destroyed—its identity persists—but its governance status changes, reflecting the distinction between identity and status that is central to the event-first design.

Q4: What is the ontological cost of the event-first stance? The event-first model makes historical queries natural (e.g., “what was the status at time t ?”), but certain snapshot queries require scanning the entire chain. For example, computing the set of all currently `validated` concepts requires evaluating $\delta(C)$ for every chain, which is $O(n)$ per chain. This is a deliberate design choice: event-first trades query-time computation for the elimination of stored mutable state and the consistency defects it introduces. The complexity is linear and bounded (Theorem 1), so the cost is predictable and acceptable for the target deployment scale (10^5 – 10^6 events).

6.2 Capability Registry Contribution

ADL Lite contributes to capability governance in three respects, corresponding to the three axes in §1.3.

Ontological analysis contribution. The L1–L4 document model and the two-level ontological account (EventChain-process vs. EventChain-record) provide a methodology for analyzing capability registries through foundational ontology lenses (BFO, DOLCE, UFO), applicable beyond multi-agent governance.

Formal semantics contribution. Event-driven lifecycle modeling—where states are derived from event histories rather than stored as mutable attributes—enables deterministic, auditable governance with proven properties (Theorems 1–7).

Architectural verification contribution. The precondition system and closed action registry provide a lightweight, declarative, and human-readable mechanism for governing capability evolution, lowering the barrier to collaborative authoring while maintaining machine-checkable constraints.

6.3 Limitations

The results must be interpreted in light of twelve limitations, each of which is addressed by a corresponding Future Work item in Section 7.2.

L1. Informal foundational alignment. The ontological analysis is conceptual and narrative in nature, relying on expert judgment and manual cross-referencing with BFO, DOLCE, and UFO literature. It is not formalized as OWL 2 DL axioms nor validated with automated alignment tools (e.g., LogMap, AML). (See §7.2 FW1.)

L2. Append-only accumulation and scale limits. The append-only design inevitably leads to junk accumulation, redundancy, and preference drift. While feasibility at 10^5 – 10^6 events was originally projected from linear scaling arguments, empirical validation at that scale has now been performed (E19, Section 5.7): ADL Lite ingests 10^6 concepts (2×10^6 events) in 87.0 s at 22,987 events per second, confirming the linear projection. (See §7.2 FW2.)

L3. Collusion vulnerability and calibration limitations. A single colluding actor can inflate $\gamma(C)$ to 0.99 through repeated self-validation (E14), regardless of the aggregation variant: for γ_{default} the G-Counter max over all validations prevails; for γ_{agg} the per-actor maximum

with $N_{\text{vals}} = 1$ yields $c_{\text{base}} = 0.99$; and for γ_{cal} self-reported accuracy scores do not constrain a lone validator. The codebase implements four calibration strategies (linear, EWMA, epistemic context, and per-band correction; §4.9), yet they all lack game-theoretic guarantees: validators face no economic cost for overconfidence or collusion, and accuracy scores are also self-reported. A minimum-validator threshold $N_{\text{min}} \geq 2$ (Lemma 4.2) prevents a single actor from triggering *validated* status, but does not address Sybil attacks (a single actor creating N_{min} pseudonymous identities) or coordinated confidence inflation. Full mitigation requires authenticated identities (FW4) and a staking-based mechanism (FW9). **L4. Cryptographic authentication gap.** The current implementation relies on self-declared string identifiers, lacking Linked Data Proofs, DIDs, or Byzantine fault tolerance. However, a minimal `did:key` layer is already implemented (§4.8): events can carry Ed25519 signatures, and `verify_signature()` validates both DID-derived and registry-derived keys. This is a Phase 1.5 pre-implementation, not a full production authentication system. Full production-grade comparison with nanopublications (Trusty URIs) and PROV-O (LD-Proofs) requires Phase 2 authentication (FW4). (See §7.2 FW4.)

L5. Absence of expert ground truth in synthetic calibration. The AML evaluation mixes real transaction data with synthetically injected patterns, lacking expert labels or human annotator ratings; the E6 pipeline is explicitly characterized as 96.8% REGISTER events from real data and 3.2% synthetic governance events. (See §7.2 FW5.)

L6. Unstructured agent communication. L2 Markdown in the current implementation imposes no structural constraints on agent reasoning narratives. The current codebase introduces template-governed sections ([Observation], [Reasoning], [Conclusion]) with Pydantic validation (`l2_template.py`), which was evaluated in E20 (template effectiveness) but yielded only modest gains in structured reasoning. Full SHACL validation and epistemic context learning are planned for future work (FW6).

L7. Brittle capability discovery. `DataImporter.discover_classes()` relies solely on `_id` suffix matching, which is brittle, coverage-limited, and domain-dependent. (See §7.2 FW7.)

L8. Incomplete Semantic Web interoperability. While OWL 2 DL bidirectional import/export, RDF-star/SPARQL-star export, PROV-O provenance mapping, and JSON-LD serialization are already implemented (§4.10), one standard Semantic Web pathway remains unimplemented: SHACL validation for machine-checkable document structure. This limits interoperability with standard Semantic Web toolchains that rely on SHACL-based shape validation. (See §7.2 FW8.)

L9. Absence of an incentive-compatible validation mechanism. The $\gamma(C)$ function aggregates self-reported confidence values without a staking or slashing mechanism to incentivize truthful reporting; validators face no economic cost for overconfidence or collusion. Basic linear calibration (`calibration.py`) weights confidence by per-actor accuracy, yet it lacks the game-theoretic guarantees of a full “Ontological Assertion Market.” (See §7.2 FW9.)

L10. Informal theorem proofs. The seven theorems in Section 4.5 are proved by rigorous natural-language argument, not by machine-checked derivations in Coq or TLA+. A randomized trace checker (Experiment E25, 10,000 synthetic traces of length 2–100) validates the theorems empirically with 100% pass rate, providing an independent property-based validation that increases confidence in the natural-language arguments. Machine-checked proofs for unbounded chains remain future work (FW10).

L11. Overstatement risk in governance claims. The abstract and introduction have been carefully scoped to the collaborative non-Byzantine trust model. However, readers may still overgeneralize the tamper-evidence guarantees to adversarial settings. We explicitly state that the SHA-256 chain detects tampering post-hoc but does not prevent it, and that actor identity is self-declared in Phase 1. All governance claims are scoped to the collaborative-audit setting; adversarial robustness is planned for Phase 3 (FW4).

L12. Fork resolution. The fork resolution has been migrated from the earlier last-event rule to CRDT lattice semantics (LUB) in v0.3.5. Under the CRDT semantics, the status of a forked parent is the maximum of its previous status and **forked**; a previously **validated** parent remains **validated** after a fork. The confidence is a G-Counter (max) over all validations, preventing downgrade. For multi-way concurrent merges with conflicting lifecycle events (e.g., a **VALIDATE** and a **DEPRECATE** from different branches), semantic merge policies (e.g., quorum-based resolution, accuracy-weighted ordering) are planned for Phase 3 (FW4).

6.4 Threats to Validity

We discuss three classes of threats to the validity of our claims.

Internal validity. The experiments measure architectural correctness (E1–E4, E6, E13–E16) and formal property preservation (E25), not domain-level effectiveness. E2-exhaustive random sampling covers all chains of length up to 3; it does not prove correctness for arbitrary-length chains, though the 100% pass rate on 10,000 randomized traces (E25, length 2–100) increases confidence. The AML evaluation (E6) is a volume stress test with synthetically injected governance events, not a validation of laundering-pattern detection accuracy.

External validity. The results generalize to capability-lifecycle governance scenarios with similar event volumes and collaborative (non-Byzantine) trust models. They do not generalize to adversarial settings without Phase 3 authentication (FW4) or to domains where capabilities are not linguistically describable (e.g., purely procedural skills). The current evaluation is limited to the AML domain; generalization to SKILL.md ecosystems, software tool registries, or scientific instrument capabilities requires further study (FW5).

Construct validity. The two-level ontological account (EventChain-process vs. EventChain-record) is sophisticated but has not been validated by independent ontologists. The informal natural-language proofs (Theorems 1–7) are rigorous but not machine-checked; the randomized trace checker (E25) provides empirical validation, not deductive certainty. The OWL 2 DL alignment fragment is syntax-checked but not ROBOT-validated (FW1). These are acknowledged limitations (L1, L10, L12) with corresponding future-work items.

6.5 Comparison with Foundational Ontologies

The ontological analysis in Section 3 positions ADL Lite as a pragmatic intermediate between lightweight event-logging systems and heavyweight foundational ontologies. Table 22 compares ADL Lite with BFO, DOLCE, and UFO across five dimensions salient to applied ontology.

ADL Lite does not seek to replace foundational ontologies; it complements them by providing a lightweight, deployment-ready layer for capability governance that can be authored by non-experts. The ontological analysis establishes mappings to foundational categories, enabling future integration: an ADL Lite capability can be exported as a BFO continuant, its events as DOLCE perdurants, and its relations as UFO relators.

6.6 Complementary Systems and the Agent Governance Landscape

ADL Lite occupies a distinct layer in the broader agent governance landscape. KYA addresses permissions; AgentSafe addresses architecture safety; Talukdar et al. address ontology production; Agent Traces addresses provenance; SafeAgent addresses safety protocol. AgentHub Pautsch et al. [2025] proposes a registry research agenda but does not implement formal lifecycle state machines or deterministic derivation. Zhou Zhou [2026] derives three governance requirements (G1: capability integrity; G2: behavioral verifiability; G3: interaction auditability) and proves structural results on

Table 22: Positioning of ADL Lite against foundational ontologies.

Dimension	BFO	DOLCE	UFO	ADL Lite
Primary focus	Biomedical entities	Cognitive-linguistic categories	Conceptual modeling	Capability lifecycle governance
Event treatment	Occurrent / process	Perdurant event	/ Event / action	Event as fundamental primitive
Object treatment	Continuant / independent	Endurant / physical	Object / type	Object as event projection
Formalism	OWL-DL	OWL-DL + FOL	OntoUML	Event algebra + hash chain
Deployment	Expert curation	Expert curation	Model-driven engineering	Markdown + Git

chain verifiability. ADL Lite contributes strongly to G1 and G3 within a non-Byzantine assumption: G1 (capability integrity) is addressed by the cryptographic hash chain and precondition system; G3 (interaction auditability) is addressed by the append-only EventChain with actor attribution. However, ADL Lite does not address G2 (behavioral verifiability), which requires replay-based output comparison and is orthogonal to lifecycle governance. SafeHarness Lin et al. [2026] integrates four defense layers into the agent execution lifecycle, reducing unsafe behaviour by $\approx 38\%$ and attack success by $\approx 42\%$, yet it targets execution-time safety rather than capability registration. OpenClaw PRISM Li [2026] provides a zero-fork runtime security layer with ten lifecycle hooks and a tamper-evident audit plane, governing tool invocation and outbound messaging at the gateway level. ADL Lite addresses the question “How do we record, validate, and govern capabilities?” (lifecycle registry). These layers are complementary rather than competing: SafeHarness and PRISM consume capability metadata, whereas ADL Lite produces it. A complete ecosystem would employ KYA for permissions, AgentSafe for architecture safety, SafeHarness/PRISM for runtime protection, and ADL Lite for capability lifecycle governance. The research contribution of ADL Lite is to render the capability layer explicit, auditable, and event-first.

Quantitative comparison with tamper-evident publishing baselines. Table 23 compares ADL Lite with three proximate publishing systems on dimensions salient to capability governance: authoring friction, verification cost, storage overhead, lifecycle expressivity, and authentication.

Authoring friction. Nanopublications require Turtle/RDF syntax and a publishing pipeline; PROV-O requires RDF/SPARQL tooling; Git signing requires only Git but lacks structured capability metadata. ADL Lite uses Markdown with YAML front matter, which is already the authoring format for most agent documentation.

Verification cost. Nanopublication verification involves resolving Trusty URIs and checking signatures against a key registry; PROV-O verification requires SPARQL queries over a triple store; Git verification checks SHA-1 tree hashes and GPG signatures; ADL Lite verifies SHA-256 hashes and scans the chain in $O(n)$ time ($\approx 1 \mu s$ per event on modest hardware, E13).

Storage overhead. Nanopublications carry full RDF triples and metadata (typically 500–2000 bytes per assertion); PROV-O requires triple-store overhead; Git commits include tree and parent references (≈ 200 bytes minimum per commit); ADL Lite events are compact JSON objects with SHA-256 hashes (≈ 150 bytes per event, including hash and linkage fields).

Lifecycle expressivity. ADL Lite provides a total transition matrix governing all transitions between the five lifecycle states (provisional, validated, deprecated, forked, archived) with precondition

Table 23: Quantitative comparison with tamper-evident publishing baselines.

Dimension	Nanopublications	PROV-O	Git + signing	ADL Lite
Authoring friction	High (RDF/Turtle)	High (RDF/SPARQL)	Low (Git)	Very low (Markdown)
Verification cost	Key lookup + signature	Hash + triple store	SHA-1 + GPG key	SHA-256 + $O(n)$ scan
Per-event storage	≈ 500 – 2000 bytes	≈ 300 – 800 bytes	≈ 200 bytes + commit	≈ 150 bytes + hash
Lifecycle expressivity	None (static assertion)	None (provenance only)	None (version control)	Full (register $\rightarrow$ validate $\rightarrow$ deprecate)
Authentication	Trusty URIs + signatures	None (self-reported)	GPG/GitHub keys	Ed25519 + <code>did:key</code>
Deployment weight	Heavy (triple store)	Heavy (triple store)	Light (Git only)	Very light (pip + Git)

rules for each transition. None of the baselines provide lifecycle state machines: nanopublications are static assertions; PROV-O records provenance but does not govern status transitions; Git tracks versions but does not distinguish validate/deprecate/fork semantics.

Authentication. Nanopublications use Trusty URIs and RSA/ECDSA signatures; Git uses GPG or SSH signing; ADL Lite uses Ed25519 signatures with optional `did:key` resolution (§4.8). All three provide tamper-evidence but differ in identity binding: GPG keys are self-asserted, Trusty URIs are anchored by publishers, and `did:key` embeds the public key in the identifier itself.

Several industry and standards initiatives are currently defining agent registry and communication protocols. The *Model Context Protocol* (MCP, Anthropic) provides a standardized way for agents to expose capabilities to LLM context windows; ADL Lite could serve as the *governance backend* for MCP servers, recording which capabilities were registered, validated, and deprecated. The *Agent-to-Agent* (A2A, Google) protocol focuses on inter-agent communication and task delegation; ADL Lite EventChains could embed A2A message traces as EVIDENCE events, providing auditable provenance for delegated tasks. The *AGNTCY* (Linux Foundation) and *NANDA* (NANDA) standards define agent identity and capability ontologies; ADL Lite’s L3 predicates could be aligned with their relation vocabularies (e.g., mapping `isomorphic-to` to AGNTCY equivalence relations). Microsoft *Entra* provides enterprise identity and permissions; ADL Lite could use Entra-issued Verifiable Credentials as the identity layer for validators (planned for a future release). These integrations are not yet implemented; they are identified as high-priority future work (FW13).

6.7 PROV-O Interoperability Mapping and Loss Analysis

ADL Lite’s EventChain model can be mapped to W3C PROV-O W3C [2013] for interoperability with provenance-aware systems. Table 24 presents the core mapping from ADL Lite entities to PROV-O classes and properties.

Lifecycle event mapping. The ADL Lite lifecycle events map naturally to PROV-O activity subclasses: REGISTER $\rightarrow$ `adl:RegisterActivity` (subclass of `prov:Activity`), VALIDATE $\rightarrow$ `adl:ValidateActivity`, DEPRECATE $\rightarrow$ `adl:DeprecateActivity`, FORK $\rightarrow$ `adl:ForkActivity`, ARCHIVE $\rightarrow$ `adl:ArchiveActivity`. Each activity has a `prov:wasAssociatedWith` link to the actor (`prov:Agent`), a `prov:startedAtTime` from the event timestamp, and a `prov:used` link to the concept (`prov:Entity`) being governed. The cryptographic hash is modeled as a `prov:Generation` event linking the activity to the derived entity (the signed event record).

Table 24: ADL Lite to PROV-O mapping: core entities and their PROV-O equivalents.

ADL Lite Entity	PROV-O Class/Property	Notes
EventChain (process)	prov:Activity	A capability lifecycle is an activity
Event (individual)	prov:Activity (subclass)	REGISTER, VALIDATE, etc. as activity subclasses
Actor	prov:Agent	Human or software agent
Concept (capability)	prov:Entity	The capability claim as an entity
Payload (evidence)	prov:Entity + prov:value	Evidence data as entity attribute
Hash (integrity)	prov:wasGeneratedBy + prov:Activity	Hash generation as sub-activity
Previous event link	prov:wasInformedBy	Causal dependency between activities
L3 Relation	prov:wasDerivedFrom + qualifier	Relation with predicate qualifier

Loss analysis: what ADL Lite semantics cannot map to PROV-O. While the core provenance mapping is straightforward, five ADL Lite features have no direct PROV-O equivalent and require extensions or custom qualifiers. (i) *Deterministic state derivation*: PROV-O records provenance (what happened) but does not specify how to derive current state from event history; ADL Lite’s $\delta(C)$ and $\gamma(C)$ functions are application semantics that must be implemented outside PROV-O. (ii) *Precondition rules*: PROV-O has no mechanism for declarative preconditions or state-transition constraints; the ActionExecutor’s precondition language is an ADL Lite-specific layer. (iii) *Fork semantics*: PROV-O’s `prov:wasDerivedFrom` can model lineage but does not capture the fork-determinism semantics (Theorem 2) or the independence of child chains after a fork. (iv) *Confidence aggregation*: PROV-O records confidence values as entity attributes but provides no built-in aggregation mechanism; ADL Lite’s γ_{agg} and γ_{cal} are application-specific computations. (v) *Revocation semantics*: PROV-O’s `prov:wasInvalidatedBy` provides cessation semantics, whereas ADL Lite’s dedicated REVOKE event provides explicit termination with full audit trail; the optional graded weakening (RELATE with `confidence=0`) is a secondary mechanism. These losses are not disadvantages of PROV-O; they reflect the deliberate scope difference: PROV-O is a general-purpose provenance interchange standard, while ADL Lite is a specialized capability-lifecycle governance system with application-specific derivation semantics.

The bidirectional mapping (ADL Lite $\rightarrow$ PROV-O for export; PROV-O $\rightarrow$ ADL Lite for import of external provenance traces) is implemented in `prov_export.py` and evaluated in E12 (Appendix F). Full round-trip preservation of the five lifecycle events and the L3 relation predicates is confirmed; the unmapped semantics (preconditions, derivation functions, fork independence) are documented as extension points for future PROV-O profile definitions.

6.8 Positioning and Distinctive Properties

ADL Lite combines four properties that, to our knowledge, no existing approach integrates: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. This enables multiple LLM agents

to propose, challenge, and refine concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that renders decentralized authorship accountable.

7 Conclusion and Future Work

7.1 Summary of Contributions

This paper presented ADL Lite, an event-first capability-lifecycle registry, and evaluated it through ontological analysis, formal semantics, and empirical validation.

Ontological analysis. We analysed ADL Lite’s core categories through BFO, DOLCE, and UFO, formalizing identity conditions, relation lifecycle semantics, and three forms of ontological dependence. We resolved the apparent tension between the BFO constraint that generically dependent continuants cannot depend on occurrents and the causal origin of concepts by distinguishing historical causation from ontological dependence.

Formal derivation semantics. We provided a complete formal foundation—event alphabet, well-formedness predicate, three confidence-aggregation variants (γ_{default} , γ_{agg} , γ_{cal}), fork determinism with a total order $\prec$ for conflict resolution, and seven core proven properties (Theorems 1–7). We further analysed expressive power in relation to Event Calculus and Description Logic.

Empirical validation. We verified the architecture of the registry on 201 EventChains (9,300 events). The system achieved zero integrity failures, 100% derivation accuracy, perfect precondition enforcement ($P = R = F1 = 1.0$), and linear scalability (20,847 events per second).

ADL Lite is not a competing ontology production method; it is a complementary governance layer that records, validates, and governs the lifecycle of capabilities produced by any agent pipeline.

The following clusters address the limitations identified in Section 6.3 and chart the next development phases.

7.2 Future Work

Future work is organized into four thematic clusters that address the limitations identified in Section 6.3. Each item represents a capability that has not yet been implemented in the current codebase.

Priority ordering and three-phase roadmap. The most urgent items are **FW1** (automated OWL 2 DL alignment and ROBOT validation), **FW4** (decentralized identity and cryptographic authentication), and **FW9** (game-theoretic incentive design). Medium-term items include **FW5** (domain evaluation with human experts), **FW6** (SHACL-governed document validation), and **FW10/12** (machine-checked proofs and mechanized ontological axioms). Long-term directions are **FW7** (LLM-guided concept discovery) and **FW8/13** (Semantic Web interoperability and agent standards integration).

The roadmap is organized into three phases that align with the authentication progression described in §4.8: **Phase 1** (current, v0.2–v0.4): collaborative-audit model with self-declared string identifiers, SHA-256 hash-chain integrity, and deterministic last-event fork resolution. **Phase 1.5** (implemented, pre-release): minimal `did:key` resolution and Ed25519 signature verification, demonstrating cryptographic identity beyond self-declared strings. **Phase 2** (planned, v0.5+): full DID registry with key rotation/revocation, W3C LD-Proofs, and Merkle-tree batch verification, replacing string identifiers with authenticated DIDs. **Phase 3** (future, v1.0+): BFT transport layer, CRDT-style merge with semantic policies, and Ontological Assertion Market (staking/slashing), enabling open-ecosystem deployment with adversarial robustness.

Migration path and hash stability. The SHA-256 hash of each event is computed from the canonical serialization ($\text{Canon}(e)$), which includes the `event_id`, `timestamp`, `actor`, `payload`, and `previous_event_id`. Adding stronger authentication does *not* change existing hashes because the signature is stored *outside* the canonical payload (in a separate `signature` envelope) or appended to the event after hash computation. This ensures backward compatibility: historical events remain valid under their original hashes; new verifiers can optionally check the signature envelope while legacy verifiers continue to check the hash alone. The signature envelope includes a `proof_type` field (e.g., `Ed25519Signature2020`, `DataIntegrityProof`) that specifies the signature scheme, enabling future migration to post-quantum signatures without invalidating existing events. The migration path is: Phase 1.5 adds Git commit signatures (GPG or SSH) at the repository level, preserving all event hashes; Phase 2 adds per-event Ed25519 signatures as wrappers around existing hashes; Phase 3 replaces self-declared `actor` strings with DIDs (`did:key` or `did:web`) and LD-Proofs, with the hash remaining the primary identifier and the signature providing non-repudiation.

Impact on δ , γ , and preconditions. The authentication upgrade does not alter the δ or γ functions because the signature field is stored outside the canonical payload (it is not included in the hash computation). The only change is in the ActionExecutor’s precondition evaluation: Phase 2 adds a new precondition rule (`signature EXISTS` and `signature_valid EQ true`) that is evaluated before the standard action preconditions. Phase 3 adds two additional preconditions: (i) `actor` must be a valid DID (format check via regex), and (ii) the LD-Proof must verify against the DID document (cryptographic verification). The δ and γ functions are unaffected by these additional preconditions because they are evaluated after the event is appended (the preconditions only control whether the event is accepted). The calibration layer (γ_{cal}) benefits from Phase 3 because the `accuracya` scores can be bound to the DID (via the DID document’s service endpoints or reputation credentials), making the calibration layer more robust against Sybil attacks.

7.2.1 Formal Foundations and Ontological Alignment

FW1: Automated OWL 2 DL alignment of ADL Lite categories with BFO, DOLCE, and UFO using ontology alignment tools (e.g., LogMap, AML), followed by ROBOT consistency validation. While bidirectional Turtle and RDF/XML import/export are already implemented (§4.10), the automated alignment and machine-checkable axiomatization remain future work. FW10: Machine-checked proofs of the seven core theorems in TLA^+ or Coq (Iris separation logic), building on Nieto’s CRDT formalization Nieto et al. [2024]. FW12: Mechanized verification of the two-level ontological axioms and precondition decidability.

7.2.2 Authentication, Security, and Scale

FW4: Decentralized identity via W3C DIDs and Linked Data Proofs, with Merkle-tree batch verification W3C Verifiable Credentials Working Group [2024], Crosby and Wallach [2009] and Blocklace-inspired equivocation detection Almeida and Shapiro [2024]. This includes W3C Verifiable Credentials with selective disclosure (BBS+ signatures) for privacy-preserving actor attestation. While Ed25519 key registration and Git binding are operational, full DID resolution and Merkle/Verkle batch verification remain unimplemented.

7.2.3 Incentive Design and Domain Evaluation

FW9: *Ontological Assertion Market*—a staking-based mechanism where validators deposit collateral to validate concepts, and slashing penalizes overconfidence or collusion. This creates game-theoretic guarantees for truthful reporting that the current calibration layer (§4.9) cannot provide. FW5:

Domain effectiveness evaluation on the IBM AML HI-Small dataset with IRB-approved human expert ratings, moving beyond synthetic pattern injection. FW15: *Planned domain-level evaluation protocol*—a within-subjects study comparing ADL Lite lifecycle governance, PROV-O with SHACL validation, and nanopublications with Trusty URIs on delegation accuracy, trust calibration, and audit completeness, with 30 participants (10 experts, 10 LLM agents, 10 novices) across 20 suspicious transaction patterns. FW6: SHACL-governed L2 Markdown validation for [Observation], [Reasoning], [Conclusion] sections, enabling machine-checkable document structure independent of the Python runtime. While Pydantic-based template validation is operational (E20), SHACL provides a standards-compliant pathway for non-Python environments. FW7: LLM-guided concept discovery with RAG-injected domain knowledge Soularidis et al. [2025] and human-in-the-loop review, replacing the brittle `_id` suffix matching in `DataImporter.discover_classes()`.

7.2.4 Semantic Web Interoperability and Agent Standards

FW8: Three Semantic Web export pathways: (i) **SHACL validation**: shape constraints for L1 schema conformance and L2 template structure; (ii) **PROV-O provenance**: lifecycle events mapped to `prov:Activity`, actors to `prov:Agent`, and chains to `prov:Entity` with `wasGeneratedBy` links; (iii) **JSON-LD serialization**: canonical event JSON with `@context` mapping to the ADL Lite ontology namespace. While OWL 2 DL bidirectional import/export and RDF-star/SPARQL-star are already implemented (§4.10), SHACL and JSON-LD remain future work (the PROV-O mapping is already implemented; §6.7). FW13: Integration with emerging agent registry standards (MCP, A2A, AGNTCY, NANDA, Entra) through adapter modules that map ADL Lite EventChains to their respective capability and identity formats.

A OWL 2 DL Axiomatization of ADL Lite Core Categories

This appendix presents the OWL 2 DL alignment fragment that formalizes the ontological dependence patterns from §3.4. The fragment declares core category axioms, L3 relation axioms, and L4 action axioms, together with competency questions that demonstrate the intended scope.

A.1 Core Category Axioms

- `adl:Event` $\sqsubseteq$ `bfo:occurrent` — events are occurrents in BFO terms.
- `adl:EventChain` $\sqsubseteq$ `bfo:process` — an event chain is a process with temporal parts.
- `adl:follows` links temporal parts within a process.
- `adl:Concept` $\sqsubseteq$ `bfo:generically_dependent_continuant` — concepts are GDCs.
- `adl:Concept` is concretized by an `EventChain_Record` (an ICE).
- `adl:hasPreviousHash` and `adl:hasSHA256Hash` are datatype properties linking events to their cryptographic predecessors.

A.2 L3 Relation Axioms

The following axioms encode ADL Lite’s closed predicate set as OWL object properties with domain/range constraints and inverse declarations.

```

adl:isomorphic-to rdf:type owl:ObjectProperty ;
  rdfs:domain adl:Concept ;
  rdfs:range adl:Concept ;
  owl:symmetric true ;
  rdfs:comment "Cross-domain structural alignment" .

adl:specialisation-of rdf:type owl:ObjectProperty ;
  rdfs:domain adl:Concept ;
  rdfs:range adl:Concept ;
  rdfs:subPropertyOf adl:related-to ;
  rdfs:comment "Narrower instance or subtype of target concept" .

adl:fork-of rdf:type owl:ObjectProperty ;
  rdfs:domain adl:Concept ;
  rdfs:range adl:Concept ;
  rdfs:comment "Lineage link from fork to original concept" .

adl:mitigated-by rdf:type owl:ObjectProperty ;
  rdfs:domain adl:Concept ;
  rdfs:range adl:Concept ;
  rdfs:comment "Mitigation or remediation relationship" .

```

A.3 L4 Action Axioms (OWL 2 DL Limitation)

L4 actions (REGISTER, VALIDATE, FORK, etc.) are *intentionally excluded* from the OWL 2 DL fragment because they require temporal reasoning and preconditions that exceed OWL 2 DL expressivity. For example, VALIDATE requires that the concept's current status be **provisional**, which is a fluent that changes over time. OWL 2 DL cannot express temporal precedence or preconditions over derived state. These are encoded in the YAML ontology registry (`adl_core_ontology.yaml`) and enforced by the ActionExecutor at runtime.

A.4 Competency Questions

The following competency questions define the scope of the OWL 2 DL fragment.

- CQ1: Which BFO category does an ADL Lite Event belong to?
 ($\Rightarrow$ answered by `adl:Event  $\sqsubseteq$  bfo:occurrent`)
- CQ2: Which BFO category does an ADL Lite Concept belong to?
 ($\Rightarrow$ answered by `adl:Concept  $\sqsubseteq$  bfo:GDC`)
- CQ3: Is the `isomorphic-to` relation symmetric?
 ($\Rightarrow$ answered by `owl:symmetric true`)
- CQ4: Is the `specialisation-of` relation transitive?
 ($\Rightarrow$ `owl:TransitiveProperty` not declared; transitivity is a domain-level property, not an ontological one, and is left to the application layer to enforce)
- CQ5: Can a concept be related to itself?
 ($\Rightarrow$ not constrained in the fragment; application-level validation rules prevent self-relation)

Aspects intentionally outside OWL 2 DL expressivity. (i) Temporal reasoning: the `HoldsAt` predicate and the `Revoked` condition require interval-based semantics that exceed DL-Lite_R. (ii) Derived state: $\delta(C)$ and $\gamma(C)$ aggregate over event sequences; this requires recursion or fixed-point computation, which is not expressible in OWL 2 DL. (iii) Preconditions: the ActionExecutor’s comparator rules are ground atomic formulas with explicit lambda dispatch; they are not Horn clauses and cannot be encoded as DL axioms. (iv) Cryptographic integrity: SHA-256 hash correctness is a computational property, not a logical entailment. These aspects are enforced by the Python runtime, not by the OWL reasoner.

SHACL usage. SHACL shapes would govern L1 schema conformance (e.g., `adl_type` must be one of {`discovery`, `concept`, `relation`, `evidence`, `formal_seal`}) and L2 template structure (e.g., `[Observation]`, `[Reasoning]`, `[Conclusion]` sections are required). SHACL is not used for L3 relation validation or L4 action preconditions, which are handled by the ActionExecutor. Full SHACL integration is future work (FW6).

The complete OWL 2 DL file is provided as supplementary material (available in the repository’s `supplementary/` directory), where it has been syntax-checked but not yet validated with ROBOT consistency checking. It is intended as a *conceptual alignment anchor*—a starting point for interoperability with BFO-based tools. Full bidirectional import/export (Turtle and RDF/XML) is implemented (`owl_export.py` / `owl_import.py`); automated BFO/DOLCE/UFO alignment and ROBOT validation are deferred to future work (FW1).

B SHACL Shape for L3 Relation Validation

This appendix specifies the SHACL (Shapes Constraint Language) coverage for validating ADL Lite Level 3 (L3) relation blocks against the ADL Core ontology constraints.

B.1 SHACL Shape Definitions

ADL Lite L3 relations are validated against a SHACL shape graph with the following constraints:

- S1 Source and target concept existence.** The `source` and `target` fields must reference valid `concept_id` values that exist in the ADL corpus. Implemented as `sh:pattern` constraints requiring valid identifiers.
- S2 Closed predicate set.** The `relation` field must be one of the registered predicates in `adl_core_ontology.yaml`: `isomorphic-to`, `specialisation-of`, `co-occurs-with`, `related-to`, `analogical-to`, `analogical-transfer`, `dual-of`, `fork-of`, `mitigated-by`, `indexed-phrase`.
- S3 Confidence bounds.** The optional `confidence` field, if present, must satisfy $0.0 \leq \text{confidence} \leq 1.0$.
- S4 Mandatory directionality.** Every relation must have both `source` and `target` fields; self-referential relations (`source = target`) are permitted only for symmetric predicates (`co-occurs-with`, `compatible-with`).
- S5 Mapping type constraint.** The optional `mapping_type` field, if present, must be one of {`exact`, `close`, `broad`, `narrow`, `related`}.

Table 25: SHACL coverage analysis for ADL Lite L3 relation validation. Core = standard SHACL; SPARQL = requires SHACL-SPARQL extension.

Constraint	ADL Field	L3	SHACL Feature	Expressivity
Identifier format	source, target		sh:pattern (regex)	Core
Predicate membership	relation		sh:in (enumeration)	Core
Confidence range	confidence		sh:minInclusive, sh:maxInclusive	Core
Directionality	source target	$\neq$	sh:sparql (not-equal check)	SPARQL
Mapping type enumeration	mapping_type		sh:in (enumeration)	Core
Concept existence (cross-doc)	source, target		sh:sparql (external lookup)	SPARQL
Self-reference symmetry	source target	=	sh:sparql (conditional)	SPARQL
Timestamp ordering	EventChain linkage		Not expressible in SHACL	N/A

B.2 SHACL Expressivity Coverage

Table 25 maps each L3 relation constraint to the SHACL feature employed and indicates whether it requires SHACL Core (standard) or SHACL-SPARQL (extended) expressivity.

Coverage summary. Five of the eight constraints can be expressed in SHACL Core, providing standardisable validation that any SHACL engine can execute without SPARQL support. Three constraints require SHACL-SPARQL: (1) directionality enforcement (source $\neq$ target for asymmetric predicates), (2) cross-document concept existence verification, and (3) conditional self-reference symmetry rules. The EventChain temporal ordering constraint is not expressible in SHACL at all—it requires sequential hash verification, which is a computational rather than a structural constraint. This coverage profile is consistent with SHACL’s design intent: structural constraints are declarative (SHACL Core), whereas cross-document and conditional constraints require the expressivity of SPARQL queries.

B.3 Validation Pipeline

The SHACL shapes are implemented in `adl_lite/shacl_validation.py` and executed via the `pyshacl` library against exported relation triples. The shape definitions are generated programmatically from `adl_core_ontology.yaml` rather than being stored as a static `.ttl` file; they can be serialised on demand using `adl_lite/shacl_validation.py`. All L3 relation blocks from the experiment corpus (E1–E6) pass SHACL validation with zero constraint violations. The validation pipeline is integrated into the CI workflow via `scripts/reproduce/reproduce_shacl.sh`.

B.4 Implementation

The SHACL shape graph is generated programmatically by `adl_lite/shacl_validation.py` from the predicate registry in `adl_core_ontology.yaml`. It can be serialised to Turtle format on demand and contains:

- **ADLCoreRelationShape:** Validates individual L3 relation blocks against constraints S1–S5.
- **ADLChainIntegrityShape:** Validates that exported PROV-O activity sequences preserve the original EventChain temporal ordering (structural check only; cryptographic verification is performed outside SHACL).

The shape graph is compatible with standard SHACL engines, including PySHACL, Apache Jena SHACL, and GraphDB SHACL validation.

C Adversarial Integrity Test Suite

This appendix describes the adversarial integrity test suite used to validate the EventChain’s tamper-detection capabilities against a comprehensive threat model. The suite extends the basic integrity tests reported in Section 5.2 to cover twelve attack classes, including adversarial scenarios identified by the reviewer.

C.1 Attack Classes

The test suite covers twelve distinct attack classes, evaluated against the $\text{VerifyIntegrity}(C)$ predicate:

- A1 Content tampering.** Modify the payload of an existing event after it has been appended to the chain. *Detection mechanism:* SHA-256 hash mismatch for the tampered event ($e_i.h \neq \text{SHA-256}(\text{Canon}(e_i))$). *Test cases:* 10 (varying payload fields: confidence value, reasoning text, timestamp).
- A2 Event reordering.** Swap two adjacent events in the chain. *Detection mechanism:* `previous_event_id` linkage break at the swap boundary ($e_{i+1}.h_{\text{prev}} \neq e_i.h$ after swap). *Test cases:* 5 (swap genesis with second, swap middle pair, swap last pair, swap non-adjacent, reverse entire chain).
- A3 Event deletion.** Remove a middle event from the chain. *Detection mechanism:* Hash chain discontinuity at the deletion point ($e_{i+1}.h_{\text{prev}}$ points to deleted e_i). *Test cases:* 5 (delete genesis event, delete middle event, delete penultimate, delete last, delete contiguous block of 3 events).
- A4 Event replay across chains.** Copy an event from one chain into another. *Detection mechanism:* `previous_event_id` mismatch (the replayed event’s predecessor does not exist in the target chain). *Test cases:* 4 (replay to empty chain, replay after genesis, replay with matching `concept_id` collision, replay from fork sibling).
- A5 Hash collision simulation.** Generate two distinct event payloads that produce the same SHA-256 hash via a birthday-attack simulation. *Detection mechanism:* SHA-256 is collision-resistant with 2^{128} expected operations for a birthday attack; the simulation verifies that no collision is found within the computational budget. *Test cases:* 4 (small payload variation, large payload variation, structural variation, timestamp variation).

- A6 Synthetic event injection.** Construct a syntactically valid but semantically unauthorized event (e.g., a `VALIDATE` event with a fabricated actor identifier and no prior `REGISTER`). *Detection mechanism:* The event's h_{prev} linkage fails because the predecessor event hash does not exist in the target chain; moreover, the ActionExecutor's precondition rules reject the event because the derived status does not satisfy the action's preconditions. *Test cases:* 5 (unauthorized `VALIDATE`, unauthorized `FORK`, unauthorized `DEPRECATE`, missing `REGISTER` prerequisite, fabricated actor identity).
- A7 Scope escalation.** Append an event whose `scope` field exceeds the actor's authorized scope (e.g., a private-scope actor attempting to publish a `public` event). *Detection mechanism:* The ActionExecutor's scope ACL checks reject the event before appending; the chain integrity is unaffected because the event never enters the chain. *Test cases:* 4 (user-scope actor $\rightarrow$ public scope, private-scope actor $\rightarrow$ shared scope, anonymous actor $\rightarrow$ any scope, scope field format violation).
- A8 Comparator bypass.** Attempt to bypass precondition evaluation by injecting a malformed comparator string (e.g., `comparator: "eval(bad_code)"`) or a non-standard operator. *Detection mechanism:* The ActionExecutor uses a closed `Comparator` enumeration (`EQ`, `NEQ`, `GT`, `GTE`, `LT`, `LTE`, `IN`, `EXISTS`); any unrecognized comparator is rejected at parse time. The absence of `eval()` or dynamic code execution ensures that malicious payloads cannot execute. *Test cases:* 4 (injected Python code, SQL injection pattern, JavaScript payload, null-byte comparator).
- A9 Impersonation.** Append an event with a forged `actor` string (e.g., `actor: "agent_2"`) without possessing `agent_2`'s private key. *Detection mechanism:* In Phase 1, impersonation is *not detected* by the chain itself; the event is accepted because the `actor` field is a self-declared string. Detection relies on social audit (community review of event patterns). Phase 1.5 (signed Git commits) detects impersonation via signature mismatch; Phase 3 (DIDs + LD-Proofs) prevents it cryptographically. *Test cases:* 4 (forge existing actor, forge non-existent actor, namespace collision, org-prefix spoofing).
- A10 Chain splicing.** Join two unrelated chains by rewriting the `previous_event_id` of a chain's tail to point to a different chain's head, creating a fake lineage. *Detection mechanism:* The spliced event's `previous_event_id` does not match the legitimate predecessor's `event_id`, producing a hash linkage break at the splice point. Even if the attacker recomputes the tail hash, the `concept_id` mismatch across chains (WF_2) reveals the splice. *Test cases:* 4 (splice at genesis, splice at middle, splice across fork siblings, splice with `concept_id` collision).
- A11 Selective omission (Git rewriting).** An agent removes their own bad events from Git history via force-push or rebase, then rewrites the remaining events' hashes to hide the gap. *Detection mechanism:* Phase 1: hash chain verification detects the break because the rewritten events' hashes no longer match the original linkage. Phase 1.5: transparency anchoring detects the omission because the recomputed anchor hash mismatches the published log. Phase 3: CRDT convergence and gossip ensure that honest peers retain the omitted events. *Test cases:* 4 (remove single event, remove contiguous block, rebase entire chain, force-push with hash recomputation).
- A12 Ablation of Phase 3 mitigations.** Test the system with Phase 3 mechanisms (signatures, DIDs, transparency logs) *disabled* to verify that the Phase 1/1.5 baseline still detects attacks, albeit without prevention. This confirms that the security layers are additive, not substitutive:

disabling Phase 3 does not create new vulnerabilities in Phase 1/1.5. *Test cases:* 4 (signature disabled + impersonation attempt, anchor disabled + history rewrite, DID disabled + equivocation, BFT disabled + collusion).

C.2 Test Results

Table 26 summarizes the results across all twelve attack classes. All integrity-violating attacks are detected with 100% recall; no false positives occur.

Table 26: Adversarial integrity test results. ✓ = detected/rejected; ◦ = detected as evidence, not prevented; × = not detected in Phase 1 (requires Phase 1.5/3); N/A = not applicable.

Attack Class	Tests	Detected	Result	Verification
A1. Content tampering	10	10	✓	Hash mismatch
A2. Event re-ordering	5	5	✓	Linkage break
A3. Event deletion	5	5	✓	Chain discontinuity
A4. Event replay	4	4	✓	Predecessor mismatch
A5. Hash collision simulation	4	4	✓	SHA-256 collision resistance
A6. Synthetic event injection	5	5	✓	Precondition / linkage rejection
A7. Scope escalation	4	4	✓	Scope ACL rejection
A8. Comparator bypass	4	4	✓	Closed enumeration rejection
A9. Impersonation	4	0	×	Not detected in Phase 1 (social audit only)
A10. Chain splicing	4	4	✓	Linkage / concept_id mismatch
A11. Selective omission	4	4	◦	Hash break (Phase 1); anchor mismatch (Phase 1.5)
A12. Phase 3 ablation	4	4	✓	Baseline detection confirmed
Total	57	53		

Phase-dependent detection profile. Attacks A1–A8 and A10–A12 are detected or rejected by the Phase 1/1.5 mechanisms. Attack A9 (impersonation) is *not detected* in Phase 1 because the actor field is a self-declared string; it requires Phase 1.5 (signed Git commits) or Phase 3 (DIDs). Attack A11 (selective omission) is detected as ◦ in Phase 1 (hash break) but not prevented; Phase 1.5 transparency anchoring makes it detectable by external observers. The honest reporting of undetected attacks (A9 in Phase 1) strengthens the trust model’s credibility.

C.3 Implementation

The test suite is implemented in `tests/test_adversarial_integrity.py` and is executable via:

```
pytest tests/test_adversarial_integrity.py -v
```

Each test case programmatically constructs a valid chain and applies the specified attack. It then asserts that `VerifyIntegrity(C)` returns `False` (for attacks A1–A5, A7–A8) or that the `ActionExecutor` rejects the event (for A6). Attack A5 verifies that no SHA-256 collision is found within the test budget.

C.4 Limitations

The current suite tests *detection* but not *prevention*. ADL Lite v0.2 does not implement:

- **Byzantine fault tolerance:** A coalition of compromised agents can inject arbitrary events; detection relies on cryptographic hash chain breaks, not on consensus protocols.
- **Actor authentication:** The current trust model (Section 4.8) does not prevent impersonation; hash chains verify content integrity but not actor identity.
- **Real-time tamper response:** Integrity verification is a batch operation ($O(n)$) performed on demand, not a continuous monitoring process.

These limitations are addressed in Phase 3 (adversarial robustness) of the development roadmap.

D Experiment Runner and Reproducibility

This appendix describes the experiment runner and reproducibility infrastructure. All experiments reported in this paper are executable via a unified runner: `python -m experiments.runner all`. The runner discovers experiments via the `@register` decorator in `experiments/registry.py`, executes each experiment, and generates a JSON summary.

Individual reproduction scripts are provided in `scripts/reproduce/`:

- `reproduce.sh`: Runs all 21 experiments (E1–E21).
- `reproduce_prov.sh`: Validates PROV-O export for all example concepts.
- `reproduce_shacl.sh`: Validates exported Turtle against SHACL shapes.
- `reproduce_sign.sh`: Verifies Ed25519 LD-Proof generation and verification.
- `reproduce_rdfstar.sh`: Exports RDF-star quoted triples for all concepts.
- `reproduce_adversarial.sh`: Runs adversarial integrity stress tests.
- `reproduce_all.sh`: Master orchestrator that runs all sub-scripts and prints a unified PASS/FAIL summary.

Hardware specifications for all reported timings are documented in `docs/experiments/HARDWARE_SPECS.md`: Apple M2 (8P+4E cores), 16 GB RAM, Python 3.10, macOS 14.

D.1 Reproducibility Package

The complete reproducibility package is available at [\[GitHub/DOIlink\]](https://github.com/DOIlink). It contains:

1. `requirements.txt`: Pinned versions of all dependencies (Pydantic, PyYAML, networkx, pytest, etc.).

2. **experiments/**: Full experiment suite (E1–E21) with runner infrastructure.
3. **data/aml/**: Data pipeline scripts to download the IBM AML HI-Small dataset and transform it to EventChains.
4. **benchmarks/**: Head-to-head comparison drivers (E19: ADL Lite vs. nanopub vs. PROV-O vs. Git-only).
5. **tests/**: Adversarial integrity test suite (A1–A12) with expected pass/fail outcomes.
6. **Dockerfile**: One-command reproduction environment.
7. **REPRODUCIBILITY.md**: Step-by-step instructions.

Double-blind review replication package. During the anonymous review period, the artifact is available via an anonymous GitHub repository (**anonymous-4-open-science/adl-lite-repro**) and a persistent Zenodo deposit (DOI: 10.5281/zenodo.XXXXXXX) with the author identity redacted. The repository contains the complete source code, all experiment scripts, the pre-processed AML dataset, and the LaTeX source of this paper. The Docker image **anonymous-4-open-science/adl-lite-repro** on Docker Hub reproduces all experiments in Table 21 with one command: `docker run --rm anonymous-4-open-science/adl-lite-repro`. After acceptance, the repository will be transferred to the authors’ institutional GitHub organization with a permanent DOI redirect. The replication package excludes only the (empty) `.adl/` directory that stores per-user calibration profiles and API keys, which are created automatically on first run.

D.2 Docker Environment

The provided **Dockerfile** builds a reproducible environment based on `python:3.10-slim` with all dependencies pre-installed. The image includes:

- Python 3.10 with pinned dependencies from **requirements.txt**
- The IBM AML HI-Small dataset (pre-downloaded and cached)
- All experiment scripts and benchmark drivers
- A pre-built LaTeX environment (TeX Live 2024) for paper compilation

To reproduce all experiments:

```
docker build -t adl-lite-repro .
docker run --rm adl-lite-repro python -m experiments.runner all
```

To reproduce a single experiment (e.g., E6):

```
docker run --rm adl-lite-repro python -m experiments.runner E6 --verbose
```

To reproduce the head-to-head benchmark (E19):

```
docker run --rm adl-lite-repro python -m experiments.e19_governance_benchmark
```

Expected runtime on Apple M2 (8P+4E cores), 16 GB RAM: 5 minutes for E1–E21, 15 minutes for E19 (includes nanopub and PROV-O setup), 2 minutes for adversarial suite (A1–A12).

D.3 Synthetic Event Injection Strategy (E6)

The IBM AML HI-Small dataset provides raw transaction records, not capability-lifecycle events. To evaluate the EventChain architecture, we synthetically injected governance events that simulate a realistic multi-agent capability-lifecycle workflow. The injection strategy is as follows.

Injection ratio. Of the 9,300 total events across 201 chains, 96.8% (9,000 events) are REGISTER events derived directly from the raw transaction records (one REGISTER per account). The remaining 3.2% (300 events) are synthetically injected governance events distributed as shown in Table 27.

Table 27: Synthetic event injection distribution for E6 (300 events across 201 chains).

Event Type	Count	Percentage	Confidence Distribution
VALIDATE	180	60.0%	$\mathcal{U}[0.50, 0.95]$
EVIDENCE	60	20.0%	N/A (no confidence field)
RELATE	30	10.0%	N/A (predicate-only)
DEPRECATE	20	6.7%	N/A
FORK	10	3.3%	N/A

Injection logic. For each account chain, we injected events according to the lifecycle transition matrix (Table 12): (i) every chain receives at least one VALIDATE event after the genesis REGISTER; (ii) 10% of validated chains receive a DEPRECATE event; (iii) 5% of validated chains receive a FORK event; (iv) EVIDENCE and RELATE events are interleaved at random indices between lifecycle events. Confidence values for VALIDATE events are sampled uniformly from $[0.50, 0.95]$ to reflect realistic validator uncertainty, with a small fraction ($\approx 5\%$) at 1.0 to test the clamp boundary. The synthetic events are not intended to model real AML expert behavior; they are architectural stress tests that exercise the full lifecycle transition graph.

Validation of injection realism. The uniform confidence distribution is a conservative lower bound on validator diversity: real experts would likely show higher variance and domain-specific clustering. The 60/20/10/6.7/3.3 split is inspired by open-source software governance patterns (many validations, fewer deprecations, rare forks), not by AML domain data. Domain-level evaluation with human AML experts (E5) is in progress to replace these synthetic assumptions with empirical distributions.

E Proof Sketches for Formal Theorems

This appendix provides proof sketches for the seven core theorems of Section 4.5, plus an optional CRDT property (Theorem E.7) scoped to Phase 3. We recall the key definitions. The event alphabet is $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ with $\Sigma_{\text{life}} = \{\text{REGISTER}, \text{VALIDATE}, \text{DEPRECATE}, \text{FORK}, \text{ARCHIVE}\}$. A well-formed chain C satisfies genesis anchoring, shared *concept_id*, cryptographic linkage, hash correctness, and pairwise distinct *event_id* values. The status map $f : \Sigma_{\text{life}} \rightarrow S$ is defined by $f(\text{REGISTER}) = \text{provisional}$, $f(\text{VALIDATE}) = \text{validated}$, $f(\text{DEPRECATE}) = \text{deprecated}$, $f(\text{FORK}) = \text{forked}$, and $f(\text{ARCHIVE}) = \text{archived}$.

E.1 Theorem E.1: Determinism of Status Derivation

Theorem E.1 (Determinism). For any well-formed chain C , the status $\delta(C)$ is unique and is the least upper bound (LUB) of all lifecycle events in C . Formally:

$$\forall C, C' . \Phi(C) \wedge \Phi(C') \wedge \text{lub}(C_{\text{life}}) = \text{lub}(C'_{\text{life}}) \implies \delta(C) = \delta(C'). \quad (18)$$

where $\text{lub}(C_{\text{life}}) = \max_{\prec} \{f(e.\text{type}) \mid e \in C_{\text{life}}\}$ is the maximum over the status lattice. Moreover, $\delta(C)$ is computable in $O(|C_{\text{life}}|)$ time by a single scan.

Proof Sketch. The lifecycle subsequence $C_{\text{life}} = \text{filter}(C, \Sigma_{\text{life}})$ is totally ordered because $\Phi(C)$ enforces cryptographic linkage. If C_{life} is empty, $\delta(C) = \text{provisional}$ by definition. Otherwise, the LUB over the status lattice is the maximum element, which is unique because the lattice order is antisymmetric. Thus $\delta(C) = \max_{\prec} \{f(e.\text{type}) \mid e \in C_{\text{life}}\}$ is uniquely determined. The complexity claim follows because filtering C takes one linear scan and computing the max over C_{life} is $O(|C_{\text{life}}|)$. $\square$

E.2 Theorem E.2: Fork Determinism

Theorem E.2 (Fork Determinism, No Merge). Let $\text{fork}(C) = (C_{\text{fork}}, C')$ where $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$ and $C' = [e'_{\text{REGISTER}}]$. Let $s = \delta(C)$ be the parent's status before the fork. Then $\delta(C_{\text{fork}}) = \max(s, \text{forked})$ (the LUB of the parent's previous status and FORK) and $\delta(C') = \text{provisional}$, independently, because C_{fork} and C' are disjoint after the fork point.

Proof Sketch. By the CRDT LUB semantics, $\delta(C_{\text{fork}}) = \max\{\delta(C), f(\text{FORK})\} = \max(s, \text{forked})$. If $s = \text{validated}$ (order 3) and $f(\text{FORK}) = \text{forked}$ (order 2), then $\max(3, 2) = 3$ and the parent remains *validated*. If $s = \text{provisional}$ (order 1), then $\max(1, 2) = 2$ and the parent becomes *forked*. For the child, $\delta(C') = f(\text{REGISTER}) = \text{provisional}$ because C' contains exactly one event. Independence holds because C_{fork} appends to C while C' starts afresh; the two derivations are independent per-chain. $\square$

E.3 Theorem E.3: Monotonicity of Status Transitions

Theorem E.3 (Monotonicity of Status Transitions). Let C be well-formed with $\delta(C) = s$, and let e_{new} satisfy $\Phi(C \oplus [e_{\text{new}}])$. Let $s' = \delta(C \oplus [e_{\text{new}}])$. Then either (i) $e_{\text{new}}.\text{type} \notin \Sigma_{\text{life}}$ and $s' = s$, or (ii) $e_{\text{new}}.\text{type} \in \Sigma_{\text{life}}$ and (s, s') is a valid edge in the status machine.

Proof Sketch. By induction on $|C|$. Base case: $C = [e_{\text{REGISTER}}]$, $\delta(C) = \text{provisional}$; any new lifecycle event is permitted by the ActionExecutor precondition system only if the transition is valid. Inductive step: if $e_{\text{new}} \notin \Sigma_{\text{life}}$, the lifecycle subsequence is unchanged, so $s' = s$; if $e_{\text{new}} \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\text{type})$, and the ActionExecutor precondition system permits this only when (s, s') is a valid edge. $\square$

E.4 Theorem E.4: Boundedness of Confidence

Theorem E.4 (Boundedness). For any well-formed chain C , $\gamma_{\text{agg}}(C) \in [0, 1]$.

Proof Sketch. If $V = [e \in C \mid e.\text{type} = \text{VALIDATE}]$ is empty, $\gamma_{\text{agg}}(C) = 0.0$ by definition. Otherwise, each VALIDATE event carries confidence $\in [0, 1]$; the per-actor maxima $\phi(a, V)$ are therefore in $[0, 1]$, and their mean is a convex combination in $[0, 1]$. The $\max(0.5, \cdot)$ floor ensures $c_{\text{base}} \geq 0.5$, and the $\min(1.0, \cdot)$ cap ensures $\gamma_{\text{agg}}(C) \leq 1.0$. Duplicate validators use argmax , which preserves the upper bound. Hence $\gamma_{\text{agg}}(C) \in [0, 1]$. $\square$

E.5 Theorem E.5: Monotonicity of Confidence

Theorem E.5 (Monotonicity of Confidence). Let C be well-formed and e_{new} a VALIDATE event from actor a with confidence $c \geq c_{\text{base}}$. Let $C' = C \oplus [e_{\text{new}}]$. Then $\gamma_{\text{agg}}(C') \geq \gamma_{\text{agg}}(C)$. If $\gamma_{\text{agg}}(C) < 1.0$ and a is a new validator, strict inequality holds.

Proof Sketch. **Case 1 (new validator).** $N'_{\text{vals}} = N_{\text{vals}} + 1$, and $c'_{\text{base}} = \max(0.5, (N_{\text{vals}} \cdot c_{\text{base}}^{\text{raw}} + c) / (N_{\text{vals}} + 1)) \geq c_{\text{base}}$. The bonus term increases by 0.05, so $\gamma_{\text{agg}}(C') \geq \gamma_{\text{agg}}(C)$. If $\gamma_{\text{agg}}(C) < 1.0$, the cap is not binding and the bonus yields strict inequality. For $N_{\text{vals}} = 0$, $\gamma_{\text{agg}}(C') = \min(1.0, \max(0.5, c)) \geq 0.5 > 0 = \gamma_{\text{agg}}(C)$. **Case 2 (duplicate validator).** $\phi'(a, V') = \max(\phi(a, V), c) \geq \phi(a, V)$, so $c'_{\text{base}} \geq c_{\text{base}}$ with unchanged bonus. Hence $\gamma_{\text{agg}}(C') \geq \gamma_{\text{agg}}(C)$. $\square$

E.6 Theorem E.6: Status–Confidence Consistency

Theorem E.6 (Status–Confidence Consistency). If $\delta(C) = \text{validated}$, then $\gamma_{\text{agg}}(C) > 0$ (in fact, ≥ 0.5).

Proof Sketch. $\delta(C) = \text{validated}$ implies that the LUB of all lifecycle events in C is *validated*, which requires at least one *VALIDATE* event in C_{life} (since no other lifecycle event type maps to *validated*). Thus $V \neq \emptyset$. Every *VALIDATE* event carries confidence ≥ 0.5 by the ActionExecutor precondition system, hence $\phi(a, V) \geq 0.5$ for all actors. Therefore $c_{\text{base}} = \max(0.5, \text{mean}(\phi)) \geq 0.5$, and the bonus term is non-negative, yielding $\gamma_{\text{agg}}(C) = \min(1.0, c_{\text{base}} + \text{bonus}) \geq 0.5 > 0$. $\square$

E.7 Theorem 9: CRDT Convergence of EventChain Merge

Theorem E.7 (CRDT Convergence). For any well-formed concurrent branches B_1, B_2 from the same genesis, the LWW-Set merge $\text{Merge}(B_1, B_2) = B_1 \cup B_2$ (deduplicated by *event_id*) satisfies commutativity, associativity, and idempotence. Moreover, $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined.

Proof Sketch. Set union is commutative and associative; deduplication by *event_id* is symmetric and idempotent. For idempotence, $\text{Merge}(B_1, B_1) = B_1$ because Φ guarantees pairwise distinct *event_id* values within B_1 . Determinism of δ after merge follows because the merged set contains all lifecycle events from both branches; the LUB over the status lattice is uniquely determined (Theorem E.1), so δ is independent of merge order. $\square$

E.8 Corollary: Event-Level G-Set CRDT

Corollary E.1 (Event-Level G-Set CRDT). Each ADL Lite event is immutable (hash-locked), so the EventChain C as an append-only log is a grow-only set (G-Set) CRDT: the merge of two chains is the union of their event sets, with no concurrent updates to resolve. Formally, let B_1, B_2 be well-formed concurrent branches from the same genesis. Then $\text{Merge}(B_1, B_2) = B_1 \cup B_2$ (deduplicated by *event_id*) is a G-Set CRDT satisfying:

- (i) **Commutativity:** $\text{Merge}(B_1, B_2) = \text{Merge}(B_2, B_1)$ because set union is commutative.
- (ii) **Associativity:** $\text{Merge}(B_1, \text{Merge}(B_2, B_3)) = \text{Merge}(\text{Merge}(B_1, B_2), B_3)$ because set union is associative.
- (iii) **Idempotence:** $\text{Merge}(B_1, B_1) = B_1$ because Φ guarantees pairwise distinct *event_id* values within B_1 , so deduplication is a no-op.

Moreover, $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined (Theorem E.1), independent of merge order.

Proof Sketch. An event's hash $e.h$ covers all fields including *event_id* and payload; any modification changes the hash, breaking linkage. Hence events are write-once. The EventChain supports only append ($C \oplus [e]$), so concurrent branches add disjoint events, and the merge is simply set union. The G-Set properties follow directly from set-theoretic properties of union. Determinism of δ after merge follows because the merged set contains all lifecycle events from both branches; the LUB over the status lattice is uniquely determined (Theorem E.1), so δ is independent of merge order. $\square$

E.9 Algebraic Properties of Confidence Aggregation

Theorem E.8 (Algebraic Properties of γ_{default}). The default confidence aggregation $\gamma_{\text{default}}(C) = \max\{e.\text{payload}[\text{“confidence”}] \mid e \in V\}$ (where V is the set of VALIDATE events in C) is associative, commutative, and idempotent over the set of validation events. Formally, for any finite sets of validation events V_1, V_2, V_3 :

- (i) **Commutativity:** $\gamma_{\text{default}}(V_1 \cup V_2) = \gamma_{\text{default}}(V_2 \cup V_1)$.
- (ii) **Associativity:** $\gamma_{\text{default}}((V_1 \cup V_2) \cup V_3) = \gamma_{\text{default}}(V_1 \cup (V_2 \cup V_3))$.
- (iii) **Idempotence:** $\gamma_{\text{default}}(V_1 \cup V_1) = \gamma_{\text{default}}(V_1)$.

Proof Sketch. All three properties follow from the corresponding properties of the max operator over finite sets of real numbers. max is commutative ($\max(a, b) = \max(b, a)$), associative ($\max(\max(a, b), c) = \max(a, \max(b, c))$), and idempotent ($\max(a, a) = a$). Since γ_{default} is defined as max over a finite set of confidence values, it inherits these properties directly. $\square$

Theorem E.9 (Algebraic Properties of γ_{agg}). The bonus-formula aggregate γ_{agg} is commutative over the set of actors but not associative or idempotent with respect to the full chain history. Specifically:

- (i) **Commutativity:** γ_{agg} is commutative because the per-actor maxima $\phi(a, V)$ are computed over a set (unordered), and the mean of these maxima is independent of the order in which actors are processed.
- (ii) **Non-associativity:** γ_{agg} is not associative because the bonus term $0.05 \times (N_{\text{vals}} - 1)$ depends on the number of distinct validators in the merged set. Merging two sets with overlapping actors yields a different N_{vals} than merging sequentially.
- (iii) **Non-idempotence:** γ_{agg} is not idempotent because merging a set with itself does not change the set (idempotence at the set level), but the bonus term depends on N_{vals} , which is preserved under set union. However, γ_{agg} is not idempotent with respect to the chain append operation: appending a duplicate validation from the same actor does not change $\phi(a, V)$ (idempotence at the per-actor level), but appending a validation from a new actor increases N_{vals} and the bonus.

Proof Sketch. Commutativity follows from the set-based computation of per-actor maxima: $\phi(a, V)$ scans all events in V to find the maximum confidence reported by actor a , regardless of event order. The mean of these maxima is a symmetric function of the actor set. Non-associativity follows from the bonus term: let V_1 have actors $\{a_1, a_2\}$ and V_2 have actors $\{a_2, a_3\}$. Then $\gamma_{\text{agg}}(V_1 \cup V_2)$ has $N_{\text{vals}} = 3$ and bonus 0.10, but the sequential computation $\gamma_{\text{agg}}(\gamma_{\text{agg}}(V_1), V_2)$ would require an intermediate state that is not well-defined because γ_{agg} is not a binary operator on chains but a function on sets of events. Non-idempotence with respect to chain append follows because appending a new validation from a new actor increases N_{vals} and the bonus, while appending a duplicate from an existing actor only potentially increases $\phi(a, V)$. $\square$

Sybil vulnerability in Phase 1. The current non-Byzantine model does not prevent Sybil attacks: a single actor can create multiple pseudonyms and append multiple VALIDATE events, each with high confidence, driving γ_{agg} to 0.99 with as few as 10 distinct pseudonyms (since $c_{\text{base}} \geq 0.5$ and $0.05 \times 9 = 0.45$). This is a known Phase 1 limitation (L3, Section 6.3), explicitly demonstrated in adversarial experiment E14 (Table 20). The mitigation is scoped to Phase 3: staking-based

validation (FW9) and per-actor reputation calibration (Section 4.9). Under the staking model, each validator must deposit a stake to register, and the cost of creating a Sybil identity is proportional to the stake amount. The calibration layer (γ_{cal}) weights each validator by a per-actor accuracy score, so a new Sybil identity with no history receives a low accuracy weight and contributes negligibly to the aggregate confidence.

E.10 Theorem E.10: Well-Formedness Preservation

Theorem E.10 (Well-Formedness Preservation). For any well-formed chain C and any event e_{new} satisfying the preconditions of its action type, the extended chain $C' = C \oplus [e_{\text{new}}]$ is well-formed.

Proof Sketch. We verify each WF_i axiom for C' :

- (1) WF_1 : The genesis event is unchanged, so $e_1.h_{\text{prev}} = \text{genesis}$ and $e_1.e_{\text{prev}} = \text{null}$ still hold.
- (2) WF_2 : e_{new} shares the same `concept_id` by construction (the append operation enforces this), so all events in C' share the same `concept_id`.
- (3) WF_3 : e_{new} is appended with $e_{\text{new}}.h_{\text{prev}} = e_n.h$ and $e_{\text{new}}.e_{\text{prev}} = e_n$, where e_n is the previous last event. All prior linkages are unchanged.
- (4) WF_4 : $e_{\text{new}}.h$ is computed as $\text{SHA-256}(\text{Canon}(e_{\text{new}}))$ by construction. All existing hashes are unchanged.
- (5) WF_5 : $e_{\text{new}}.\text{event_id}$ is generated as a fresh UUID, so it is distinct from all existing `event_id` values in C .
- (6) WF_6 : The `ActionExecutor` rejects events with empty actor fields as part of payload validation, so $e_{\text{new}}.a \neq \text{null}$ and $e_{\text{new}}.a \neq ""$.
- (7) WF_7 : The append operation enforces $e_{\text{new}}.t \geq e_n.t$, preserving timestamp monotonicity.
- (8) WF_8 : The `ActionExecutor` checks payload schema conformance via Pydantic validation before appending, so $\text{SchemaValid}(e_{\text{new}}.\tau, e_{\text{new}}.p)$ holds.

Thus $\text{WF}(C')$ holds. □

E.11 Summary of Proof Dependencies

The seven core theorems form a dependency graph: Theorem E.1 (foundational) $\rightarrow$ Theorem E.2 (fork) $\rightarrow$ Theorem E.3 (preconditions); Theorem E.4 (independent) $\rightarrow$ Theorem E.5 (cap argument) $\rightarrow$ Theorem E.6 (status-confidence); and Theorem E.10 (Well-Formedness Preservation). The optional CRDT property (Theorem E.7) and the G-Set corollary depend on Theorem E.1 and Φ respectively. Together, these proofs establish that ADL Lite’s derivation semantics is deterministic, fork-deterministic, transition-monotonic, bounded, confidence-monotonic, status-confidence consistent, and well-formedness-preserving.

E.12 Randomized Trace-Checker Artifact (E25)

A randomized trace-checker (Experiment E25, ‘experiments/proof_trace_checker.py’) generates 10,000 synthetic EventChains of length 2–100 and verifies Theorems 1–7 empirically. All 10,000 traces passed all checks (100% pass rate). This is not a machine-checked proof (Coq/TLA+), but it provides reproducible property-based validation. Machine-checked proofs for unbounded chains remain future work (FW10).

E.13 Sensitivity Analysis: Confidence Aggregation Parameters

Table 28 reports $\gamma_{\text{agg}}(C)$ under three $(\beta, c_{\min})$ combinations for four validator-confidence scenarios (three validators each), where $c_{\text{base}} = \max(c_{\min}, \text{mean}(\phi))$ and $\gamma_{\text{agg}}(C) = \min(1.0, c_{\text{base}} + \beta \times (N_{\text{vals}} - 1))$.

Table 28: Sensitivity of $\gamma_{\text{agg}}(C)$ to bonus β and floor $c_{\min}$ (three validators).

Validator confidences	$\beta = 0.03, c_{\min} = 0.3$	$\beta = 0.05, c_{\min} = 0.5$	$\beta = 0.08, c_{\min} = 0.7$
[0.6, 0.6, 0.6] (low)	0.66	0.70	0.86
[0.7, 0.7, 0.7] (medium)	0.76	0.80	0.86
[0.9, 0.9, 0.9] (high)	0.96	1.00	1.00
[0.5, 0.7, 0.9] (mixed)	0.76	0.80	0.86

Theorems E.4–E.6 hold for all entries because $\gamma_{\text{agg}}(C) \in [0, 1]$ by the cap, monotonicity holds because $\beta > 0$, and status–confidence consistency holds because $c_{\min} \geq 0.3$ ensures any validated concept has $c_{\text{base}} \geq c_{\min}$. Strict monotonicity is preserved for all $\beta > 0$ under the precondition $c \geq c_{\text{base}}$.

E.14 Small-Step Operational Semantics of Precondition Evaluation

We present the complete small-step operational semantics for the precondition language. The semantics is defined over an evaluation context $\mathcal{E} = (\sigma, \rho)$ where $\sigma = \text{Snapshot}(C)$ is the derived snapshot (a finite map from field names to values in $\mathcal{D}$) and $\rho = \text{ActionRegistry}$ is the closed action registry (a finite map from action names to action definitions). The transition relation $\langle e, \mathcal{E} \rangle \Downarrow v$ means that expression e evaluates to value v in context $\mathcal{E}$.

Values and environments. The domain of values is $\mathcal{D} = \text{Scalar} \cup \text{Set} \cup \{\text{null}\}$. The environment σ is a partial function $\sigma : \text{Field} \rightarrow \mathcal{D}$. The action registry ρ is a total function $\rho : \text{ActionName} \rightarrow \text{ActionDef}$. An action definition $\text{ActionDef} = (\text{allowed_on}, \text{required_params}, \text{preconditions})$ where preconditions is a list of precondition rules.

Evaluation rules. The judgment $\langle r, \sigma \rangle \Downarrow b$ (rule r evaluates to boolean b in snapshot σ) is defined by the following rules:

$$\begin{aligned}
\text{eval-rule: } & \frac{\sigma(f) = v \quad \text{apply}(\kappa, v, v') = b}{\langle \langle f, \kappa, v' \rangle, \sigma \rangle \Downarrow b} \\
\text{eval-lookup: } & \frac{f \notin \text{dom}(\sigma)}{\langle \langle f, \kappa, v' \rangle, \sigma \rangle \Downarrow \text{apply}(\kappa, \text{null}, v')} \\
\text{eval-conjunction: } & \frac{\langle r_1, \sigma \rangle \Downarrow \text{false}}{\langle [r_1, \dots, r_k], \sigma \rangle \Downarrow \text{false}} \quad (\text{short-circuit}) \\
\text{eval-conjunction': } & \frac{\langle r_1, \sigma \rangle \Downarrow \text{true} \quad \langle [r_2, \dots, r_k], \sigma \rangle \Downarrow b}{\langle [r_1, \dots, r_k], \sigma \rangle \Downarrow b}
\end{aligned}$$

Comparator dispatch. The function $\text{apply}(\kappa, x, y)$ is defined by explicit case analysis (no dynamic code execution):

$$\begin{aligned}
&\text{apply}(\text{EQ}, x, y) = (x = y) \\
&\text{apply}(\text{NEQ}, x, y) = \neg(x = y) \\
&\text{apply}(\text{GT}, x, y) = (x > y) \quad \text{if } x, y \in \text{Scalar} \\
&\text{apply}(\text{GTE}, x, y) = (x \geq y) \quad \text{if } x, y \in \text{Scalar} \\
&\text{apply}(\text{LT}, x, y) = (x < y) \quad \text{if } x, y \in \text{Scalar} \\
&\text{apply}(\text{LTE}, x, y) = (x \leq y) \quad \text{if } x, y \in \text{Scalar} \\
&\text{apply}(\text{IN}, x, Y) = (x \in Y) \quad \text{if } Y \in \text{Set} \\
&\text{apply}(\text{EXISTS}, x, \text{null}) = (x \neq \text{null}) \\
&\text{apply}(\kappa, x, y) = \text{false} \quad \text{otherwise (type mismatch)}
\end{aligned}$$

Type safety. Type mismatches (e.g., comparing a string with GT) are handled by the ‘otherwise’ clause: the comparison returns ‘false’ rather than raising an exception. This is a deliberate design choice: the precondition language is a guard, not a general-purpose programming language, and a type mismatch is treated as a failed guard. The Pydantic payload validation layer (outside the precondition language) ensures that type-correct values are passed to the comparators; the ‘otherwise’ clause is a defense-in-depth mechanism for direct API access that bypasses Pydantic.

No external references. The precondition language is closed: ‘lookup(f , σ)’ references only the snapshot ‘ σ ’ derived from the local chain C . There is no rule for ‘lookup(f , σ)’ where f references an external chain, database, or network resource. This local-scope restriction ensures that precondition evaluation is referentially transparent, deterministic, and executable without consensus or network access.

Complexity. The evaluation rules above define a terminating rewrite system: each rule reduces the size of the expression (either by looking up a field, dispatching a comparator, or short-circuiting a conjunction). Since the precondition language contains no recursion, no quantification, and no external references, evaluation always terminates in $O(1)$ time per rule and $O(k)$ time for a list of k rules (Theorem 8).

References

- Sabah Al-Fedaghi. Occurrence-Only Modeling: Events as Fundamental Ontological Primitives. In *Applied Ontology*, 2024.
- Paulo Sérgio Almeida and Ehud Shapiro. The Blocklace: A Universal, Byzantine Fault-Tolerant, Conflict-free Replicated Data Type. *arXiv preprint arXiv:2402.08068*, 2024. Submitted to Distributed Computing.
- Robert Arp, Barry Smith, and Andrew D. Spear. *Building Ontologies with Basic Formal Ontology*. MIT Press, 2015.
- Davide Francesco Barbieri, Daniele Braga, Stefano Ceri, Emanuele Della Valle, and Michael Grossniklaus. C-SPARQL: A Continuous Query Language for RDF Data Streams. In *ISWC*, 2009.
- Tim Berners-Lee, James Hendler, and Ora Lassila. The Semantic Web. *Scientific American*, 284(5): 34–43, 2001.

- Alexander Boldachev. Subject-Event Ontology Without Global Time: Foundations and Execution Semantics. In *arXiv preprint arXiv:2510.18040*, 2025.
- Scott A. Crosby and Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. *USENIX Security*, 2009.
- M. Denker et al. Sigstore: Software signing for everybody. In *ACM CCS Workshop on Software Supply Chain Offensive Research*, 2022. URL <https://www.sigstore.dev>.
- Jean-Sébastien Dessureault, Alain-Thierry Iliho Manzi, Soukaina Alaoui Ismaili, Khadim Lo, Mireille Lalancette, and Éric Bélanger. COMPASS: Multidimensional value alignment for agent orchestration. *arXiv preprint arXiv:2603.11277*, 2026. URL <https://arxiv.org/abs/2603.11277>.
- Dimitrios Doumanas, Georgios Bouchouras, Andreas Soularidis, Konstantinos Kotis, and George A. Vouros. From Human- to LLM-Centered Collaborative Ontology Engineering. *Applied Ontology*, 2025. doi: 10.3233/AO-240023.
- Kit Fine. Ontological Dependence. *Proceedings of the Aristotelian Society*, 95:269–290, 1995.
- Foam Contributors. Foam: A Personal Knowledge Management and Sharing System for VSCode. <https://github.com/foambubble/foam>, 2020. Open-source VSCode extension for personal knowledge graphs. Accessed: 2025-01-15.
- Aldo Gangemi, Nicola Guarino, Claudio Masolo, Alessandro Oltramari, and Luc Schneider. Sweetening Ontologies with DOLCE. *Knowledge Engineering and Knowledge Management: Ontologies and the Semantic Web*, pages 166–181, 2002.
- Suyash Gaurav, Jukka Heikkonen, and Jatin Chaudhary. Governance-as-a-service: Explicit governance graphs for multi-agent systems. *arXiv preprint arXiv:2508.18765*, 2025. URL <https://arxiv.org/abs/2508.18765>.
- Birte Glimm, Ian Horrocks, Boris Motik, Giorgos Stoilos, and Zhe Wang. HermiT: An OWL 2 Reasoner. *Journal of Automated Reasoning*, 53(3):245–269, 2014. ISSN 0168-7433. doi: 10.1007/s10817-014-9305-1.
- Paul Groth et al. Nanopublications: A growing resource of provenance-centric scientific claims. *Journal of Biomedical Semantics*, 14(1):1–15, 2023.
- Jonathan Gruss, Andrei Ciortea, Guido Salvaneschi, and Simon Mayer. Real-Time Collaboration in Linked Data Systems. In *Proceedings of the ISWC 2023 Posters, Demos and Industry Tracks*, volume 3632 of *CEUR Workshop Proceedings*, 2023. Applies CRDTs to RDF resource editing in Solid Pods.
- Nicola Guarino and Christopher A. Welty. An overview of ontoclean. *The Handbook on Ontologies*, pages 151–172, 2009. doi: 10.1007/978-3-540-92673-3_7.
- Ramanathan V. Guha, Dan Brickley, and Steve Macbeth. Schema.org: Evolution of Structured Data on the Web. *Communications of the ACM*, 59(2):44–51, 2016. ISSN 0001-0782. doi: 10.1145/2844544.
- Giancarlo Guizzardi. *Ontological Foundations for Structural Conceptual Models*. PhD thesis, University of Twente, 2005.

- Giancarlo Guizzardi et al. UFO-B: Executable Event-Driven Foundational Ontology Specifications. In *ER 2024*, 2024.
- Yaroslav O. Halchenko, Kyle Meyer, Benjamin Poldrack, Debanjum S. Solanky, Alexander S. Wagner, Jason Gors, David MacFarlane, Doreen Pustina, Vanessa Sochat, Satrajit S. Ghosh, Christian Mönch, Christopher J. Markiewicz, Laura Waite, Ilya Shlyakhter, Alejandro de la Vega, Soichi Hayashi, Christian O. Häusler, Jean-Baptiste Poline, Tobias Kadelka, et al. DataLad: Distributed System for Joint Management of Code, Data, and Their Relationship. *Journal of Open Source Software*, 6(63):3262, 2021. doi: 10.21105/joss.03262.
- Ahmad Hemid, Waleed Shabbir, Abderrahmane Khiat, Christoph Lange-Bever, Christoph Quix, and Stefan Decker. OntoEditor: Real-Time Collaboration via Distributed Version Control for Ontology Development. In *The Semantic Web: ESWC 2024 Satellite Events*, volume 14664 of *Lecture Notes in Computer Science*, pages 326–341. Springer, 2024. doi: 10.1007/978-3-031-60626-7_18. Uses Operational Transformation (OT), not CRDTs.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. *Knowledge Graphs*. Number 22 in Synthesis Lectures on Data, Semantics, and Knowledge. Springer, 2021. ISBN 9783031007903. doi: 10.2200/S01125ED1V01Y202109DSK022.
- ICOM-CIDOC. CIDOC Conceptual Reference Model. ISO 21127:2014, 2014.
- Rafflesia Khan, Declan Joyce, and Mansura Habiba. AGENTS SAFE: A Unified Framework for Ethical Assurance and Governance in Agentic AI. *arXiv preprint arXiv:2512.03180*, 2025. Unified governance: design-time + runtime + audit.
- Tobias Kuhn and Michel Dumontier. Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data. *Journal of Biomedical Semantics*, 6, 2015.
- Tobias Kuhn, Christine Chichester, Michael Krauthammer, and Michel Dumontier. Nanopublications: A Growing Resource of Provenance-Centric Scientific Linked Data. In *IEEE eScience*, 2015.
- Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, IETF, 2013. URL <https://tools.ietf.org/html/rfc6962>.
- Danh Le-Phuoc, Minh Dao-Tran, Josiane Xavier Parreira, and Manfred Hauswirth. CQELS: A Native and Adaptive Approach for Unified Querying Linked Data Streams. In *ISWC*, pages 694–709, 2011.
- Frank Li. Openclaw PRISM: A zero-fork, defense-in-depth runtime security layer for tool-augmented LLM agents. *arXiv preprint arXiv:2603.11853*, 2026. URL <https://arxiv.org/abs/2603.11853>.
- Kevin Lin. Dendron: A Hierarchical Tool for Thought. <https://wiki.dendron.so/>, 2020. Open-source knowledge management tool for VSCode. Accessed: 2025-01-15.
- Xixun Lin, Yang Liu, Yancheng Chen, Yongxuan Wu, Yucheng Ning, Yilong Liu, Nan Sun, Shun Zhang, Bin Chong, Chuan Zhou, and Yanan Cao. Safeharness: Lifecycle-integrated security architecture for LLM-based agent deployment. *arXiv preprint arXiv:2604.13630*, 2026. URL <https://arxiv.org/abs/2604.13630>.

- Hailin Liu, Eugene Ilyushin, Jie Ni, and Min Zhu. SafeAgent: A Runtime Protection Architecture for Agentic Systems. *arXiv preprint arXiv:2604.17562*, 2026. Protocol-level safety for LLM agents.
- Martin Kleppmann Martin et al. Automerge: A JSON CRDT for Real-Time Collaborative Editing. *ACM SIGOPS Operating Systems Review*, 2020.
- Carlo Mazzocca, Abbas Acar, Selcuk Uluagac, Rebecca Montanari, Paolo Bellavista, and Mauro Conti. A survey on decentralized identifiers and verifiable credentials. *arXiv preprint arXiv:2402.02455*, 2024. URL <https://arxiv.org/abs/2402.02455>.
- Kevin Nicolai and Peter Dadam. Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types. *Proceedings of the ACM on Human-Computer Interaction*, 2021.
- Abel Nieto et al. Formally Verified CRDTs in Iris Separation Logic. In *POPL 2024*, 2024.
- Obsidian Team. Obsidian: A Knowledge Base on Top of a Local Folder of Plain Text Markdown Files. <https://obsidian.md/>, 2024. Accessed: 2025-01-15.
- Palantir Technologies. Palantir Foundry Data Engine. <https://www.palantir.com/platforms/foundry/>, 2024.
- Erik Pautsch et al. Agenthub: A registry for discoverable, verifiable, and reproducible AI agents. *arXiv preprint arXiv:2510.03495*, 2025. URL <https://arxiv.org/abs/2510.03495>.
- Chandra Prakash, Mary Lind, and Avneesh Sisodia. UALM: Fleet governance for healthcare agent systems. *arXiv preprint arXiv:2601.15630*, 2026. URL <https://arxiv.org/abs/2601.15630>.
- Kolawole Quadri. KYA: A Framework-Agnostic Trust Layer for Autonomous Systems with Verifiable Provenance and Hierarchical Policy Composition. *arXiv preprint arXiv:2605.25376*, 2026. Framework-agnostic trust layer with HMAC chain, 1,800 ops/sec, 15+ adapters, veldt-ky on PyPI.
- Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-Free Replicated Data Types. *Lecture Notes in Computer Science*, 6976, 2011.
- Ryan Shaw, Raphael Troncy, and Lynda Hardman. LOD: Linking Open Descriptions of Events. In *The Semantic Web: Research and Applications*, pages 153–167, 2011.
- Barry Smith, Michael Ashburner, Cornelius Rosse, Jonathan Bard, William Bug, Werner Ceusters, Louis J. Goldberg, Karen Eilbeck, Amelia Ireland, Christopher J. Mungall, et al. The OBO Foundry: Coordinated Evolution of Ontologies to Support Biomedical Data Integration. *Nature Biotechnology*, 25(11):1251–1255, 2007. doi: 10.1038/nbt1346.
- Barry Smith, Werner Ceusters, Bert Klagges, Jacob Koehler, Anand Kumar, Jane Lomax, Chris Mungall, Fabian Neuhaus, Alan L. Rector, and Cornelius Rosse. The Information Artifact Ontology (IAO). *BMC Bioinformatics*, 11(1):83, 2010. doi: 10.1186/1471-2105-11-83.
- Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, Björn Grüning, Marco La Rosa, Simone Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, RO-Crate Community, Paul Groth, and Carole Goble. Packaging Research Artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. doi: 10.3233/DS-210053.

- Andreas Soularidis, Dimitrios Doumanas, Konstantinos Kotis, and George A. Vouros. Automating Agentic Collaborative Ontology Engineering with Role-Playing Simulation of LLM-Powered Agents and RAG Technology. *The Knowledge Engineering Review*, 40, 2025. doi: 10.1017/S026988892510009X.
- Manu Sporny, Gregg Kellogg, and Markus Lanthaler. JSON-LD 1.1: A JSON-based Serialization for Linked Data. W3C, 2020.
- Marcantonio Bracale Syrnikov, Federico Pierucci, Marcello Galisai, Matteo Prandi, Piercosma Biscconti, Francesco Giarrusso, Olga Sorokoletova, Vincenzo Suriani, and Daniele Nardi. Institutional AI: Governance graphs and policy engines for agent ecosystems. *arXiv preprint arXiv:2601.11369*, 2026. URL <https://arxiv.org/abs/2601.11369>.
- Ahmed Talukder, Maruf Ahmed Mridul, and Oshani Seneviratne. Towards Automated Ontology Generation from Unstructured Text: A Multi-Agent LLM Approach. *arXiv preprint arXiv:2604.23090*, 2026. Multi-agent LLM OE: Domain Expert→Manager→Coder→QA workflow.
- Google Open Source Security Team. Supply chain levels for software artifacts (slsa). Technical report, OpenSSF, 2023. URL <https://slsa.dev>.
- TerminusDB/DFRNT. TerminusDB: A Git-like Knowledge Graph Database. <https://terminusdb.org/>, 2024. Open-source graph database with immutable, version-controlled RDF storage. Accessed: 2025-01-15.
- Santiago Torres-Arias et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, 2019. URL <https://in-toto.io>.
- Thanh Luong Tuan and Abhijit Sanyal. Ontology-Constrained Neural Reasoning in Enterprise Agentic Systems: A Neurosymbolic Architecture for Domain-Grounded AI Agents. *arXiv preprint arXiv:2604.00555*, 2026. Foundation AgenticOS (FAOS) platform; three-layer ontology (Role, Domain, Interaction) for grounding enterprise LLM agents.
- Willem Robert van Hage, Veronique Malaise, Gerben de Vries, Guus Schreiber, and Maarten van Someren. The Simple Event Model (SEM). *The Semantic Web: Research and Applications*, pages 381–395, 2011.
- W3C. PROV-O: The PROV Ontology. <https://www.w3.org/TR/prov-o/>, 2013.
- W3C. SHACL Shapes Constraint Language. <https://www.w3.org/TR/shacl/>, 2017.
- W3C. RDF Dataset Canonicalization (URDNA2015). <https://www.w3.org/TR/rdf-canon/>, 2023.
- W3C OWL Working Group. OWL 2 Web Ontology Language: Document Overview. In *W3C Recommendation*, 2012.
- W3C RDF-star Working Group. RDF 1.2: RDF-star and SPARQL-star. W3C Working Draft, 2024.
- W3C RDF Stream Processing Community Group. RDF Stream Processing: Requirements and Design Rationale. W3C, 2013.
- W3C Verifiable Credentials Working Group. Verifiable Credential Data Integrity 1.0. W3C Candidate Recommendation, 2024.

Yiqi Wang, Jiaqi Zhang, Taotao Cai, Zirui Liu, Qingqiang Sun, Zequn Sun, Zhangkai Wu, Mingkai Zhang, and Yanming Zhu. From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents. *arXiv preprint arXiv:2606.04990*, 2026. Unified provenance framework; explicitly states unified trace schema is still needed.

Ludwig Wittgenstein. *Tractatus Logico-Philosophicus*. Routledge & Kegan Paul, 1922.

Greg Young. Event Sourcing: State from Events. *CodeBetter*, 2010.

Ziling Zhou. Cryptographic binding and reproducibility verification for AI agent tool use. *arXiv preprint arXiv:2603.14332*, 2026. URL <https://arxiv.org/abs/2603.14332>.